Large Language Models

A Complete Self-Study & Exam Preparation E-Book

Covering Weeks 1 – 12

From Attention to Agents

Sequence models → Transformers → BERT/GPT → Decoding → Tokenizers → Scaling → PEFT & Instruction Tuning → Long-context attention → Multimodal LLMs

Made by Anmol Kansal with AI

Course reference: Prof. Mitesh M. Khapra (AI4Bharat, IIT Madras)

May 21, 2026

How to use this book

This book is built to *replace* the original lecture slides and transcripts. Each chapter follows the same rhythm:

1. **Big picture first** — what problem does the chapter solve?
2. **Build intuition** — analogies and toy examples.
3. **Math derivations** — rigorous, with every symbol explained.
4. **Architecture walkthrough** — with shapes, dimensions, and TikZ diagrams.
5. **Worked examples** — numerical / NLP / LLM.
6. **Boxes** — definitions, intuition, exam tips, common mistakes, mnemonics.
7. **Quick revision & practice questions** — one-page summary + MCQs + numericals.

Reading the boxes:

- **Definition** — formal statement to memorise.
- **Intuition** — the "feel" behind the math.
- **Exam Tip** — likely quiz/exam point.
- **Formula Insight** — why the formula looks the way it does.
- **Common Mistake / Quiz Trap** — where students slip.
- **Research/Industry Insight** — depth for interviews & research.

Contents

How to use this book **1**

Part I — Complete Notes **3**

1 Week 1 — From Sequence Models to Transformers: Multi-Headed Self-Attention & Cross-Attention **3**

2 Week 2 — Completing the Transformer: Decoder, Masked & Cross-Attention, Positional Encoding, LayerNorm, Residuals, Training **20**

3 Week 3 — Language Modelling, GPT, and the Pre-train / Fine-tune Paradigm **33**

4 Week 4 — Decoding Strategies & BERT (Bidirectional Encoder Representations) **44**

5 Week 5 — Tokenizers: BPE, WordPiece, and SentencePiece **59**

6 Week 7 — Encoder-Decoder LLMs: BART, GPT-2, T5 and the Choices that Affect Performance **71**

7 Week 8 — The LLM Landscape, Scaling Laws, and Data Pipelines **85**

8 Week 11 — Long Sequences, Complexity, and Fast Attention **98**

9 Week 12 — Position Encodings: APE, RPE, RoPE, ALiBi, NoPE, and Length Generalization **113**

Closing Remarks **127**

Part I — Complete Notes (Weeks 1–12)

1 Week 1 — From Sequence Models to Transformers: Multi-Headed Self-Attention & Cross-Attention

Chapter goals. By the end of this chapter you will be able to:
1. Explain *why* we moved from RNN seq2seq to Transformers.
2. Derive scaled dot-product self-attention from first principles.
3. Understand the role of the query, key, value matrices (W_Q, W_K, W_V).
4. Justify and compute multi-head attention.
5. Trace the data flow through one encoder layer, count its parameters, and explain every dimension.

1.1 Setting the Stage: What is a Sequence Model?

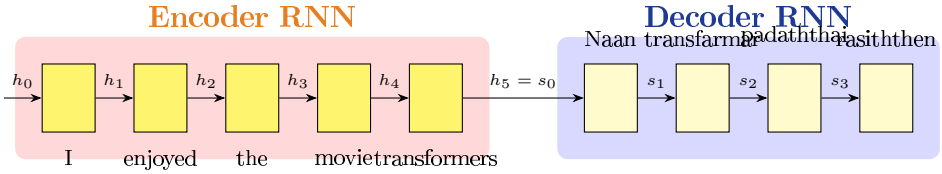
A *sequence model* processes an ordered list of tokens $x_1, x_2, \ldots, x_T$ and produces either a single output (e.g. a sentiment label) or another ordered list of tokens $y_1, y_2, \ldots, y_{T'}$ (e.g. a translation). Most early NLP successes used *recurrent neural networks* (RNNs) — including vanilla RNNs, LSTMs, and GRUs — to perform this mapping.

Definition

A **Sequence-to-Sequence (seq2seq) model** consists of two RNNs:
- An **Encoder** that reads the source sequence and compresses it into a fixed-length vector (the *context* or *thought* vector).
- A **Decoder** that, conditioned on this context vector, generates the target sequence one token at a time.

Introduced by Sutskever et al. (2014) and Cho et al. (2014).

1.1.1 The Encoder-Decoder Picture



The final encoder state h_5 is called the **context vector**, **thought vector**, or **annotation**. It is supposed to summarise the *entire* input sentence into a single vector — a heavy demand for long sentences.

1.1.2 The Two Critical Limitations of RNN-Based seq2seq

1. **The information bottleneck.** A single fixed-size vector h_T has to encode the whole sentence. The longer the sentence, the more crushing the bottleneck. Empirically, BLEU scores degrade sharply once sentence length exceeds ~ 30 tokens.
2. **No parallelism along the time axis.** To compute h_t we need h_{t-1}, which needs h_{t-2}, etc. This makes training painfully slow on modern GPUs, which prefer wide parallel matrix multiplies. Vanishing and exploding gradients only worsen the situation.

Intuition

Imagine you must summarise a 1000-word essay in a 5-word tweet, hand the tweet to a friend, and ask them to reconstruct the essay sentence-by-sentence. That's the seq2seq decoder's job using only h_T. Painful, right? **Attention** fixes this by giving the decoder *the entire essay*, not just the tweet — and letting the decoder *re-read* the most relevant parts for each output word.

1.2 The Attention Mechanism — A Quick Tour (Bahdanau, 2015)

The cure for the bottleneck is conceptually simple:

Intuition

Don't crush all encoder states into one vector. Keep them all, and at every decoder step, blend them with weights α_{ti} that depend on what the decoder needs *right now*.

Formally, for decoder time-step t, define a context vector

$$\boxed{\mathbf{c}_t = \sum_{i=1}^{n} \alpha_{ti}\, \mathbf{h}_i} \tag{1}$$

where n is the input length, $\mathbf{h}_i$ is the i-th encoder hidden state, and α_{ti} is the attention weight on input position i when producing output y_t. The weights are computed by softmax-normalising an *alignment score*:

$$\alpha_{ti} \;=\; \frac{\exp\big(\text{score}(\mathbf{s}_{t-1}, \mathbf{h}_i)\big)}{\sum_{i'=1}^{n} \exp\big(\text{score}(\mathbf{s}_{t-1}, \mathbf{h}_{i'})\big)}. \tag{2}$$

A common Bahdanau-style scoring function uses a tiny feed-forward network:

$$\text{score}(\mathbf{s}_{t-1}, \mathbf{h}_j) \;=\; V_a^{\top} \tanh\big(U_{att}\, \mathbf{s}_{t-1} + W_{att}\, \mathbf{h}_j\big). \tag{3}$$

Formula Insight

Equation (3) has three vectors — the decoder state s, the encoder state h, and the score vector v — combined by two linear transformations (U_{att}, W_{att}), one non-linearity (tanh), and a final dot product with V_a. The output is a single scalar: "how relevant is encoder position j for predicting the t-th output token?"

1.2.1 Decoding step-by-step with attention

At decoder step t, the recipe is:

1. Use the previous decoder state $\mathbf{s}_{t-1}$ to compute scores against *every* encoder state $\mathbf{h}_1, \ldots, \mathbf{h}_n$.
2. Softmax-normalise to get $\alpha_{t,1}, \ldots, \alpha_{t,n}$.
3. Form $\mathbf{c}_t = \sum_i \alpha_{ti} \mathbf{h}_i$.
4. Feed $\mathbf{c}_t$ together with the previous decoder hidden state and the previously emitted token into the decoder RNN to compute $\mathbf{s}_t$ and emit y_t.

1.2.2 What does the matrix α_{ti} look like?

Stacked over all output positions, α is a soft alignment matrix: high values are roughly diagonal (input/output positions align), but they pick up real linguistic facts — e.g. when translating English → Tamil, "transformers" aligns with the Tamil word at a non-monotonic position because of word-order differences. This was the breakthrough of Bahdanau et al. (2015): attention *discovers* alignments without supervision.

1.2.3 A crucial observation: attention parallelises

Look again at Eq. (2). At time t, $\mathbf{s}_{t-1}$ is fixed, and $\mathbf{h}_1, \ldots, \mathbf{h}_n$ are all known. Therefore $\alpha_{t1}, \ldots, \alpha_{tn}$ can be computed *in parallel*. So can $\mathbf{c}_t$. That is, attention is intrinsically a parallel operation.

Remember This

The big takeaway from Bahdanau attention: the attention layer itself is parallelisable. The bottleneck is only the RNN that produces $\mathbf{h}_i$ and $\mathbf{s}_t$ — *not* the attention block.
This question motivates: *What if we replace the RNN with something that is also parallelisable?* The answer is the Transformer.

1.3 The Wishlist that Births the Transformer

Summarising the limitations and desired properties, the "wishlist" is:

- Keep attention (it solved the bottleneck).
- Eliminate recurrence (to allow parallel training).
- Avoid vanishing / exploding gradients of deep RNNs.
- Allow each token to look at every other token in $O(1)$ path length (instead of $O(T)$ in RNNs).

The 2017 paper of Vaswani et al. — *Attention is All You Need* — delivers exactly this. The model is called the **Transformer**.

Transformer at a Glance

- Encoder: `Word Embedding` → `Self-Attention` → `FFN` (stacked $N = 6$ times).
- Decoder: `Self-Attention` → `Encoder-Decoder Attention` → `FFN` (stacked $N = 6$ times).
- All computation is matrix multiplication and softmax. No recurrence.

1.4 Self-Attention — The Heart of the Transformer

The key new idea is to replace the RNN encoder by a layer that lets every word attend to every other word *in the same sentence*. Because the words are attending to themselves (the same sequence), this is called **self-attention**.

1.4.1 Why "self"? A motivating example

Consider two sentences:

*"The animal didn't cross the street because **it** was too tired"*
*"The animal didn't cross the street because **it** was congested"*

In the first, "**it**" refers to "**animal**". In the second, "**it**" refers to "**street**". *Resolving the referent* requires looking at *other words in the same sentence.* Self-attention is the mechanism that computes, for every word, a contextualised representation by weighting all the other words.

Intuition

For each word in the sentence, self-attention asks: "Which other words in this same sentence should I pay attention to, and how much?" — and produces a new vector that is a weighted mix of all the words' value vectors.

1.4.2 The Goal Stated as a Table

Imagine an 8×8 table where row i holds the attention scores of word i towards every word j. In the running example, the row of *"it"* should have a large value in the column *"animal"* (or *"street"* depending on context). Filling this table for all rows is what self-attention does.

Importantly, all the input vectors (word embeddings) $h_1, \ldots, h_T$ are *available from the start* — unlike the seq2seq decoder where the decoder state is computed one step at a time. This means **all rows of the attention table can be computed simultaneously.**

1.5 Q, K, V — The Three Roles a Word Plays

Recall the Bahdanau score (Eq. 3) needs three vectors: (s, h, v) where s is the decoder state (the "query"), h is the encoder state (the "key"), and v is the same encoder state re-used as the "value".

In self-attention, we have only *one* input: the word embedding vector h_j. So how do we produce three different vectors per word?

Solution: Three Linear Projections

For each word embedding $h_j \in \mathbb{R}^{d_{\text{model}}}$ we produce three vectors by multiplying with three learnable matrices:

$$\begin{aligned} q_j &= W_Q\, h_j \quad \text{(query)} \\ k_j &= W_K\, h_j \quad \text{(key)} \\ v_j &= W_V\, h_j \quad \text{(value)} \end{aligned}$$

W_Q, W_K, W_V are learned during training (one set per attention head).

Intuition

A useful library analogy: in a library catalog system,
- the **query** is the search term you type;
- the **key** is the catalog index (each book has a key);
- the **value** is the actual content of the book.

The librarian (softmax) returns a blended summary of the values, weighted by how well their keys match your query.

1.5.1 Computing the first output z_1

1. Form q_1 from h_1 and form k_j, v_j for every $j = 1, \ldots, T$.
2. Compute raw scores $e_{1j} = q_1 \cdot k_j$ for $j = 1, \ldots, T$.
3. Softmax over j: $\alpha_{1j} = \text{softmax}_j(e_{1j})$.

4. Output $z_1 = \sum_{j=1}^{T} \alpha_{1j}\, v_j$.

For the second output z_2, we use q_2 instead of q_1; every other step is identical and runs *in parallel* for all i.

Formula Insight

Why the dot product? The dot product $q_i \cdot k_j$ is a natural similarity measure between two vectors of the same dimension. The bigger the dot product, the more aligned (similar) q_i and k_j are, hence the more attention word i pays to word j.

1.6 Vectorising into Matrices: One Equation Per Encoder Layer

Let $H \in \mathbb{R}^{d_{\text{model}} \times T}$ stack all the input word embeddings as columns. Form

$$Q = W_Q H \in \mathbb{R}^{d_q \times T}, \tag{4}$$

$$K = W_K H \in \mathbb{R}^{d_k \times T}, \tag{5}$$

$$V = W_V H \in \mathbb{R}^{d_v \times T}. \tag{6}$$

By convention $d_q = d_k = d_v = d_k$ (the paper uses $d_k = 64$ per head).

1.6.1 Scaled dot-product attention

$$\boxed{Z = \text{softmax}\left(\frac{Q^\top K}{\sqrt{d_k}}\right) V^\top} \tag{7}$$

Let's verify dimensions step-by-step (using the column-vector convention):

- $Q^\top \in \mathbb{R}^{T \times d_k}$ and $K \in \mathbb{R}^{d_k \times T}$, so $Q^\top K \in \mathbb{R}^{T \times T}$ — one similarity score per (query, key) pair.
- Dividing by $\sqrt{d_k}$ leaves the shape unchanged.
- Row-wise softmax produces a $T \times T$ matrix whose rows sum to one — these are the attention weights.
- $V^\top \in \mathbb{R}^{T \times d_v}$, so $Z \in \mathbb{R}^{T \times d_v}$ — one output vector per query.

Remember This

The **Vaswani** paper uses row vectors ($X \in \mathbb{R}^{T \times d}$), giving the more common form

$$Z = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

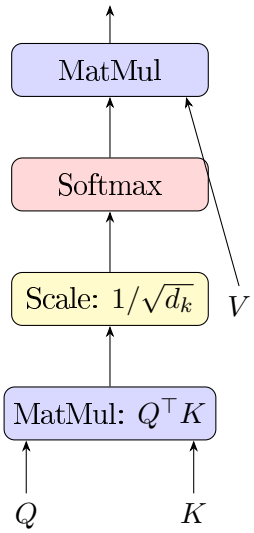
The two formulations are transposes of each other — conceptually identical.

1.6.2 Why divide by $\sqrt{d_k}$?

Formula Insight

For random query and key vectors with components drawn i.i.d. with zero mean and unit variance, $q \cdot k = \sum_{m=1}^{d_k} q_m k_m$ has mean 0 and variance d_k. Large variance pushes the softmax into saturating regions where gradients vanish. Dividing by $\sqrt{d_k}$ standardises the dot product to unit variance, keeping the softmax in a healthy regime.

1.6.3 Scaled Dot-Product Attention block (visual)



1.7 Multi-Head Attention — Why One Head Is Not Enough

Empirically, a *single* attention head over-mixes information: it tends to average out distinct relationships (subject–verb agreement, coreference, syntactic role, etc.) into a single soup. A CNN solves this by using *many filters* per layer to learn different feature detectors. The Transformer copies this idea.

Definition

Multi-Head Attention runs h scaled dot-product attentions in parallel, each with its own learned projections $\{W_Q^i, W_K^i, W_V^i\}_{i=1}^h$. The outputs of the heads are *concatenated* along the feature dimension and projected by a final linear W^O:

$$\text{MultiHead}(H) = W^O \,\text{Concat}\big(\text{head}_1, \text{head}_2, \ldots, \text{head}_h\big), \qquad \text{head}_i = \text{Attention}(W_Q^i H,\, W_K^i H,\, W_V^i H).$$

1.7.1 Why concatenation works without blowing up dimensions

In the original paper, $d_{\text{model}} = 512$ and $h = 8$, so each head uses $d_k = d_q = d_v = 512/8 = 64$. Concatenating 8 heads of width 64 gives back 512. The final $W^O \in \mathbb{R}^{512\times512}$ mixes information from all heads.

Intuition

Different heads end up specialising in different patterns: one head may track syntactic dependencies (subject ↔ verb), another may track coreference (it ↔ animal), another may track positional neighbours. The lecturer showed visualisations: a single-head model links "it" mainly to "was" (a syntactic neighbour), whereas a two-head model also links "it" to "animal" (semantic coreference) — two different patterns, two heads.

1.7.2 Dimensions cheat-sheet (per encoder layer)

Object	Shape	Comment
Input X	$T \times 512$	sequence of word embeddings
W_Q^i, W_K^i, W_V^i	512×64	per head, per role
Q_i, K_i, V_i	$T \times 64$	per head outputs
Attention matrix	$T \times T$	softmax over rows
Head output	$T \times 64$	per head
Concat	$T \times 512$	all 8 heads stitched
W^O	512×512	final projection
MHA output	$T \times 512$	ready for the FFN

Quiz Trap

Do not confuse d_k (size of each head, e.g. 64) with d_{model} (size of the embedding, e.g. 512). They satisfy $h \cdot d_k = d_{\text{model}}$ in the standard Transformer.

1.8 The Position-wise Feed-Forward Network (FFN)

After multi-head attention, every token's vector passes through a small *point-wise* feed-forward network — the same network applied independently to each position (i.e. a 1×1 convolution if you prefer that lens):

$$\text{FFN}(z) = \max\big(0,\, W_1 z + b_1\big)\, W_2 + b_2. \tag{8}$$

In the original paper, $W_1 \in \mathbb{R}^{512\times 2048}$ (expand!), and $W_2 \in \mathbb{R}^{2048\times 512}$ (project back). The hidden width $l = 2048$ gives the layer expressive power to mix the attention-aggregated information.

Intuition

Think of MHA as "gathering information across the sentence" and the FFN as "thinking about each position independently with the information so far". The two alternate, and stacking $N = 6$ such pairs in the encoder gradually builds a deep contextual representation.

1.9 Counting Parameters in One Encoder Layer

Let $d_{\text{model}} = 512$, $h = 8$, $d_k = d_q = d_v = 64$, FFN width $l = 2048$.

1.9.1 Multi-head attention parameters

- Per head: three projection matrices of shape 64×512.
  Parameters per head $= 3 \cdot 64 \cdot 512 = 98{,}304$.
- For 8 heads: $8 \cdot 98{,}304 = 786{,}432$.
- Output projection $W^O \in \mathbb{R}^{512\times 512}$: $512 \cdot 512 = 262{,}144$.
- **MHA total** $\approx 1.05 \times 10^6$.

1.9.2 FFN parameters

$$512 \cdot 2048 + 2048 + 2048 \cdot 512 + 512 \;\approx\; 2.1 \times 10^6.$$

1.9.3 Per-layer total and stack total

- Per layer: ≈ 3 million parameters.
- Stack of $N = 6$ layers: ≈ 18 million parameters in the encoder.

Exam Tip

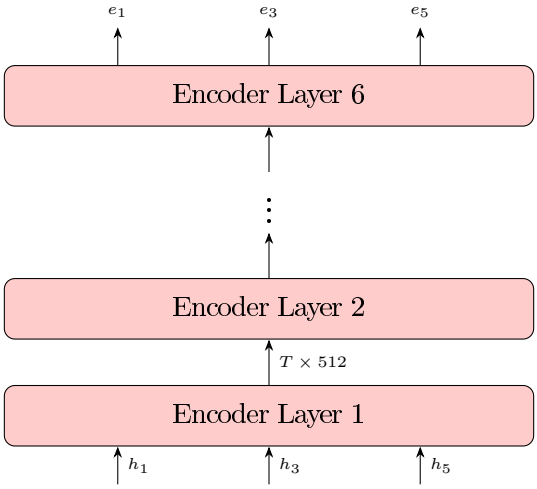
A very frequent quiz question is: *"How many parameters are in one encoder layer of the Transformer (base configuration)?"* The standard answer is "**about 3 million**", distributed as ≈ 1 M in MHA and ≈ 2 M in the FFN.

1.10 The Encoder Stack and Data Flow

The encoder consists of $N = 6$ *identical* layers stacked vertically. Each layer reads $T \times 512$ and outputs $T \times 512$, so we can stack arbitrarily many. Crucially:

- Computation is **parallelised across the sequence dimension** (all T token outputs of a layer are produced in one matrix multiplication).
- Computation is **not** parallel across layers: layer 2 needs the output of layer 1.

We denote the encoder's final output sequence as $e_1, e_2, \ldots, e_T$. These will be fed to the decoder's encoder-decoder (cross-)attention block — studied next week.



1.11 What We Have *Not* Yet Discussed

To keep Week 1 focused, the following items are deferred to later weeks (but are part of the full Transformer):

- **Positional encodings** (Week 2) — self-attention is order-blind!
- **Residual connections and layer normalisation** (Week 2).
- **Masked self-attention in the decoder, and cross-attention** (Week 2).
- **Final softmax over the vocabulary and training objective** (Week 2).

1.12 Practical Applications of What We Learned So Far

Even the encoder alone — with multi-head self-attention and FFNs — is the workhorse behind:

- **BERT** and family (text understanding, embeddings used in search).
- **Sentence-transformers** (semantic search, RAG retrieval).
- **ViT, Swin** (vision transformers — self-attention on image patches).
- **Encoder of T5 / BART** (used in summarisation and translation).

1.13 Common Mistakes & Quiz Traps

Common Mistake

Mistake 1. "Attention is just for the decoder."
Fix: *Self-attention* is used inside both the encoder and the decoder, plus *cross-attention* links them. The attention idea is universal in transformers.

Common Mistake

Mistake 2. "W_Q, W_K, W_V are the same matrix."
Fix: They are three *different* learnable matrices. Sharing them turns the model into something else (e.g. Reformer-LSH attention has shared $Q = K$ by construction; standard Transformer does not).

Common Mistake

Mistake 3. "Self-attention is $O(T)$ per token."
Fix: It is $O(T)$ per *token's* computation but $O(T^2 d)$ for the layer because every query attends to every key. This quadratic cost is the central pain point we will attack in Week 11.

Quiz Trap

Trap: "Multi-head attention is more parameters than single-head attention of the same d_{model}."
Reality: *No.* With h heads of width $d_k = d_{\text{model}}/h$, the total parameter count of the W_Q, W_K, W_V matrices is the same as a single-head attention with $d_k = d_{\text{model}}$. The only extra cost is the projection W^O. Multi-head "free lunch" = more expressive at (almost) the same parameter budget.

1.14 Memory Tricks

Memory Trick

"Q is the question, K is the catalog, V is the content."
A library analogy: I bring a *query* (Q). Books on the shelf have a *key* (K) summarising them. The actual book content is the *value* (V). The librarian (softmax) returns a weighted mix of contents.

Memory Trick

"Scale to keep softmax sober."
Without the $1/\sqrt{d_k}$ scaling, large dot products push softmax into saturation, killing gradients. Scaling keeps softmax "sober" — in a non-saturated, well-behaved regime.

Memory Trick

"MHA = many filters of self-attention."
Just as a CNN learns many filters to detect different features, MHA learns many heads to detect different relations.

1.15 Comparison Table: RNN/LSTM vs Attention vs Transformer Encoder

Property	Vanilla RNN/LSTM	Bahdanau attention (RNN + attention)	Transformer encoder
Sequence modelling unit	recurrent cell	recurrent cell	self-attention + FFN
Path length between distant tokens	$O(T)$	$O(T)$ (still goes through RNN)	$O(1)$
Parallelism over t	no	RNN no, attention yes	yes (whole layer)
Long-range modelling	poor (gradient decay)	better	strong
Quadratic cost in T	no	no (linear in T)	yes ($O(T^2 d)$)
Parameters per layer (base config)	$O(d^2)$	$O(d^2)$	$\sim$ 3M ($d = 512$)
Best for	short sequences	medium sequences (legacy MT)	most modern tasks

1.16 Quick Revision — Week 1 in One Page

Quick Revision

Concept tree.

- **seq2seq:** encoder RNN $\rightarrow$ context $\rightarrow$ decoder RNN. Bottleneck and not parallelisable.
- **Attention (Bahdanau):** $\mathbf{c}_t = \sum_i \alpha_{ti} \mathbf{h}_i$ with softmax-normalised scores. Fixes the bottleneck and is itself parallelisable.
- **Self-attention:** attention with s_i, h_j both coming from the same sequence. Q, K, V from three linear projections of the same input.
- **Scaled dot-product attention:** $Z = \text{softmax}(Q^\top K/\sqrt{d_k})\, V^\top$.
- **Multi-head attention:** $h = 8$ heads, each $d_k = 64$, concatenate $\rightarrow W^O$.
- **Encoder layer:** MHA $\rightarrow$ FFN; each layer outputs $T \times 512$.
- **Stack:** $N = 6$ encoder layers $\Rightarrow \approx$ 18M parameters.

Must-remember formulas.

$$Z = \text{softmax}\left(\frac{Q^\top K}{\sqrt{d_k}}\right) V^\top,$$

$$\text{FFN}(z) = \max(0, W_1 z + b_1)\, W_2 + b_2.$$

Why Transformer beats RNN. (i) Path length $O(1)$ between any two positions; (ii) full parallelism across the sequence; (iii) no vanishing/ exploding gradients along time; (iv) easier to scale to billions of parameters.

1.17 Practice Questions — Week 1

Multiple Choice

1. **(Q1)** The reason for dividing $QK^\top$ by $\sqrt{d_k}$ in scaled dot-product attention is to:

(A) reduce memory usage,

(B) prevent the softmax from saturating when d_k is large,

(C) normalise the output of attention to have unit norm,

(D) match the dimensionality of V.

Solution. Recall that for random q, k with i.i.d. unit-variance components, $\mathrm{Var}(q \cdot k) = d_k$. Large variance pushes the softmax inputs into extreme regions where one entry dominates and gradients vanish. Dividing by $\sqrt{d_k}$ rescales the variance to 1. **Answer: (B).** (A) is wrong — it does not change memory; (C) is wrong — attention outputs are not unit-norm; (D) is wrong — the scaling is along the score axis, not the value axis.

2. **(Q2)** In multi-head attention with $h = 8$ and $d_{\text{model}} = 512$, the per-head dimension d_k is:

   (A) 8

   (B) 64

   (C) 256

   (D) 512

   Solution. $d_k = d_{\text{model}}/h = 512/8 = 64$. **Answer: (B).**

3. **(Q3)** Compared to single-head attention with $d_k = d_{\text{model}} = 512$, multi-head attention with $h = 8, d_k = 64$ has:

   (A) more W_Q, W_K, W_V parameters in total,

   (B) fewer W_Q, W_K, W_V parameters in total,

   (C) the same total number of W_Q, W_K, W_V parameters,

   (D) zero additional parameters.

   Solution. Single-head: each W has shape 512×512. Multi-head: 8 matrices of shape 64×512 each, total $8 \cdot 64 \cdot 512 = 512^2$ — the same. The only extra cost is W^O. **Answer: (C).**

4. **(Q4)** Which property is NOT shared by vanilla RNN seq2seq and the Transformer encoder?

   (A) Can process variable-length input.

   (B) Produces a fixed-size representation per token.

   (C) Parallelises computation across positions of one sequence.

   (D) Uses learnable parameters.

   Solution. The Transformer encoder parallelises across positions; the RNN does not. **Answer: (C).**

Conceptual / Short Answer

5. **(Q5)** In one or two sentences, explain why a single context vector h_T is a bottleneck for long sentences in seq2seq.

   Answer. Compressing arbitrary-length source information into a single fixed-size vector loses information that scales with sentence length; the decoder cannot recover details it never sees. Empirically, BLEU degrades sharply beyond ~ 30 tokens. Attention solves this by exposing all encoder states $h_1, \ldots, h_n$ to the decoder.

6. **(Q6)** Why are W_Q, W_K, W_V separate matrices instead of one?

   Answer. The same word plays three roles — as a question (query), as an addressable index (key), and as content (value). Forcing all three to coincide would constrain the geometry of the attention space. Separate matrices let the model learn three different subspaces, each tuned for one role.

7. **(Q7)** Show that the attention weights for output position t can be computed in parallel for all input positions i.

   Answer. The score $\text{score}(s_{t-1}, h_i)$ depends only on s_{t-1} (fixed at step t) and h_i (precomputed by the encoder). Therefore all i-indexed scores are independent and can be evaluated as one matrix multiplication, followed by a row-wise softmax.

8. **(Q8) Tensor-shape tracing.** Let $T = 10, d_{\text{model}} = 512, h = 8$. Give the shapes of (a) the per-head Q_i, K_i, V_i; (b) the per-head attention matrix α_i; (c) the per-head output; (d) the concatenated output; (e) the final layer output.

   Answer. (a) 10×64; (b) 10×10; (c) 10×64; (d) 10×512 (concat of 8 heads); (e) 10×512 after W^O then FFN.

Numerical / Derivation

9. **(Q9)** Compute the number of learnable parameters in a single encoder *layer* (one MHA + one FFN, $d_{\text{model}} = 512$, $h = 8, d_k = 64$, FFN width $l = 2048$). Ignore biases for MHA but include biases for FFN.

   Solution.

$$\begin{aligned}
\text{MHA: } & 3 \cdot (h \cdot d_k) \cdot d_{\text{model}} + d_{\text{model}}^2 \\
&= 3 \cdot 512 \cdot 512 + 512^2 \\
&= 4 \cdot 512^2 \;=\; 1{,}048{,}576 \\
\text{FFN: } & 512 \cdot 2048 + 2048 + 2048 \cdot 512 + 512 \\
&\approx 2.10 \times 10^6 \\
\text{Per layer: } & \;\approx\; 3.15 \times 10^6.
\end{aligned}$$

10. **(Q10)** Suppose $d_k = 4$ and a single query $q = [1, 0, 1, 0]$ attends to two keys $k_1 = [1, 0, 1, 0]$, $k_2 = [0, 1, 0, 1]$ with values $v_1 = [2, 0]$, $v_2 = [0, 2]$. Compute the output of scaled dot-product attention (single head).

    Solution. $q \cdot k_1 = 2$, $q \cdot k_2 = 0$. Scaled: $2/\sqrt{4} = 1$ and $0/2 = 0$. Softmax: $\alpha = (e/e+1, 1/e+1) \approx (0.731, 0.269)$. Output $z = 0.731 \cdot (2, 0) + 0.269 \cdot (0, 2) \approx (1.462, 0.538)$.

11. **(Q11)** Derive the softmax of a vector $x = (x_1, x_2)$ and show that adding a constant c to both entries does not change the result. (Hint: this is why we can subtract $\max(x)$ in the safe-softmax trick.)

    Solution. $\text{softmax}(x_1 + c, x_2 + c)_i = \frac{e^{x_i + c}}{e^{x_1 + c} + e^{x_2 + c}} = \frac{e^c e^{x_i}}{e^c (e^{x_1} + e^{x_2})} = \text{softmax}(x)_i$. So softmax is translation-invariant — we exploit this in the safe-softmax used in Flash Attention (Week 11).

1.18 Week 1 Addendum — Deeper Dives

This addendum revisits four topics that deserve more space than the main Week 1 narrative gave them. They are flagged in the supplementary lecture material (*Counting Encoder Parameters* and *MultiHeadAttention* handouts).

1.18.1 1A — A Fully Worked Multi-Head Attention Example (by hand)

Intuition

Equations are easy to nod along to. Until you grind through a 2-token, 2-head example by hand, multi-head attention does not really click. We will use *small numbers* specifically so we can verify every step.

Setup.

- Sequence length $t = 2$ (two tokens).
- Number of heads $h = 2$.
- Embedding dimension $d_{\text{model}} = 4$.
- Per-head $d_k = d_q = d_{\text{model}}/h = 2$.
- Per-head value size $d_v = 2$.
- Input $X \in \mathbb{R}^{d_{\text{model}} \times t} = \mathbb{R}^{4\times 2}$:

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 0 \\ 0 & 2 \end{bmatrix}.$$

(Each column is one token; this convention follows the lecturer's slides.)

Head 1 weights (we deliberately set $W_Q^1 = W_K^1 = W_V^1$ to keep the arithmetic transparent):

$$W_Q^1 = W_K^1 = W_V^1 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \end{bmatrix} \in \mathbb{R}^{d_q \times d_{\text{model}}} = \mathbb{R}^{2\times 4}.$$

Head 2 weights:

$$W_Q^2 = W_K^2 = W_V^2 = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 1 & 1 & 2 & 1 \end{bmatrix}.$$

Output projection $W^O \in \mathbb{R}^{d_{\text{model}} \times (h \cdot d_v)} = \mathbb{R}^{4\times 4}$:

$$W^O = \begin{bmatrix} 0.5 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Step 1 — Head 1: compute Q^1, K^1, V^1.

$$Q^1 = K^1 = V^1 = W_Q^1 X = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 4 & 8 \end{bmatrix}.$$

Step 2 — Head 1: scaled dot product. With $\sqrt{d_k} = \sqrt{2}$:

$$\frac{(Q^1)^\top K^1}{\sqrt{d_k}} = \frac{1}{\sqrt{2}} \begin{bmatrix} 2 & 4 \\ 4 & 8 \end{bmatrix} \begin{bmatrix} 2 & 4 \\ 4 & 8 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 20 & 40 \\ 40 & 80 \end{bmatrix} \approx \begin{bmatrix} 14.14 & 28.28 \\ 28.28 & 56.56 \end{bmatrix}.$$

Step 3 — Head 1: row-wise softmax. For row 1, $e^{14.14}/e^{14.14}+e^{28.28} \approx 0$ and the other entry ≈ 1. Same pattern for row 2:

$$\text{softmax}_{\text{row}}\left(\frac{(Q^1)^\top K^1}{\sqrt{d_k}}\right) \approx \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}.$$

Note that one of the entries is essentially 0 and the other essentially 1 — the softmax saturated because the score gap (14 vs 28) is huge. This is exactly the saturation problem that motivates the $1/\sqrt{d_k}$ scaling — and here, even with scaling, the toy weights are extreme enough to saturate it.

Step 4 — Head 1: multiply by V.

$$\text{head}_1 = \text{softmax}(\ldots)\,(V^1)^\top = \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}\begin{bmatrix} 2 & 4 \\ 4 & 8 \end{bmatrix} = \begin{bmatrix} 4 & 8 \\ 4 & 8 \end{bmatrix} \in \mathbb{R}^{t\times d_v} = \mathbb{R}^{2\times 2}.$$

Both tokens end up with the same head-1 output because both rows of attention weights were essentially identical $(0, 1)$.

Step 5 — Head 2: same procedure.

$$Q^2 = K^2 = V^2 = W_Q^2 X = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 1 & 1 & 2 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 3 & 4 \end{bmatrix}.$$

After scaled dot product and softmax (you can verify):

$$\text{softmax}\left(\frac{(Q^2)^\top K^2}{\sqrt{d_k}}\right) \approx \begin{bmatrix} 0.67 & 0.33 \\ 0.06 & 0.94 \end{bmatrix}.$$

This time the softmax did *not* saturate because the score differences are small.

$$\text{head}_2 \approx \begin{bmatrix} 0.67 & 0.33 \\ 0.06 & 0.94 \end{bmatrix}\begin{bmatrix} 2 & 0 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 1.34 & 1.33 \\ 0.11 & 3.94 \end{bmatrix}.$$

(Slight differences from the slide are due to rounding.)

Step 6 — Concatenate heads. Concatenate along the feature axis ($Z \in \mathbb{R}^{t\times(h\cdot d_v)} = \mathbb{R}^{2\times 4}$):

$$Z = [\text{head}_1 \mid \text{head}_2] = \begin{bmatrix} 4 & 8 & 1.34 & 1.33 \\ 4 & 8 & 0.11 & 3.94 \end{bmatrix}.$$

Step 7 — Apply W^O to get the final MHA output. The lecturer writes $\text{MHA} = W^O Z^\top$ to keep tokens as columns (output $\in \mathbb{R}^{d_{\text{model}}\times t} = \mathbb{R}^{4\times 2}$):

$$\text{MHA} = W^O Z^\top = \begin{bmatrix} 0.5 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} 4 & 4 \\ 8 & 8 \\ 1.34 & 0.11 \\ 1.33 & 3.94 \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 4 & 4 \\ 1.34 & 0.11 \\ 1.33 & 3.94 \end{bmatrix}.$$

The dimensions check out: 4×2 — one column per token, $d_{\text{model}} = 4$ features per column.

Remember This

What this exercise teaches.

1. Each head independently produces a $t \times d_v$ matrix — here 2×2.
2. Concatenation gives a wide $t \times (h \cdot d_v)$ matrix — here 2×4.
3. W^O projects from concatenated heads back to d_{model} — here it maps 4-d $\to$ 4-d. In the base Transformer, it maps 512-d $\to$ 512-d.
4. Different heads can produce different attention patterns: head 1 was saturated (each token mostly attends to token 2), head 2 was soft (mix of both tokens).
5. Even with very small numerical examples, the saturation effect of softmax is visible — this is why we scale by $\sqrt{d_k}$ and why initialisation matters.

1.18.2 1B — The Role of W^O (Why We Need It)

Intuition

After concatenating h heads of width d_v, you have a wide vector of $h \cdot d_v$ features — but the rest of the Transformer expects vectors of width d_{model}. If $h \cdot d_v = d_{\text{model}}$ (the standard case), W^O is "just" a $d_{\text{model}} \times d_{\text{model}}$ linear map. So why bother?

Reason 1: dimensional matching. In the general case, the h heads can have different d_{v_i} values, so the concatenation is $\mathbb{R}^{d_{v_1}+d_{v_2}+\cdots+d_{v_h}}$. Then $W^O \in \mathbb{R}^{d_{\text{model}} \times \sum_i d_{v_i}}$ maps to the canonical width needed by the next sub-layer.

Reason 2: mixing across heads. Without W^O, the h heads would write into h disjoint blocks of the output vector — they could never interact. W^O lets the model *learn* how to combine information from different heads on a per-feature basis. This is what makes "many heads" more than just "many parallel attentions glued together."

Reason 3: optional dimensionality change. You could choose $h \cdot d_v \neq d_{\text{model}}$ (with appropriate W^O). Some architectures use this freedom, although base Transformer keeps the two equal.

Formula Insight

Putting it all together, the per-head and overall MHA equations are:

$$\text{head}_i = \text{Attention}\big(W_Q^i X,\ W_K^i X,\ W_V^i X\big)$$
$$\text{MHA}(X) = W^O\ \text{Concat}\big(\text{head}_1, \text{head}_2, \ldots, \text{head}_h\big).$$

The W^O is the only *cross-head* learnable interaction in MHA.

1.18.3 1C — Alignment Diagonal, Diagonality, and Word Order

In slide 11, the lecturer showed a 4-output $\times$ 5-input attention heatmap for an English $\to$ Tamil translation. Four entries were highlighted:

- α_{11} — "**Naan**" (output 1) aligned with "**I**" (input 1). On the diagonal.
- α_{25} — "**transfarmar**" (output 2) aligned with "**transformers**" (input 5). *Off-diagonal* — Tamil moves the noun.
- α_{34} — "**padaththai**" aligned with "**film**".
- α_{42} — "**rasiththen**" aligned with "**enjoyed**". *Off-diagonal.*

Intuition

For monotonically aligned language pairs (English-French), attention is roughly diagonal (output t attends to input t). For free-word-order languages (English-Tamil, English-Japanese), attention develops non-monotonic patterns. Critically, the model *learns* this without any explicit alignment supervision — this is the breakthrough of the Bahdanau paper (*Neural Machine Translation by Jointly Learning to Align and Translate*, 2015).

Research / Industry Insight

Visualising attention has become a standard interpretability tool. Tools like BertViz, exBERT, and the Tensor2Tensor visualiser let you inspect attention heads layer-by-layer. Several heads in BERT correspond to interpretable linguistic relations: "next token", "previous token", "coreference", "direct object", etc. This was popularised by Clark et al. (2019) in *What Does BERT Look At?*.

1.18.4 1D — The Detailed Parameter Counting Walkthrough

In the main Week 1 narrative, we said "$\sim$ 3 million parameters per encoder layer". The supplementary "Counting Encoder Parameters" slide does it in painstaking detail for $T = 40, h = 8, d_k = d_q = d_v = 64, d_{\text{model}} = 512$, FFN width $l = 2048$.

Per-head W_Q, W_K, W_V:

$$\begin{aligned} W_Q &: 64 \times 512 = 32{,}768 \quad \text{(per head; the slide rounds to 32,178 — a typo)} \\ W_K &: 64 \times 512 = 32{,}768 \\ W_V &: 64 \times 512 = 32{,}768. \end{aligned}$$

All 8 heads combined:

$$\begin{aligned} 8 \text{ heads of } W_Q &: 8 \cdot 64 \cdot 512 = 262{,}144 \\ 8 \text{ heads of } W_K &: 262{,}144 \\ 8 \text{ heads of } W_V &: 262{,}144 \\ \text{Subtotal} &: 786{,}432. \end{aligned}$$

Output projection:

$$W^O : 512 \times 512 = 262{,}144.$$

Total MHA parameters per layer:

$$786{,}432 + 262{,}144 = 1{,}048{,}576 \approx 1.05 \times 10^6 \approx 1\,\text{M}.$$

FFN parameters (with biases):

$$\underbrace{512 \cdot 2048}_{W_1} + \underbrace{2048}_{b_1} + \underbrace{2048 \cdot 512}_{W_2} + \underbrace{512}_{b_2} = 1{,}048{,}576 + 2048 + 1{,}048{,}576 + 512 \approx 2.1 \times 10^6 \approx 2\,\text{M}.$$

Total per encoder layer: $\approx 1.05\text{M} + 2.1\text{M} \approx 3.15\,\text{M}$.

Total encoder stack ($N = 6$): $\approx 6 \times 3.15\text{M} = 18.9\,\text{M}$.

Exam Tip

The Khapra mnemonic: "MHA is 1, FFN is 2, layer is 3."
One million parameters in MHA, two million in FFN, three million per encoder layer. For $N = 6$ layers: **18 million encoder parameters.**

Quiz Trap

Trap. "Increasing the number of heads h (with $d_k = d_{\text{model}}/h$) increases the parameter count."
Reality. No. The total parameter count of W_Q, W_K, W_V is $3 \cdot (h \cdot d_k) \cdot d_{\text{model}} = 3 \cdot d_{\text{model}}^2$ regardless of h. Doubling h halves d_k at no parameter cost. Only W^O remains fixed at d_{model}^2. The constraint $h \cdot d_k = d_{\text{model}}$ is what makes this work.

1.18.5 1E — Extra Practice Questions for the Addendum

1. **(A1)** In the worked 2-head example above, why do both tokens get the same output from head 1 but different outputs from head 2?

   Answer. In head 1, both rows of the softmax were essentially $(0, 1)$ — both queries gave nearly identical attention patterns due to large dot products and softmax saturation. So both tokens attend almost entirely to token 2's value, producing the same head-1 output. In head 2, the weights are softer (e.g., row 1 is $(0.67, 0.33)$), giving distinct outputs per token.

2. **(A2)** Suppose you remove W^O from multi-head attention. What is lost?

   Answer. Without W^O: (i) different heads occupy disjoint slices of the output, never interacting; (ii) the output dimension is fixed at $h \cdot d_v$ without the ability to project to a different size; (iii) the model loses the only cross-head mixing parameter. Empirically this dramatically hurts performance.

3. **(A3)** If you double d_{model} from 512 to 1024 (and keep $h = 8$, so $d_k = 128$), by what factor do MHA parameters change per layer?

   Solution. MHA parameters scale as $4\, d_{\text{model}}^2$ (three W_Q, W_K, W_V totalling $3d^2$, plus one W^O of d^2). Doubling d quadruples the count: from $\sim$ 1M to $\sim$ 4M. *Lesson:* MHA cost grows quadratically in d_{model} — this is why scaling embeddings is expensive.

4. **(A4)** For an encoder with $N = 12$ layers, $d_{\text{model}} = 768$, $h = 12$, FFN width $l = 3072$ (**BERT-base** config), estimate the total encoder parameters.

   Solution.

   - MHA per layer: $4 \cdot 768^2 \approx 2.36$M.
   - FFN per layer: $2 \cdot 768 \cdot 3072 + 3072 + 768 \approx 4.72$M.
   - Per layer total: ≈ 7M.
   - $N = 12$ layers: ≈ 85M parameters.

   With embeddings ($\sim$ 24M for $|V| = 30{,}000$), BERT-base has the well-known ≈ 110M total parameters. *Quick interview answer: BERT-base* $\sim 110M$.

5. **(A5)** Why is the attention matrix *row-stochastic* (each row sums to 1) but *not* column-stochastic?

   Answer. Softmax is applied along the row direction (over keys, for each query). Each query's attention weights sum to 1 — it must put 100% of its attention somewhere. But the same key can be attended to by many queries, so the column sum is unconstrained.

2 Week 2 — Completing the Transformer: Decoder, Masked & Cross-Attention, Positional Encoding, LayerNorm, Residuals, Training

Chapter goals. By the end of this chapter you will be able to:

1. Explain the three sub-layers of the Transformer decoder and the role of each.
2. Understand *teacher forcing* and why it makes parallel training possible.
3. Derive and implement *masked* self-attention using $-\infty$ masking.
4. Explain encoder–decoder (*cross*) attention and contrast it with self-attention.
5. Derive the sinusoidal positional encoding and explain its design choices.
6. Distinguish *Batch Normalization* from *Layer Normalization* and explain why Transformers use the latter.
7. Trace the residual "Add & Norm" wiring, count the decoder parameters, and assemble the full Transformer architecture.

2.1 Where We Are and Where We Are Going

In Week 1 we built the encoder: word embeddings → multi-head self-attention → feed-forward → stack of $N = 6$. The encoder produces a sequence of context-aware vectors $e_1, \ldots, e_T$, each in $\mathbb{R}^{512}$. Now we ask:

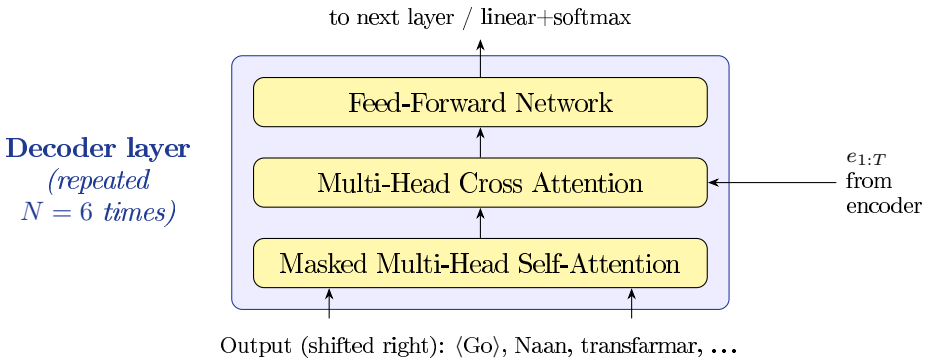
Intuition

How does the decoder use these encoder outputs to actually *generate* a target sentence one word at a time, in parallel during training and autoregressively during inference?

The decoder differs from the encoder in three important ways:

1. It has **three** sub-layers (masked self-attention, cross-attention, FFN) instead of two.
2. Its self-attention is **masked** so that position t cannot peek at positions $> t$ during training.
3. It uses **cross-attention**: its queries come from the decoder, but the keys and values come from the encoder. This is how the encoder's information flows into the decoder.

2.2 Decoder Stack at a Glance



The decoder layer is wrapped (just like the encoder) with residual connections and layer normalisation around each sub-layer; we will study those shortly.

2.3 Teacher Forcing — Why the Decoder Sees the Target

If you look at the decoder during training, it receives the *ground-truth* target sentence (shifted right) as input. *Why*?

2.3.1 The naive alternative: free running

We could feed the decoder its own previous prediction at every step. If at step 1 the decoder predicts "**D**" (wrong, instead of "Naan"), then at step 2 it conditions on "**D**" and most likely emits another wrong word, and the error compounds. This is called *exposure bias* or *error accumulation*.

2.3.2 The fix: Teacher Forcing

Definition

Teacher Forcing feeds the *ground-truth* previous token to the decoder at each step during training, regardless of what the decoder predicted. The decoder is "forced" to follow the teacher's example word for word.

Intuition

You are learning to write a translation. Two ways:

- **Free running:** You write word by word, never seeing the correct answer until the end. One early mistake derails the whole sentence.
- **Teacher forcing:** After each word, the teacher tells you the correct previous word. Even if you slipped, the next word is conditioned on the right history. You learn faster and more stably.

Remember This

Teacher forcing enables **parallel training** of the decoder.
Because the inputs at all positions are the ground-truth tokens (known in advance), the decoder can compute all output positions in *one* forward pass — not sequentially. This is critical: without teacher forcing, training a Transformer would be as slow as an RNN.

Common Mistake

Mistake: "Teacher forcing is used at inference too."
Fix: No. At inference, there *is* no ground-truth. The decoder runs autoregressively: it predicts y_1, feeds it back, predicts y_2, etc. The training/inference mismatch caused by teacher forcing is one motivation behind techniques like *scheduled sampling*.

2.4 Masked Self-Attention — Forbidding the Future

Here is the problem with putting the full target sentence into the decoder. If at decoder position 2 we let position 2 attend to position 3, the model can simply copy the future word and "cheat" — learning nothing.

Definition

Masked self-attention prevents position t from attending to any position $t' > t$ by setting those positions' attention scores to $-\infty$ *before* the softmax. After softmax, $\exp(-\infty) = 0$, so the corresponding weights are exactly zero.

2.4.1 The mask matrix

Let the unmasked attention score matrix be $A = Q^\top K/\sqrt{d_k} \in \mathbb{R}^{T\times T}$ (row i = scores from query q_i to all keys). Define the mask matrix

$$M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases}. \tag{9}$$

Then masked attention is

$$\boxed{Z = \text{softmax}(A + M)\, V.} \tag{10}$$

Explicitly, for $T = 5$:

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty & -\infty \\ 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

The lower triangle (including the diagonal) is 0 — those positions are visible. The upper triangle is $-\infty$ — those positions are forbidden.

Formula Insight

Adding $-\infty$ before softmax is the cleanest way to "delete" those entries: the softmax denominator naturally re-normalises over only the visible positions. Numerically, in PyTorch one usually adds a very large negative number like -10^9 rather than literal $-\infty$ to avoid `NaN` from $0 \cdot \infty$.

Quiz Trap

Trap: "Masking can be applied to the value vectors V."
Reality: Masking is applied to the *scores*, before softmax. Masking V would not zero out the attention; it would alter the values, which is wrong.

2.4.2 Encoder also uses masking — for padding

The encoder's self-attention is not causally masked (every token can see every other token). But in practice it *does* use a different mask: the **padding mask**. When sentences in a batch have different lengths, shorter ones are padded with a special `[PAD]` token, and the mask prevents real tokens from attending to padding.

2.4.3 What happens at inference?

The decoder cannot use teacher forcing because the ground truth is unknown. Instead:

1. Feed ⟨Go⟩ as input. Predict y_1.
2. Feed ⟨Go⟩, y_1. Predict y_2.
3. Continue until ⟨EOS⟩ or max length.

The triangular mask is still useful even here, because batched implementations process multiple beams simultaneously.

2.5 Multi-Head Cross Attention — Where Encoder Meets Decoder

In the encoder, self-attention's Q, K, V all come from the same sentence (source). In the decoder's masked self-attention, Q, K, V all come from the (shifted) target. In the **cross-attention** sub-layer:

Definition

Cross-Attention (encoder–decoder attention) computes:

- **Queries** from the decoder's previous sub-layer output $S = (s_1, \ldots, s_T)$: $Q_2 = W_{Q_2} S$.
- **Keys and Values** from the top encoder layer's output $E = (e_1, \ldots, e_T)$: $K_2 = W_{K_2} E$, $V_2 = W_{V_2} E$.

The attention itself is the same scaled dot-product:

$$Z = \text{softmax}\left(\frac{Q_2 K_2^\top}{\sqrt{d_k}}\right) V_2.$$

No causal mask is needed (the decoder can look at any encoder position — the source sentence is fully available).

Intuition

Cross-attention is the spiritual heir of Bahdanau attention from Week 1. The decoder *queries* the encoder's representations and pulls in relevant context for the next target word. "Which source word should I look at to translate the next target word?"

2.5.1 Why use the top encoder's output for *all* decoder layers?

In the original Transformer, every decoder layer's cross-attention reads from the *same* encoder output (the top layer). This keeps the architecture simple. Variants (e.g., some recent multi-source models) allow different decoder layers to look at different encoder layers.

2.5.2 Self vs Cross Attention — a comparison table

	Self-Attention	**Cross-Attention**
Source of Q	same sequence	decoder sub-layer output
Source of K, V	same sequence	encoder output
Used in	encoder & decoder	decoder only
Causal mask?	decoder yes; encoder no	no (source is fully visible)
Padding mask?	yes	yes (pad source positions)
Role	contextualise within a sentence	condition target on source

2.6 Decoder Output: Linear + Softmax over Vocabulary

The top decoder layer produces a sequence of vectors $o_{1:T} \in \mathbb{R}^{T \times 512}$. To turn each into a probability over the vocabulary V, we apply a linear projection $W_D \in \mathbb{R}^{512 \times |V|}$ followed by softmax:

$$p(y_t = w \mid y_{<t}, x) = \text{softmax}(o_t W_D)_w. \tag{11}$$

2.6.1 Why W_D is parameter-heavy

Suppose $|V| = 37{,}000$ (a typical BPE vocabulary). Then W_D has $512 \cdot 37{,}000 \approx 1.9 \times 10^7 \approx 19$ million parameters. This *single matrix* accounts for ≈ 19 out of the ≈ 65 million parameters of the base Transformer (i.e., about 30% of the total!).

Research / Industry Insight

This observation motivates **weight tying**: share parameters between the input embedding matrix and $W_D^\top$ (since both map between $\mathbb{R}^{512}$ and $\mathbb{R}^{|V|}$). Weight tying cuts parameters and often improves BLEU. It is standard in GPT-2, GPT-3, and modern LLMs.

2.7 Counting Decoder Parameters Per Layer

Per decoder layer (base config, $d = 512, h = 8, l = 2048$):

- Masked MHA: $\approx$ 1M (same as encoder MHA).
- Cross-attention MHA: $\approx$ 1M.
- FFN: $\approx$ 2M.
- **Per-layer total:** $\approx$ 4M (vs $\approx$ 3M per encoder layer).

With $N = 6$ layers, decoder $\approx$ 24M. Add encoder $\approx$ 18M, plus embeddings and W_D, total $\approx$ 65M parameters — this is the famous Transformer "base" size.

Exam Tip

Frequently asked: per-layer parameter count.
- Encoder layer: $\approx$ 3M (MHA 1M + FFN 2M).
- Decoder layer: $\approx$ 4M (masked MHA 1M + cross-attn 1M + FFN 2M).

2.8 Positional Encoding — Reintroducing Order

2.8.1 Why we need it at all

Intuition

Self-attention treats the input as a *set* of vectors, not a sequence. If you shuffle the input tokens, the output set is the same (just shuffled). That is, self-attention is *permutation equivariant*. But "**dog bites man**" and "**man bites dog**" have very different meanings! We must inject position information *somewhere*.

In RNNs, position was encoded implicitly in the hidden state (you must walk through positions in order). In Transformers, every token is processed simultaneously, so we add a *positional encoding* p_j to the word embedding h_j:

$$h'_j = h_j + p_j. \tag{12}$$

2.8.2 Candidate ideas (and why they fail)

- **Constant vector** $p_j = (j, j, \ldots, j)$**:** as j grows, p_j dominates the embedding h_j, drowning out semantics.
- **One-hot vectors over positions:** the Euclidean distance between any two one-hot vectors is always $\sqrt{2}$, so position 1 looks "equally far" from position 2 and from position 100. No notion of *relative* closeness.

- **Learned position embeddings:** works fine up to the trained max length, but cannot generalise to longer sequences. (BERT and GPT use this anyway, since in practice their max length is fixed.)

2.8.3 The sinusoidal solution

Vaswani et al. proposed:

$$\boxed{\begin{aligned} PE(j, 2i) &= \sin\left(\frac{j}{10000^{2i/d_{\text{model}}}}\right), \\ PE(j, 2i+1) &= \cos\left(\frac{j}{10000^{2i/d_{\text{model}}}}\right), \end{aligned}} \tag{13}$$

for position j and embedding-dimension index $i = 0, 1, \ldots, 255$ (since $d_{\text{model}} = 512$, there are 256 sine/cosine pairs).

Formula Insight

Reading the formula carefully:
- Each **dimension** i corresponds to a sinusoid of a different frequency.
- Lower dimensions (i small) have very short wavelength ($\approx 2\pi$), encoding fine-grained position.
- Higher dimensions (i near 255) have very long wavelength (up to $2\pi \cdot 10000$), encoding coarse-grained position.
- Even dims use sin, odd dims use cos: a sine/cosine pair lets the model recover relative position via the identity $\sin(a+b) = \sin a \cos b + \cos a \sin b$.

2.8.4 The two key properties the design satisfies

1. **Unique pattern per position.** Each p_j is a vector of 256 sine/cosine values at different frequencies — no two positions share the same vector.
2. **Distance encodes (roughly) position offset.** The Euclidean distance $\|p_j - p_{j+\delta}\|$ depends on δ (the relative offset) and is roughly monotonically increasing in $|\delta|$. By contrast, one-hot distance is constant $\sqrt{2}$ for all pairs.

The lecturer demonstrated this graphically: for one-hot, the distance from position 0 to positions 1, 2, …, 100 is a flat horizontal line at $\sqrt{2}$ (uninformative). For sinusoidal PE, the distance grows roughly like log-distance with position offset (informative).

2.8.5 Linear-combination property (relative position learnable)

A beautiful property of sinusoidal encoding:

$$PE(j + \delta) = R_\delta \cdot PE(j) \tag{14}$$

for some matrix R_δ that depends only on δ, not on j. This means the model can learn a linear function (in the W_Q, W_K projections) that lets attention weights depend on the *relative* position δ, not just absolute positions.

Research / Industry Insight

This linear-combination property inspires **relative positional encodings** (Shaw et al., 2018) used in T5 and **rotary positional embeddings** (RoPE, Su et al. 2021) used in LLaMA, PaLM, Mistral, and most modern LLMs. RoPE encodes position by *rotating* the query/key vectors instead of *adding* a positional vector.

2.8.6 Visualisation in your head

For each position j along the horizontal axis and each dimension i along the vertical axis, $PE(j, i)$ produces a striped pattern: bottom rows oscillate rapidly with j, top rows oscillate slowly. The matrix is the famous "positional encoding heatmap".

2.9 Training the Transformer — Residual Connections & Normalization

Counting layers loosely (each MHA + FFN block has ~ 3 "layers"), the full Transformer with $N = 6$ encoder and $N = 6$ decoder layers is effectively ~ 42 layers deep. Two questions become urgent:

1. How do we ensure gradients flow through this depth? → **Residual connections.**
2. How do we keep activations well-scaled so training converges fast? → **Layer normalization.**

2.9.1 Residual "skip" connections

Definition

A **residual connection** around a sublayer $\mathcal{F}$ adds the input to the sublayer output:

$$\text{out} = x + \mathcal{F}(x).$$

This was popularised by ResNet (He et al., 2016). In Transformers, it is applied around *every* sub-layer (each MHA, each FFN, each cross-attention).

Intuition

Without a skip path, the only way information from layer 1 reaches the output is through 40+ non-linear transformations — a long, easily-decaying chain. The skip path gives a "highway" for both forward information and backward gradients. The sublayer learns the *residual* (small correction) on top of the identity.

2.9.2 Batch Normalization (refresher)

Batch Norm normalises activations *across the batch dimension* for each feature independently. Given a batch of m activations for neuron i in layer l:

$$\mu_i = \frac{1}{m}\sum_{j=1}^{m} x_i^j, \tag{15}$$

$$\sigma_i^2 = \frac{1}{m}\sum_{j=1}^{m}(x_i^j - \mu_i)^2, \tag{16}$$

$$\hat{x}_i = \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}, \tag{17}$$

$$\hat{y}_i = \gamma\,\hat{x}_i + \beta. \quad \text{(learnable scale and shift)} \tag{18}$$

2.9.3 Why Batch Norm fails for Transformers

- **Tiny batches kill estimation:** for sequence-of-sequences workloads (long sentences, large model), batch size is often small. With $m = 1$, $\sigma_i = 0$ and BN becomes a no-op.
- **Variable sequence lengths:** different sentences have different lengths and different distributions of activations. Averaging across the batch is meaningless when the underlying populations differ.
- **Empirical evidence:** BN underperforms in NLP tasks. The PowerNorm paper (Shen et al., 2020) systematically studied this and proposed a fix.

2.9.4 Layer Normalization

Definition

Layer Normalization normalises activations *across the feature dimension*, independently for each sample (and each sequence position). For a vector $x \in \mathbb{R}^H$ (the hidden representation of one token):

$$\mu = \frac{1}{H}\sum_{i=1}^{H} x_i, \quad \sigma^2 = \frac{1}{H}\sum_{i=1}^{H}(x_i - \mu)^2,$$
$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad \hat{y}_i = \gamma_i \hat{x}_i + \beta_i.$$

Crucially, this normalisation **does not depend on the batch.** It works the same whether $m = 1$ or $m = 1024$, perfect for variable-length sequences.

2.9.5 BN vs LN — a comparison table

	Batch Normalization	**Layer Normalization**
Normalises over	batch (across m samples)	features (across H neurons)
Statistics depend on batch?	yes	no
Works with batch size 1?	no (variance is 0)	yes
Sensitive to sequence length?	yes (different lengths per sample)	no
Default in CNN/image	yes	no
Default in Transformer/NLP	no	yes
Learnable params per layer	$2H$ (γ, β)	$2H$ (γ, β)

2.9.6 Putting it together: "Add & Norm"

The complete sublayer recipe is:

$$\boxed{\text{LayerNorm}\big(x + \text{Sublayer}(x)\big).} \tag{19}$$

This is called the **Post-LN** (post-normalisation) configuration — the original Transformer. A variant called **Pre-LN** (used by GPT-2, BERT-large, and most modern LLMs) places LayerNorm *before* the sublayer:

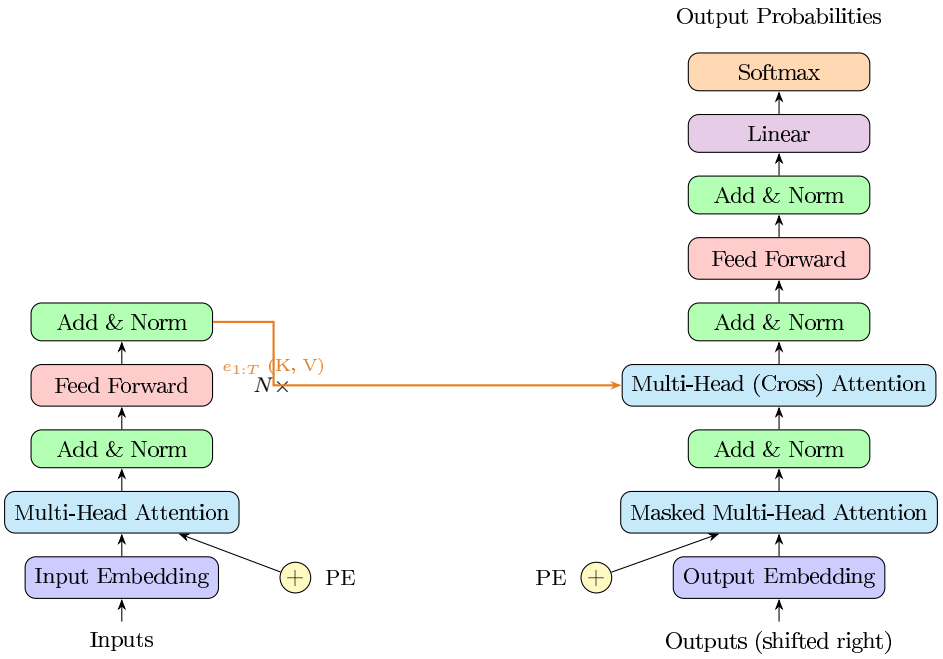
$$x + \text{Sublayer}(\text{LayerNorm}(x)). \tag{20}$$

Pre-LN is more stable to train and usually doesn't require a learning-rate warmup.

Research / Industry Insight

The original Post-LN Transformer requires careful warmup (linear ramp of learning rate for the first 4000 steps in the paper) to prevent gradient explosion. Pre-LN was found to be much more forgiving (Xiong et al., 2020). Today, most production LLMs use Pre-LN or its variants (e.g., RMSNorm in LLaMA).

2.10 Putting It All Together — The Full Transformer



2.10.1 Data flow in 6 steps

1. Source tokens → input embeddings $h_1, \ldots, h_T \in \mathbb{R}^{512}$.
2. Add positional encodings $h_j + p_j$.
3. Pass through $N = 6$ encoder layers → $e_1, \ldots, e_T$.
4. Target tokens (shifted right) → output embeddings, add PE, pass through masked self-attention.
5. Cross-attention: query from decoder, K and V from encoder output $e_{1:T}$.
6. FFN, $N = 6$ times, then linear projection to $\mathbb{R}^{|V|}$, softmax. Done.

2.11 Other Training Tricks (briefly mentioned in the paper)

- **Label smoothing.** Instead of using a one-hot target, use $\widetilde{y} = (1 - \varepsilon)e_w + \varepsilon/|V|$ for $\varepsilon \approx 0.1$. This penalises overconfident predictions and improves BLEU.
- **Learning-rate warmup.** The paper schedules $\eta_t = d_{\text{model}}^{-1/2} \cdot \min(t^{-1/2}, t \cdot t_{\text{warmup}}^{-3/2})$ with $t_{\text{warmup}} = 4000$. Without warmup, Post-LN training is unstable.

- **Dropout.** Applied to (i) attention weights, (ii) FFN, and (iii) the sum embedding+PE, with rate 0.1.
- **Adam optimizer** with $\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-9}$.

2.12 Common Mistakes & Quiz Traps — Week 2

Common Mistake

Mistake 1. "The decoder is just an encoder with a mask."
Fix: A decoder layer has *three* sub-layers (masked self-attention, cross-attention, FFN), not two. The middle cross-attention block is the crucial bridge to the encoder.

Common Mistake

Mistake 2. "Positional encoding is added to the output of self-attention."
Fix: It is added to the *input* embeddings (before the first attention). Adding it post-attention would not give attention any positional information to use.

Common Mistake

Mistake 3. "LayerNorm normalises across the batch."
Fix: LayerNorm normalises across the *feature* dimension (per token, per sample). Batch dimension is irrelevant. This is the whole point of LN vs BN.

Quiz Trap

Trap. "Cross-attention requires a causal mask because the future shouldn't leak."
Reality: The future leak we worry about is in the *target*, not the source. The source sentence is fully visible to the decoder — so cross-attention has *no* causal mask. Only padding masks (if applicable).

Quiz Trap

Trap. "Sinusoidal PE allows the model to extrapolate to any sentence length."
Reality: In theory yes, in practice the model has only seen positions up to its training max, and quality degrades beyond. RoPE and ALiBi handle extrapolation better.

2.13 Memory Tricks — Week 2

Memory Trick

"Decoder has Three: Mask, Cross, FFN."
The three decoder sub-layers in order: **M**asked self-attention → **C**ross-attention → **F**FN.

Memory Trick

"$-\infty$ before softmax = exact zero after softmax."
Masking is done *before* softmax by adding a $-\infty$ matrix to the scores. After softmax, those positions become exact 0.

Memory Trick

"BN normalises across rows, LN across columns" (if rows = batch, columns = features). LayerNorm wins when batch is small or sequence length varies — exactly the NLP setting.

Memory Trick

"Sin for even, Cos for odd", frequencies geometric from 1 to 1/10000. Adjacent dims = a sine/cosine pair = a complex exponential, which is what gives the relative-position property.

2.14 Practical Applications

- **Translation (Google Translate, NLLB, MarianMT):** the original use case.
- **Summarisation (T5, BART, Pegasus):** same encoder-decoder pattern, longer outputs.
- **Speech recognition (Whisper):** audio features as encoder input, transcription as decoder output.
- **Image captioning (BLIP, CoCa):** image patches encoded → text decoded.
- **Decoder-only LLMs (GPT, LLaMA):** drop the encoder entirely; rely on masked self-attention only. This is the dominant modern paradigm.

2.15 Quick Revision — Week 2 in One Page

Quick Revision

Decoder structure. Three sub-layers per layer, each wrapped with Add&Norm: masked self-attention → cross-attention → FFN. Stacked $N = 6$ times.
Teacher forcing. Feed ground-truth shifted-right target to decoder during training. Enables parallel training. Requires causal mask to avoid cheating.
Masked self-attention. $Z = \text{softmax}(A + M)\,V$ where $M_{ij} = -\infty$ for $j > i$. Upper triangle is $-\infty$, lower triangle (incl. diagonal) is 0.
Cross-attention. Q from decoder, K and V from *top* encoder layer. No causal mask.
Decoder output. Linear $W_D \in \mathbb{R}^{d\times|V|}$ + softmax. $\sim$ 19M parameters.
Positional encoding (sinusoidal). $PE(j, 2i) = \sin(j/10000^{2i/d})$, $PE(j, 2i+1) = \cos(j/10000^{2i/d})$. Added to input embeddings. Encodes both absolute and (via linear combinations) relative position.
Layer Normalization. Normalises across the feature dimension, per token: $\hat{x} = (x - \mu)/\sqrt{\sigma^2 + \epsilon}$, then $\gamma\hat{x} + \beta$. Works with batch size 1; LayerNorm is the right choice for NLP.
Residual connection. out = LayerNorm(x+Sublayer(x)). Allows gradient flow through deep stack.
Parameter counts (base config). Encoder layer $\approx$ 3M, decoder layer $\approx$ 4M, $W_D \approx$ 19M, total $\approx$ 65M.

2.16 Practice Questions — Week 2

Multiple Choice

1. **(Q1)** In the Transformer decoder, masked self-attention is achieved by:

   (A) Multiplying the value matrix by a triangular mask.

   (B) Adding $-\infty$ to the upper triangle of the score matrix, before softmax.

(C) Dropping out attention weights with probability 0.5.

(D) Using a different scaling factor for each position.

Solution. (B). Mask the *scores* pre-softmax. $\exp(-\infty) = 0$ in the denominator and numerator gives exact-0 weight on future positions.

2. **(Q2)** Cross-attention in the Transformer decoder uses:

   (A) Q, K, V all from the encoder.

   (B) Q, K, V all from the decoder.

   (C) Q from encoder; K, V from decoder.

   (D) Q from decoder; K, V from encoder.

   Solution. (D). The decoder asks a question; the encoder supplies the catalog and content.

3. **(Q3)** Which of the following is *not* a reason to use LayerNorm over BatchNorm in Transformers?

   (A) LayerNorm is independent of batch size.

   (B) LayerNorm handles variable sequence lengths better.

   (C) LayerNorm has fewer parameters than BatchNorm.

   (D) BatchNorm with small batches has noisy statistics.

   Solution. (C) is the odd one out — BN and LN both have $2H$ learnable parameters (γ, β).

4. **(Q4)** The sinusoidal positional encoding pairs sin and cos for adjacent dimensions *primarily* so that:

   (A) the embedding has zero mean,

   (B) relative position can be expressed as a linear transformation of absolute position,

   (C) the model can be trained with smaller batch sizes,

   (D) the maximum sentence length can be extended.

   Solution. (B). The pair acts like a complex exponential and rotates by a fixed amount when position shifts by δ, which is a linear map of the pair.

5. **(Q5)** Teacher forcing in Transformer training means:

   (A) The teacher penalises the model for wrong answers more strongly.

   (B) The decoder receives its own previous prediction as input.

   (C) The decoder receives the ground-truth previous token as input.

   (D) The encoder forces specific attention patterns.

   Solution. (C).

Conceptual / Short Answer

6. **(Q6)** Why must the decoder's self-attention be masked but the encoder's not?

   Answer. The encoder reads a fully-given source sentence; all positions are known. The decoder must *generate* the target one step at a time, so position t must not see positions $> t$

during training, or it would trivially "cheat" by copying the future target token. Masking enforces causality.

7. **(Q7)** Compute the maximum and minimum value of any element of the sinusoidal PE.

   Answer. Each entry is a single sin or cos value, so it lies in $[-1, 1]$. This bounded range is one reason PE can be *added* to embeddings without overwhelming them (unlike a raw position scalar that grows unboundedly).

8. **(Q8)** Show the dimensions inside one cross-attention sub-layer when $T_{\text{src}} = T_{\text{tgt}} = T$, $d = 512$, $h = 8$.

   Answer. Per head: $Q_2 \in \mathbb{R}^{T\times 64}$ (from decoder), $K_2, V_2 \in \mathbb{R}^{T\times 64}$ (from encoder). Attention matrix: $Q_2 K_2^\top \in \mathbb{R}^{T\times T}$. Head output: $T \times 64$. Concat heads: $T \times 512$. Output $W^O \in \mathbb{R}^{512\times 512}$ produces $T \times 512$.

9. **(Q9)** A student says "LayerNorm should use the same γ, β for all tokens in a sequence." Correct or not?

   Answer. Correct. LayerNorm's $\gamma, \beta \in \mathbb{R}^H$ are per-feature parameters, applied identically to every token in the sequence (and every example in the batch). Each token is normalised independently, but the affine transform afterwards is shared.

Numerical / Derivation

10. **(Q10)** Given attention scores $a = (2, 1, 5, 3)$ and mask $m = (0, 0, -\infty, -\infty)$, compute the softmax outputs.

    Solution. Effective scores: $(2, 1, -\infty, -\infty)$. exp: $(e^2, e^1, 0, 0) \approx (7.389, 2.718, 0, 0)$, sum ≈ 10.107. softmax $\approx (0.731, 0.269, 0, 0)$. Position 3 and 4 are exactly zero, position 1 gets the largest weight as before.

11. **(Q11)** Compute the number of parameters added by LayerNorms in a single encoder layer (after MHA and after FFN), with $H = 512$.

    Solution. Each LN has $2H = 1024$ parameters (γ, β). Two LNs per encoder layer $\Rightarrow$ 2048 parameters per layer. Negligible compared to MHA ($\approx$ 1M) and FFN ($\approx$ 2M).

12. **(Q12)** Verify that for any position j the sum of squares $\sum_{i=0}^{d/2-1} \big(\sin^2(\omega_i j) + \cos^2(\omega_i j)\big) = d/2$, where $\omega_i = 10000^{-2i/d}$.

    Answer. By $\sin^2 + \cos^2 = 1$, each i contributes exactly 1. There are $d/2$ pairs (for $d = 512$, that is 256 pairs), so the total is $d/2 = 256$. *Hence $\|PE(j)\|_2 = \sqrt{d/2}$ is the same for every position j — norms encode no positional information; the* angles *do.*

13. **(Q13)** The base Transformer has $\approx$ 65M parameters. Roughly break this down by component (encoder, decoder, embeddings, W_D).

    Solution (approximate).

    - Encoder: $6 \cdot 3\text{M} \approx 18\text{M}$.
    - Decoder: $6 \cdot 4\text{M} \approx 24\text{M}$.
    - Source + target embedding matrices: $\sim 2 \cdot 37000 \cdot 512/10^6 \approx 38\text{M}$ *but* weight tying reduces this to one $\approx 19\text{M}$ chunk.
    - Output projection W_D (shared with embedding via weight tying): already counted.
    - Total: $\approx 18 + 24 + 19 \approx 61\text{M}$ (within rounding of the paper's 65M).

3 Week 3 — Language Modelling, GPT, and the Pre-train / Fine-tune Paradigm

Chapter goals. By the end of this chapter you will be able to:

1. Explain why pre-training on raw text is the right answer to the "one model per task / not enough labels" problem.
2. Define a language model formally and apply the chain rule of probabilities.
3. Distinguish encoder-only, decoder-only, and encoder–decoder architectures for language modelling.
4. Derive **Causal Language Modelling (CLM)** and write the training objective.
5. Walk through the GPT-1 architecture, dataset (BookCorpus), hyperparameters, and parameter count ($\sim$ 117M).
6. Explain the supervised fine-tuning recipe for classification, entailment, multiple choice, and text generation.

3.1 Where We Stand: The Problem to Solve

In Weeks 1–2 we built the full Transformer for *machine translation*. At a high level, the architecture is just an encoder-decoder black box:

source text $\longrightarrow$ **Transformer** $\longrightarrow$ translated text.

A natural next question: *Can we use the same architecture for other NLP tasks — sentiment classification, summarisation, question answering, entailment, MCQs?*

3.1.1 The naive approach and its problems

We could train a separate Transformer for each task from scratch. But:

- **Convergence is slow.** Random initialisation + each task's small labelled dataset = many epochs of training.
- **Labelled data is scarce.** High-quality labels for sentiment, QA, NLI cost money and time. Many tasks have only thousands of examples.
- **No knowledge sharing.** A model that learns "apple" is a fruit in a sentiment dataset cannot reuse that for QA.

Intuition

A human child does not learn each language task separately. We acquire language by reading and listening for years *without explicit supervision*, then specialise quickly to specific tasks. We say things like "*Wow, India has reached the moon*" and people immediately understand the sentiment, the syntax, the entailment — without having been trained on a sentiment classification dataset. Can a model learn the same way?

The answer is yes, and the key insight is:

Remember This

The Pre-train + Fine-tune Paradigm. First *pre-train* a model on a huge amount of *unlabelled* text using a self-supervised objective (predict the next word). Then *fine-tune* on a small labelled dataset for each downstream task. The pre-trained model has already learned "what language looks like"; fine-tuning only adapts it to the specific task.

The empirical evidence (Radford et al., 2018, GPT-1 paper): the pre-trained model with a *linear classifier* on top reached 91.8% accuracy on sentiment classification — beating the state-of-the-art — and matched fully supervised systems while using **30–100× fewer labelled examples.** This was the empirical kick-off of the modern LLM era.

3.2 Language Modelling — Formal Definition

Definition

Let $\mathcal{V}$ be the *vocabulary* — the set of all unique tokens in a language. A sentence is a sequence $X_1, X_2, \ldots, X_n$ with each $X_i \in \mathcal{V}$.
A **language model** is a function

$$f : (X_1, X_2, \ldots, X_n) \longrightarrow [0, 1]$$

that assigns a probability to each sequence. More probable sequences should be the ones that "look like real language".

Tiny example. Let $\mathcal{V} = \{\text{an, apple, ate, I}\}$. Some possible sequences:

- "An apple ate I." — grammatical disaster, low probability.
- "I ate an apple." — correct, high probability.
- "I ate apple." — ungrammatical but plausible, medium probability.
- "An apple." — short fragment.

A good language model would assign these very different probabilities.

3.2.1 The chain rule of probability

The joint probability of a sequence factorises:

$$P(x_1, x_2, \ldots, x_T) \;=\; P(x_1)\, P(x_2 \mid x_1)\, P(x_3 \mid x_2, x_1) \cdots P(x_T \mid x_{T-1}, \ldots, x_1),$$

or compactly

$$\boxed{P(x_1, x_2, \ldots, x_T) \;=\; \prod_{i=1}^{T} P(x_i \mid x_1, \ldots, x_{i-1}).} \tag{21}$$

3.2.2 The naive independence assumption fails

If we naively assumed words were independent: $P(x_1, \ldots, x_T) = \prod_i P(x_i)$. But then "I enjoyed reading a book" and "I enjoyed reading a thermometer" would get similar probabilities (both have common words). The *presence of* "enjoyed" + "reading" should make "book" far more likely than "thermometer". Words are *conditional* on context. So the right model is Eq. (21).

Formula Insight

The chain rule turns a hard problem (modelling joint distribution over all possible sequences) into a sequence of *conditional* problems, each "predict the next word given the past". This is exactly what a left-to-right neural language model computes.

3.3 Three Ways to Parameterise a Language Model

We have a Transformer at our disposal. We can use it three ways:

Architecture	Input / Objective	Example models
Encoder-only	Input has a `[MASK]` token; predict the masked word using both left and right context.	BERT, RoBERTa, ALBERT
Decoder-only	Input is $x_1, \ldots, x_{i-1}$; predict x_i. *Left context only (causal).*	GPT-1, 2, 3, 4; LLaMA; Mistral; Claude; Gemini
Encoder–decoder	Encode an input (possibly masked) and decode an output sequence.	T5, BART, Pegasus

Intuition

Encoder-only models can use *bidirectional* context. They are best for *understanding* tasks (classification, NER, sentence embeddings).
Decoder-only models see only the past. They are best for *generation* (continuing a text, writing, dialogue).
Encoder-decoder models read one sequence and produce another. They are best when input and output are structurally different (translation, summarisation).
Modern LLMs (GPT family, LLaMA, Claude) are overwhelmingly **decoder-only** because generation is the most general capability — and a decoder can be *prompted* to do any task by phrasing it as "predict what comes next".

3.4 Causal Language Modelling (CLM)

The decoder-only language model is the canonical setup. It is sometimes called *Causal Language Modelling* because each prediction depends only on *past* (causal) context.

Definition

A **Causal Language Model** learns a parameterised distribution

$$P(x_i \mid x_1, \ldots, x_{i-1}; \theta) = f_\theta(x_i \mid x_1, \ldots, x_{i-1})$$

typically implemented as a Transformer decoder with masked self-attention. The **training objective** is to maximise the log-likelihood over a corpus:

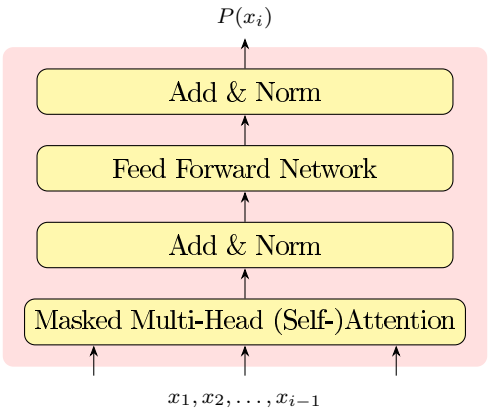
$$\mathcal{L}(\theta) = \sum_{i=1}^{T} \log P(x_i \mid x_1, \ldots, x_{i-1}; \theta).$$

3.4.1 Architecture: A pruned decoder block

We start from the original Transformer decoder (three sub-layers: masked self-attention, cross-attention, FFN). For language modelling there is *no encoder*, so we remove the cross-attention sub-layer:

$$\textbf{Causal LM block} = \text{Masked Self-Attention} \rightarrow \text{FFN}.$$

Diagram — *this is the heart of GPT*:



Stack n such blocks; the original GPT-1 used $n = 12$.

3.4.2 The output side: from hidden vectors to probabilities

Let the final-block hidden states be $h_n[1], h_n[2], \ldots, h_n[T] \in \mathbb{R}^{d_{\text{model}}}$. A single weight matrix $W_v \in \mathbb{R}^{d_{\text{model}} \times |\mathcal{V}|}$ projects each hidden state to vocabulary scores; softmax converts scores to probabilities:

$$P(x_i \mid x_1, \ldots, x_{i-1}) = \text{softmax}\big(h_n[i]\, W_v\big). \tag{22}$$

3.4.3 The masking trick again — why one forward pass suffices

Intuition

At every position i, the model predicts $P(x_i \mid \text{past})$ *from the position* $i-1$ *hidden state.* Because of the causal mask, position $i-1$ only attends to positions $1, \ldots, i-1$ — so by construction, the conditional probability respects causality.
The genius: a single forward pass over T tokens produces *all* T *conditional probabilities* of Eq. (21) simultaneously. This is exactly what makes Transformer LM training *parallel*, unlike RNN LMs.

3.5 GPT-1 — The First Big Win for Pretraining + Decoder-Only Transformers

Generative Pretrained Transformer 1 (Radford, Narasimhan, Salimans, Sutskever, 2018) used the following architecture:

Number of decoder blocks (n)	12
Embedding dimension (d_{model})	768
Attention heads (h)	12 (so $d_k = 64$)
FFN hidden width (l)	$4 \cdot 768 = 3072$
Context length (T)	512
Activation in FFN	GELU
Position embedding	*Learned* (not sinusoidal)
Vocabulary size ($\|\mathcal{V}\|$)	40,478
Tokenizer	Byte-Pair Encoding (BPE)
Optimizer	Adam, cosine LR schedule
Batch size	64 sequences
Dataset	BookCorpus

BookCorpus — 7,000 unique books, 74 million sentences, ≈ 1 billion words spanning 16 genres. Importantly, it contains *long contiguous passages* (no sentence-level shuffling), which is critical for learning long-range dependencies.

Research / Industry Insight

The choice of learned (vs. sinusoidal) position embedding became standard in decoder-only LLMs — it's a simple lookup table of size $T_{\max} \times d_{\text{model}}$. The downside is that the model cannot extrapolate beyond $T_{\max}$. GPT-2/3 inherited this; later models (GPT-NeoX, LLaMA) moved to **RoPE** (rotary position embeddings) which extrapolate better.

3.6 Counting GPT-1's Parameters — The Famous 117M

This is one of the most-asked exam / interview questions, so let's do it slowly. Three components contribute.

3.6.1 (a) Embeddings

- **Token embeddings:** $|\mathcal{V}| \cdot d_{\text{model}} = 40{,}478 \cdot 768 \approx 31.1$ M.
- **Position embeddings (learned):** $T_{\max} \cdot d_{\text{model}} = 512 \cdot 768 \approx 0.39$ M.
- **Subtotal:** ≈ 31.3 M.

3.6.2 (b) Per-block attention parameters

Per head: W_Q, W_K, W_V are each 768×64, so $3 \cdot 768 \cdot 64 \approx 147{,}456$. With $h = 12$ heads, that's $12 \cdot 147{,}456 \approx 1.77$ M. Plus the output projection W^O of $768 \times 768 = 589{,}824 \approx 0.59$ M.

Per block: $1.77 + 0.59 \approx 2.36$ M.

For all 12 blocks: $12 \cdot 2.36 \approx 28.3$ M (the slide rounds to 27.6 M).

3.6.3 (c) Per-block FFN parameters

$$2 \cdot (768 \cdot 3072) + 3072 + 768 = 4{,}722{,}432 + 3072 + 768 \approx 4.72 \text{ M per block.}$$

For 12 blocks: ≈ 56.7 M.

3.6.4 Totals

Layer	Parameters (M)
Embedding (token + position)	31.3
Attention (12 blocks)	27.6
FFN (12 blocks)	56.4
Total	**≈ 117 M**

Exam Tip

Memorise: GPT-1 has ∼ 117M parameters. Breakdown: roughly 1/4 embedding, 1/4 attention, 1/2 FFN. (Note: the FFN dominates the parameter budget at this scale — a recurring pattern in Transformers.)

3.7 Pre-Training Objective in One Equation

Given a corpus $\mathcal{X}$ of token sequences, the cross-entropy loss per sequence is:

$$\mathcal{L}_{\text{pre}}(\theta) = -\sum_{X \in \mathcal{X}} \sum_{i=1}^{T} \sum_{v \in \mathcal{V}} y_v^{(i)} \log \hat{y}_v^{(i)}, \tag{23}$$

where $y_v^{(i)}$ is 1 if the true next token at position i is v (one-hot), and $\hat{y}_v^{(i)}$ is the model's predicted probability. Equivalently:

$$\boxed{\mathcal{L}_{\text{pre}}(\theta) = -\sum_{i=1}^{T} \log P(x_i \mid x_{<i}; \theta).} \tag{24}$$

This is exactly the (negative) log-likelihood of Eq. (21).

3.7.1 Why teacher forcing again?

During pre-training we feed the *actual* ground-truth tokens at each position (not the model's own predictions). This is teacher forcing (Week 2). It enables one parallel forward pass over all T positions and dramatically speeds up training compared to auto-regressive sampling.

3.8 Fine-Tuning — Adapting GPT to Downstream Tasks

After pre-training, we have a model that has internalised a lot about language. To solve a specific labelled task, we:

1. **Initialise** all the Transformer parameters from the pre-trained checkpoint.
2. **Format the input** with special tokens (start $\langle s \rangle$, end $\langle e \rangle$, delimiter $\langle \$ \rangle$ where appropriate) so the model knows what kind of task it is solving.
3. **Replace the LM head** (W_v mapping to $|\mathcal{V}|$) with a **task-specific head** (e.g., a linear layer W_y mapping to the number of classes).
4. **Train end-to-end** on the labelled dataset — typically all parameters are updated, but the LR is much smaller than pre-training.

3.8.1 Why use the *last* position's hidden state for classification?

The output for classification is $\hat{y} = \text{softmax}(W_y\, h_m^{(n)})$ where $h_m^{(n)}$ is the *last* token's hidden state at the *last* (top) layer. Why the last token?

Intuition

Because of causal masking, only the *last* position can attend to *every* preceding token. Earlier positions see only a prefix. The last token therefore contains a summary of the entire sequence — it's the only position that has "seen everything".

3.8.2 Fine-tuning loss

$$\mathcal{L}_{\text{FT}} = -\sum_{(X,y) \in \mathcal{C}} \log P(y \mid x_1, \ldots, x_m). \tag{25}$$

(Optionally, GPT-1 used an auxiliary LM loss during fine-tuning too, mixed with the task loss — this regularises the model and slows down catastrophic forgetting.)

3.9 Task-Specific Input Formats

GPT-1 used a clever idea: instead of building task-specific architectures, encode the *task* as part of the input through special delimiter tokens. Then a single Transformer solves all tasks. Examples:

3.9.1 Sentiment classification

$$\langle s \rangle \text{ Wow, India has now reached the moon } \langle e \rangle \rightarrow \hat{y} \in \{+1, -1\}.$$

Linear head reads the hidden state at $\langle e \rangle$.

3.9.2 Textual entailment / NLI

$$\langle s \rangle \text{ Premise } \langle \$ \rangle \text{ Hypothesis } \langle e \rangle \rightarrow \hat{y} \in \{\text{entail, neutral, contradict}\}.$$

The delimiter $\langle \$ \rangle$ tells the model: "premise ends here, hypothesis begins".

3.9.3 Multiple choice (MCQ)

For an MCQ with K choices, run the model K times, once per choice:

$$\text{Run } k: \ \langle s \rangle \text{ Question } \langle \$ \rangle \text{ Choice-}k \ \langle e \rangle.$$

Each run produces a scalar score (a linear layer reads the $\langle e \rangle$ hidden state). Stack the K scores into a vector, apply softmax across choices, pick the largest.

3.9.4 Text generation

The simplest task. Feed a prompt:

$$\langle s \rangle \text{ I like}$$

and run the model auto-regressively, sampling $\hat{x}_{i+1} \sim P(\cdot \mid x_{\leq i})$ each step, until $\langle e \rangle$ is produced or a maximum length is reached. Output: "I like to think that…".

Remember This

The pre-train + fine-tune recipe — in three lines:
1. Pre-train a big decoder-only Transformer on raw text with the LM objective.
2. Add task-specific delimiters at the input; add a small linear head at the output.
3. Fine-tune end-to-end with the task's labelled loss.

This recipe is the foundation of every modern LLM (with subsequent refinements like RLHF and instruction tuning, covered in later weeks).

3.10 Wishlist for Text Generation (Preview of Week 4)

The same auto-regressive generation produces wildly different outputs depending on *how we sample* from $P(x_i \mid x_{<i})$. Three goals:

- **Discourage degenerate output:** avoid repetition ("I like to think that I like to think that…") and incoherence.
- **Encourage creativity:** for the same prompt, allow diverse continuations.
- **Accelerate generation:** make sampling fast even for long outputs.

These goals are exactly the topic of **Week 4: decoding strategies.** We will study greedy decoding, beam search, top-k, top-p (nucleus), and temperature.

3.11 Common Mistakes & Quiz Traps — Week 3

Common Mistake

Mistake 1. "GPT uses both the encoder and the decoder of the Transformer."
Fix. GPT is **decoder-only**. There is no encoder, so no cross-attention. The block is just masked self-attention + FFN.

Common Mistake

Mistake 2. "Language modelling means assigning the same probability to every grammatical sentence."
Fix. A language model assigns higher probabilities to sentences that are *frequent in real text*, not just grammatical. "The cat is on the mat" beats "A purple electron contemplates Tuesday" even though both are grammatical.

Common Mistake

Mistake 3. "The chain rule of probability requires independence."
Fix. The chain rule *never* requires independence — it is always true ($P(A, B) = P(A)P(B \mid A)$ by definition of conditional probability). The naive independence *assumption* (which we *rejected*) was a separate, wrong, simplification.

Quiz Trap

Trap. "Decoder-only models cannot do classification."
Reality. They can — read the hidden state at the last token and put a linear classifier on it. The *last* token's hidden state has attended to everything before it (because of causal masking), so it acts as a sentence summary.

Quiz Trap

Trap. "Pre-training and fine-tuning use the same loss function."
Reality. Pre-training uses the LM (cross-entropy over the next token). Fine-tuning uses a *task-specific* loss (cross-entropy over classes for classification, etc.). GPT-1 famously mixes the two during fine-tuning to reduce forgetting — an early hint at multi-task learning.

3.12 Memory Tricks — Week 3

Memory Trick

"Mask Cross FFN → Mask FFN."
The GPT block is the Transformer decoder block with the cross-attention sub-layer *removed*. From three sub-layers down to two.

Memory Trick

"BERT looks both ways, GPT only back."
BERT is encoder-only (bidirectional context, MLM objective). GPT is decoder-only (left-to-right context, CLM objective).

Memory Trick

"Predict next token = learn language."
The simple-seeming objective of next-token prediction forces the model to learn syntax, semantics, world knowledge, reasoning, style ... all to get the probabilities right. This is the central insight of modern LLMs.

Memory Trick

"GPT-1 is 117M: 31 embedding + 27 attention + 56 FFN."
Roughly $1/4 + 1/4 + 1/2$. The FFN typically dominates the parameter budget.

3.13 Practical Applications

- **All modern LLMs** (GPT-3/4, Claude, Gemini, LLaMA, Mistral) are decoder-only Transformers trained with the same CLM objective — only larger.
- **In-context learning** (few-shot prompting): instead of fine-tuning, just give the model a few examples *in the prompt* and it can solve the task. Discovered with GPT-3 in 2020.
- **Zero-shot inference**: simply ask the model in natural language to do the task; no examples needed. Possible only because pre-training has internalised such a wide distribution of tasks.
- **Instruction-tuned models** (ChatGPT, Claude): GPT/LLaMA further fine-tuned on (instruction, response) pairs and RLHF. Covered in Week 8.

3.14 Quick Revision — Week 3 in One Page

Quick Revision

Big picture. Pre-training on raw text (LM objective) + fine-tuning on labelled data (task-specific loss) = the modern recipe. GPT-1 (2018) demonstrated this works: SOTA sentiment with 30–100× less labelled data.
Language model. A function f that assigns probabilities to sequences: $f(x_1, \ldots, x_T) \in [0, 1]$.
Chain rule. $P(x_1, \ldots, x_T) = \prod_i P(x_i \mid x_{<i})$ — always true; no independence needed.
Three flavours.
- Encoder-only → MLM (BERT, RoBERTa).
- Decoder-only → CLM (GPT, LLaMA, Claude).
- Encoder-decoder → seq2seq (T5, BART).

GPT-1 block. Masked self-attention → FFN. Stack 12 blocks.
LM head. Linear $W_v \in \mathbb{R}^{d \times |\mathcal{V}|}$ + softmax.
GPT-1 config. $n = 12$, $d = 768$, $h = 12$, $l = 3072$, $T = 512$, BPE, BookCorpus, $|\mathcal{V}| = 40{,}478$. **117M parameters total.**
Pre-training loss. $-\sum_i \log P(x_i \mid x_{<i}; \theta)$.
Fine-tuning. Initialise from pre-trained checkpoint; add delimiters ($\langle s \rangle, \langle \$ \rangle, \langle e \rangle$); swap LM head for task head; train end-to-end. Read last-token hidden state for classification.

3.15 Practice Questions — Week 3

Multiple Choice

1. **(Q1)** GPT-1 is best described as:

   (A) Encoder-only Transformer with masked language modelling.

(B) Decoder-only Transformer with causal language modelling.

(C) Encoder-decoder Transformer.

(D) A recurrent network with attention.

Solution. (B).

2. **(Q2)** In Causal Language Modelling, the conditional probability $P(x_i \mid x_{<i})$ is enforced by:

   (A) adding positional encoding,

   (B) the causal (triangular) attention mask,

   (C) batch normalisation,

   (D) the GELU activation.

   Solution. (B). Without the mask, position i could attend to future positions and "cheat".

3. **(Q3)** In GPT-1's parameter budget ($\sim$ 117M), the layer with the most parameters is:

   (A) embedding,

   (B) attention,

   (C) FFN,

   (D) LayerNorm.

   Solution. (C). FFN $\sim$ 56M $>$ embedding $\sim$ 31M $>$ attention $\sim$ 27.6M $\gg$ LayerNorm.

4. **(Q4)** For classification fine-tuning with GPT-1, the linear head reads the hidden state at:

   (A) the first token $\langle s \rangle$,

   (B) every position (averaged),

   (C) the last token $\langle e \rangle$,

   (D) the [CLS] token.

   Solution. (C). Because of causal masking, only the last position has attended to every preceding token. (A) and (D) are BERT conventions, not GPT.

5. **(Q5)** Pre-training + fine-tuning is more sample-efficient than training from scratch because:

   (A) pre-training initialises parameters with rich linguistic priors,

   (B) pre-training reduces the number of parameters,

   (C) fine-tuning uses smaller batch sizes,

   (D) the optimizer converges in zero steps.

   Solution. (A). The pre-trained weights encode general language structure that fine-tuning only needs to adapt.

Conceptual / Short Answer

6. **(Q6)** Why was BookCorpus preferred over the 1B-words benchmark for GPT-1 pre-training?

   Answer. BookCorpus contains *long contiguous passages* (entire books). The 1B-words dataset shuffles sentences and breaks paragraph-level continuity. To learn *long-range dependencies* (entailment between distant sentences, narrative coherence, referent tracking), the model needs contiguous text.

7. **(Q7)** How does GPT-1 handle a multiple-choice question with 4 options at inference?

   Answer. Run the model 4 times, once per (question + choice) pair. Each run produces a scalar score (linear layer on last token's hidden state). Stack scores into a 4-vector, softmax across choices, predict the argmax.

8. **(Q8)** Explain in one sentence why GPT-1 uses learned positional embeddings instead of sinusoidal ones.

   Answer. GPT-1 has a fixed max context $T_{\max} = 512$, so a learned lookup table (512×768) is sufficient, simpler, and lets the model freely learn any positional pattern. The downside (inability to extrapolate beyond 512) was acceptable in 2018.

9. **(Q9)** Compare the pre-training objective of BERT (MLM) and GPT (CLM) in terms of context.

   Answer.

   - BERT (MLM): randomly mask 15% of tokens; predict each masked token from *both left and right* context. Bidirectional.
   - GPT (CLM): predict each token from *left context only*. Unidirectional / causal.

   BERT is better for understanding; GPT is better for generation.

Numerical

10. **(Q10)** Suppose we increase GPT-1's context length from 512 to 2048, keeping $d_{\text{model}} = 768$, $h = 12$, $n = 12$. By how many parameters do the *position embeddings* grow?

    Solution. Position embedding table grows from 512×768 to 2048×768, i.e., from ≈ 0.39M to ≈ 1.57M. Net increase ≈ 1.18M parameters. (The attention sub-layer parameters do *not* grow with context length — only the compute and KV-cache costs scale with T.)

11. **(Q11)** Compute the FFN parameters per GPT-1 block exactly.

    Solution.

    $$W_1 : 768 \cdot 3072 = 2{,}359{,}296, \quad b_1 : 3072, \quad W_2 : 3072 \cdot 768 = 2{,}359{,}296, \quad b_2 : 768.$$

    Total $= 4{,}722{,}432 \approx 4.72$ M. Across 12 blocks ≈ 56.7 M.

12. **(Q12)** A model has $d_{\text{model}} = 768$ and a vocabulary $|\mathcal{V}| = 50{,}000$. What is the size of the LM head W_v, and what fraction of GPT-1's total parameters would it represent?

    Solution. $W_v \in \mathbb{R}^{768 \times 50{,}000}$, size ≈ 38.4 M. As a fraction of 117M: $\approx 33\%$. *This is why all modern LLMs use weight tying (share embedding and output projection) — it saves $\sim 30\%$ of parameters!*

13. **(Q13)** Write down the log-likelihood of the sentence "$\langle$go$\rangle$ I like cats" under a CLM, treating $\langle$go$\rangle$ as the start token (no prediction loss on it).

    Solution.

    $$\log P(\text{"I like cats"} \mid \langle\text{go}\rangle) = \log P(\text{I} \mid \langle\text{go}\rangle) + \log P(\text{like} \mid \langle\text{go}\rangle, \text{I}) + \log P(\text{cats} \mid \langle\text{go}\rangle, \text{I}, \text{like}).$$

    Maximising this $\Leftrightarrow$ minimising the per-position cross-entropies.

4 Week 4 — Decoding Strategies & BERT (Bidirectional Encoder Representations)

Chapter goals. By the end of this chapter you will be able to:

1. Distinguish deterministic vs. stochastic decoding strategies.
2. Trace exhaustive search, greedy search, and beam search, with worked numerical examples.
3. Define and contrast top-k, top-p (nucleus), and temperature sampling.
4. Explain why bidirectional context matters and why GPT-style models cannot provide it.
5. Derive the BERT pre-training objective: Masked Language Modelling (MLM) + Next Sentence Prediction (NSP).
6. Count BERT-base parameters ($\sim$ 110M) and dissect its embedding scheme (token + segment + position).
7. Apply BERT to downstream tasks: classification, extractive question answering.

Part A — Decoding Strategies

4.1 The Wishlist Restated

In Week 3 we built a Causal Language Model that, at each position, outputs a probability distribution $P(x_i \mid x_{<i})$ over the vocabulary. *How do we actually pick the next token from this distribution*?

Intuition

Different choices of decoding strategy turn the *same* pre-trained model into completely different generators:

- For machine translation we want *the most likely* target sentence.
- For story writing we want *creativity* — diverse outputs for the same prompt, while still being coherent.
- For real-time chat we want *speed* on top of quality.

There is no single best strategy. Decoding is a knob you turn at *inference time* to match the task.

The two families of decoding strategies:

Family	Examples	Output for same prompt
Deterministic	Exhaustive search, Greedy search, Beam search	Always the same
Stochastic (Sampling)	Top-k, Top-p (nucleus), Temperature, Contrastive decoding	Different each run

4.2 Exhaustive Search — The Theoretical Ideal

Definition

Exhaustive search enumerates *every* possible sequence of length T, computes the joint probability via the chain rule, and returns the one with the highest probability:

$$\hat{Y} = \arg\max_{Y \in \mathcal{V}^T} \prod_{i=1}^{T} P(y_i \mid y_{<i}).$$

Why it is impossible at scale. With vocabulary $|\mathcal{V}| = 40{,}000$ and length $T = 5$, there are $|\mathcal{V}|^T = 40{,}000^5 \approx 10^{23}$ candidate sequences. Even at one billion sequences per second, this would take ~ 3 million years. So we need approximations.

4.2.1 Tiny illustration

Suppose $\mathcal{V} = \{A, B, C\}$ and we want length-2 sequences. At $t = 1$ the decoder outputs $P(A), P(B), P(C)$. At $t = 2$ we need to run the decoder *three times in parallel* (once for each possible x_1) to get $P(\cdot \mid A)$, $P(\cdot \mid B)$, $P(\cdot \mid C)$. That's $3 + 9 = 12$ forward passes for $T = 2$; at $T = 3$ it explodes to 27. With $|\mathcal{V}| = 40{,}000$ and $T = 5$, the count is intractable.

4.3 Greedy Search — The Other Extreme

Definition

Greedy search picks the most probable next token at each step, then conditions on that choice for the next step:

$$y_i = \arg\max_{y \in \mathcal{V}} P(y \mid y_{<i}).$$

4.3.1 Worked example

Suppose $\mathcal{V} = \{\text{cold}, \langle\text{stop}\rangle, \text{coffee}, \text{I}, \text{like}, \text{water}\}$ and the model's conditional probabilities at each of 5 time steps (greedy picks shown in bold) are:

	$t=1$	$t=2$	$t=3$	$t=4$	$t=5$
cold	0.10	0.10	**0.45**	0.15	0.10
⟨stop⟩	0.15	0.15	0.05	0.30	**0.50**
coffee	0.25	0.25	0.10	**0.35**	0.20
I	**0.40**	0.05	0.05	0.01	0.10
like	0.05	**0.35**	0.15	0.09	0.05
water	0.05	0.10	0.20	0.10	0.05

Greedy output: "I like cold coffee ⟨stop⟩".
Joint probability:

$$0.40 \times 0.35 \times 0.45 \times 0.35 \times 0.50 \;=\; 0.011.$$

4.3.2 Limitations of greedy

- **Locally optimal ≠ globally optimal.** A locally low-probability token at $t = 1$ might unlock a much higher-probability continuation. Greedy never explores that branch.
- **Deterministic.** For the same prompt, you always get the same output — no diversity.
- **Repetition.** "I like to think that I like to think that…" — once a phrase is locally most probable, the same loop repeats.

Intuition

Why greedy can be globally suboptimal. If at $t = 1$ we instead picked the second most probable token (say "coffee" with $P = 0.25$), the conditional distributions at subsequent steps change. Suppose with that prefix we then get $0.55, 0.65, 0.80, 0.50$ at subsequent steps, giving total $0.25 \times 0.55 \times 0.65 \times 0.80 \times 0.50 = 0.0357$ — *three times higher* than the greedy sequence's 0.011! This is exactly why beam search is helpful.

4.4 Beam Search — The Practical Sweet Spot

Definition

Beam search maintains the k most probable *partial sequences* (called *beams*) at every step. At step $t+1$, each beam is extended by all $|\mathcal{V}|$ possible tokens, and the top k of the resulting $k \cdot |\mathcal{V}|$ candidates are kept. The parameter k is the **beam size.**

4.4.1 Special cases

- $k = 1 \Rightarrow$ greedy search.
- $k = |\mathcal{V}|$ (or larger) $\Rightarrow$ exhaustive search.
- Typical practical values: $k = 4, 5, 8$ for translation; up to 20 for careful research.

4.4.2 Worked beam-search example with $k = 2$

Vocabulary $\{A, B, C\}$, sequence length 3. At $t = 1$:

$$P(A) = 0.5, \quad P(B) = 0.4, \quad P(C) = 0.1.$$

The top-2 beams are $\{A\}$ and $\{B\}$ (we drop C).

At $t = 2$, for each surviving beam we look at the full conditional. Suppose

$$P(\cdot \mid A) = \{A : 0.1,\ B : 0.2,\ C : 0.5\} \quad \text{(and others sum to 0.2)}$$
$$P(\cdot \mid B) = \{A : 0.2,\ B : 0.2,\ C : 0.6\}$$

We expand both beams to get 6 candidate length-2 sequences:

(A, A)	$0.5 \times 0.1 = 0.05$	(B, A)	$0.4 \times 0.2 = 0.08$
(A, B)	$0.5 \times 0.2 = 0.10$	(B, B)	$0.4 \times 0.2 = 0.08$
(A, C)	$0.5 \times 0.5 = \mathbf{0.25}$	(B, C)	$0.4 \times 0.6 = \mathbf{0.24}$

Top 2: $\{(A, C),\ (B, C)\}$. We keep these two beams forward to $t = 3$.

At $t = 3$, suppose $P(B \mid A, C) = 0.75$ and $P(A \mid B, C) = 0.55$. Then:

$$P(A, C, B) = 0.25 \times 0.75 = 0.1875,$$
$$P(B, C, A) = 0.24 \times 0.55 = 0.132.$$

Output: (A, C, B) — the beam with highest final probability.

4.4.3 Length normalisation

Formula Insight

The length-bias problem. Each multiplication by a probability < 1 shrinks the joint. Longer sequences will always have *smaller* probabilities, so beam search naturally prefers short outputs — even if they truncate the sentence prematurely.
Fix: divide by sequence length (or use a length penalty ℓ^α with $\alpha \in [0.6, 0.7]$):

$$\text{score}(Y) = \frac{1}{\ell(Y)} \sum_{i=1}^{\ell(Y)} \log P(y_i \mid y_{<i}).$$

This is standard in production MT systems.

4.4.4 Cost and behaviour

Beam search requires $k \cdot |\mathcal{V}|$ score computations per step (and k forward passes through the model in parallel, since each beam is its own context). Total: $\mathcal{O}(k \cdot |\mathcal{V}| \cdot T)$ compared to greedy's $\mathcal{O}(|\mathcal{V}| \cdot T)$.

Where beam search shines: translation, summarisation, structured generation where "correct output" is well-defined.

Where it disappoints: open-ended generation (creative writing, dialogue) — it tends to produce bland, repetitive, "newspaper-headline" text, because the most probable sequences in human language are also the most generic ones.

4.5 Sampling — Embracing Stochasticity

Intuition

A 2019 study (Holtzman et al., *The Curious Case of Neural Text Degeneration*) plotted, position-by-position, the rank of the token that a human writer chose. Humans pick the top-1 rank only $\sim 20\%$ of the time. Beam search picks rank-1 $\sim 80\%$ of the time. **Surprise is what makes text interesting.**

To inject controlled surprise, we *sample* from the distribution instead of taking the argmax. Pure "ancestral" sampling (sample directly from P) is too noisy — it sometimes picks low-quality, low-probability tokens. The three refinements below tame the noise.

4.5.1 Top-k sampling

Definition

Top-k sampling keeps only the k highest-probability tokens at each step, renormalises their probabilities to sum to 1, and samples from that truncated distribution.

Example. Suppose at $t = 1$ the model gives $P = (0.40, 0.25, 0.15, 0.10, 0.05, 0.05)$ over (I, coffee, like, cold, water, ⟨stop⟩). With $k = 2$, we keep {I: 0.40, coffee: 0.25}, renormalise to {I: 0.61, coffee: 0.39}, and sample according to those probabilities.

Limitation. A *flat* distribution (many roughly-equal options) is truncated too aggressively — we lose creativity. A *peaked* distribution (one dominant option) admits too many implausible options. *There is no single k that works for both.*

4.5.2 Temperature scaling

Definition

Temperature sampling reshapes the distribution before sampling:

$$P_T(x = u_\ell \mid x_{<i}) = \frac{\exp(u_\ell/T)}{\sum_{\ell'} \exp(u_{\ell'}/T)},$$

where u_ℓ is the pre-softmax logit and $T > 0$ is the *temperature*.

Behaviour:

- $T = 1$: original softmax distribution.
- $T \to 0$: distribution becomes a one-hot at the argmax (deterministic / greedy).
- $T \to \infty$: distribution becomes uniform (pure noise).
- Typical range: $T \in [0.5, 1.5]$ for creative generation.

Formula Insight

Why dividing by T works: writing $\text{softmax}(u/T) = \text{softmax}((u_1/T, u_2/T, \ldots))$, larger T *shrinks* the differences between logits, flattening the distribution. Smaller T *magnifies* differences, sharpening the distribution.

4.5.3 Top-p (nucleus) sampling

Definition

Top-p (nucleus) sampling chooses the smallest set of tokens whose cumulative probability exceeds a threshold $p \in (0, 1)$, then samples from that set:

1. Sort tokens by probability in descending order.
2. Accumulate probabilities top-down until the cumulative sum exceeds p.
3. Renormalise the selected tokens and sample.

Why it beats top-k. The size of the kept set is *adaptive*:

- **Flat distribution** (e.g., "she said 'I never __ '" has many plausible continuations: *thought*, *knew*, *had*, *saw*, *said*, ...): with $p = 0.6$, the model keeps perhaps 5 tokens.
- **Peaked distribution** (e.g., "I ate the pizza while it was still __" — "hot" dominates): with $p = 0.6$, the model keeps only 1-2 tokens.

Recommended setting (Holtzman et al.): $p = 0.9$ or $p = 0.95$. This is the de facto standard for open-ended generation in modern LLMs.

4.5.4 Combining temperature with top-p

In practice, ChatGPT, Claude, and similar models use *both*: temperature reshapes the distribution, then nucleus sampling truncates it. Two knobs the user can turn:

- Lower temperature (~ 0.2) + top-p (~ 0.9) → focused, deterministic-ish answers (factual QA).
- Higher temperature (~ 0.9) + top-p (~ 0.95) → creative writing, brainstorming.

4.5.5 Contrastive decoding (brief)

A newer family (Li et al., 2023): generate from a strong model, but *down-weight* tokens that a weaker reference model would also predict highly. The intuition: tokens that both the small and large model agree on are usually generic; the large model's *distinctive* preferences are more informative. Used for higher-quality long generation.

4.6 Decoding Comparison Cheat Sheet

	Greedy	**Beam**	**Top-k / p + T**	**Contrastive**
Output diversity	none	low	high (tunable)	moderate
Repetition tendency	high	medium	low	low
Best for	quick demos	translation, summarisation	dialogue, creative writing	long-form generation
Determinism	yes	yes	no (with fixed seed: yes)	yes
Latency	low	high ($k\times$)	low	medium
Quality bias	bland	generic	creative but risk noise	quality

Part B — BERT and Masked Language Modelling

4.7 Why We Need Bidirectional Context

GPT (Week 3) builds a representation *left-to-right*. This is great for generation, where the future is unknown anyway. But many tasks need *bidirectional* understanding:

- **Named Entity Recognition** (e.g., "**Nothing** has shipped its new OS to **Nothing Phone 2**"): the second mention of "Nothing" is a brand name — you need the *right* context ("Phone 2") to disambiguate it from the negation.
- **Fill-in-the-blank**: "*I* __ *to read a* __." To predict the first blank as "love", the model should see the second blank's context ("book") as well.
- **Sentence classification**: any token in the sentence can be relevant to determining the label.

Intuition

GPT can read only the past; BERT reads the whole sentence at once. Both architectures *compute* self-attention over the full sequence, but GPT *masks* the future, while BERT does not. This single design choice is the core difference between the two families.

4.8 Masked Language Modelling (MLM) — The BERT Pre-training Objective

Definition

In **Masked Language Modelling** we corrupt the input by replacing some tokens with a special `[MASK]` token, then ask the model to predict the original tokens:

$$P(y_i \mid x_1, \ldots, x_{i-1}, [\text{MASK}], x_{i+1}, \ldots, x_T) \; = ?$$

The model must use *both* left and right context to recover the masked words.

4.8.1 Why we cannot just "mask attention weights" the way GPT does

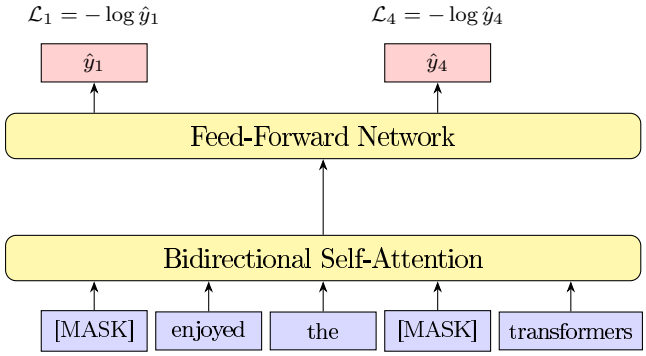
In GPT, masking the upper triangle of the attention matrix forbids attending to the future. Could we do the analogue here — mask the column corresponding to the word to predict?

No. If position i cannot attend to itself, then the *output* hidden state at position i would not depend on the input at position i at all — which is exactly the problem we want: don't tell the model the answer. But then the *other positions* also cannot attend to position i (and we don't want that either). The correct trick is different:

Remember This

Insert a special `[MASK]` token into the input itself. Add `[MASK]` as a new token in the vocabulary, learn an embedding for it, and use this replacement at the input positions we want to predict. Every other token can attend to `[MASK]` (it has its own learnable embedding), and the model is asked to recover the original token from the `[MASK]` position's final hidden state. This is exactly the *denoising* objective of an autoencoder.

4.8.2 Self-supervision in one diagram



The loss is summed only over the masked positions, not over every position:

$$\mathcal{L}_{\text{MLM}} = -\frac{1}{|M|} \sum_{i \in M} \log \hat{y}_i, \tag{26}$$

where M is the set of masked positions and $\hat{y}_i$ is the model's predicted probability of the true original token at position i.

4.8.3 The masking recipe

BERT's exact recipe:

- Sample 15% of tokens uniformly at random.
- Of those:
  - 80% are replaced by `[MASK]`.
  - 10% are replaced by a *random* token from the vocabulary.
  - 10% are left *unchanged*.

Formula Insight

Why the 80/10/10 split?
If we always replaced with `[MASK]`, the model would learn that "the question is always at a `[MASK]` position". But at fine-tuning and inference, there are no `[MASK]` tokens! The other 20% (10% random, 10% unchanged) trains the model to make predictions even at non-`[MASK]` positions, reducing the train/test mismatch.

4.8.4 Why 15%?

- Too *low* masking (e.g., 1%): gradient signal per example is tiny; training is wasteful and slow.
- Too *high* masking (e.g., 50%): the model loses too much context to learn good representations.
- 15% is the empirical sweet spot from the BERT paper. (Subsequent work — e.g., the "Should you mask 15%?" paper — shows that 40% can be optimal for larger models.)

4.9 Next Sentence Prediction (NSP)

MLM gives BERT *word-level* understanding. But many downstream tasks need *sentence-pair* understanding: question answering (does this passage answer the question?), entailment (does sentence A imply sentence B?), paraphrase detection.

Definition

Next Sentence Prediction is a binary classification task added during pre-training:
- Input: two sentences A and B, separated by a special `[SEP]` token, with a special `[CLS]` token at the start.
- Label: `IsNext` (B genuinely follows A in the source corpus) vs. `NotNext` (B is a random sentence).
- Training: 50% positive pairs, 50% random pairs.

The model uses the final hidden state at the `[CLS]` position to predict the label.

4.9.1 Combining MLM and NSP losses

BERT is pre-trained jointly: simultaneously predict masked tokens (MLM) and the `IsNext` label (NSP). The total loss is

$$\mathcal{L} = \underbrace{-\frac{1}{|M|}\sum_{i\in M}\log \hat{y}_i}_{\mathcal{L}_{\text{MLM}}} + \underbrace{-\log \hat{y}_{\text{CLS}}}_{\mathcal{L}_{\text{NSP}}}. \tag{27}$$

Research / Industry Insight

NSP was later shown to be **harmful or redundant**. RoBERTa (Liu et al., 2019) removed NSP entirely and got better downstream performance by simply pre-training MLM for longer on more data. SpanBERT replaced NSP with a span-prediction task. Today, production-grade encoder-only LLMs mostly skip NSP.

4.10 BERT's Three-Embedding Input

Because BERT processes sentence pairs and must distinguish which sentence a token belongs to, the input embedding for each position is the *sum* of three learned embeddings:

Embedding	Indexed by	Purpose
Token embedding E_T	token id ($\in \{1, \ldots, \lvert\mathcal{V}\rvert\}$)	word meaning
Segment embedding E_A / E_B	sentence A or sentence B	which side of `[SEP]`
Position embedding $E_0, E_1, \ldots, E_{511}$	position index	order in the sequence

The final input for each token:

$$E_{\text{input}}[i] = E_T[\text{tok}_i] + E_{A/B}[\text{seg}_i] + E_{\text{pos}}[i].$$

4.10.1 The two special structural tokens

- `[CLS]` (classification): always at position 0. Its final hidden state is used as a *sentence-pair* summary for NSP and for downstream classification.
- `[SEP]` (separator): marks the boundary between sentence A and sentence B, and also marks the end of the input.

Quiz Trap

Trap. "`[CLS]` is just like `[MASK]`: a placeholder token."
Reality. `[CLS]` is a structural token whose *final hidden state* is the sentence-level summary used for classification. `[MASK]` is a corruption token whose final hidden state is used to predict the *original* token. They are separate tokens with separate embeddings and separate roles.

4.11 BERT Architecture and Configurations

BERT is a stack of bidirectional Transformer encoder layers (Week 1's encoder!) with no causal masking.

Hyperparameter	BERT-base	BERT-large
Layers (n)	12	24
Hidden size (d_{model}, H)	768	1024
Attention heads (h)	12	16
Intermediate FFN size ($4H$)	3072	4096
Vocabulary size	30,522 (WordPiece)	30,522
Max context length (T)	512	512
Parameters	$\approx$ 110 M	$\approx$ 340 M
Pre-training corpus	BookCorpus (800M) + Wikipedia (2500M)	same

4.12 Counting BERT-base Parameters — The Famous 110M

4.12.1 Embedding layer

- Token embeddings: $30{,}000 \cdot 768 \approx 23.0$ M.

- Segment embeddings: $2 \cdot 768 = 1{,}536 \approx 0.002$ M (negligible).
- Position embeddings: $512 \cdot 768 \approx 0.39$ M.
- **Subtotal:** ≈ 23.4 **M.**

4.12.2 Per-layer attention + FFN

- Self-attention QKV: $3 \cdot 768 \cdot 64 \cdot 12 \approx 1.77$ M.
- Output projection W^O: $768 \cdot 768 \approx 0.59$ M.
- FFN: $2 \cdot 768 \cdot 3072 + (3072 + 768) \approx 4.72$ M.
- **Per layer:** ≈ 7 **M.**

4.12.3 Grand total

$23.4\text{M} + 12 \cdot 7\text{M} = 23.4 + 84 = 107.4\text{M}$.

The paper rounds to ~ 110M (the slight difference comes from the actual vocab size of 30,522, LayerNorm parameters, and the MLM head). **Memorise: BERT-base** ≈ 110**M, BERT-large** ≈ 340**M.**

Exam Tip

Mnemonic comparison. GPT-1 was ~ 117M and BERT-base was ~ 110M. Roughly the same size, but radically different objectives and capabilities. Scale alone is not destiny — the *objective* matters.

4.13 Adapting BERT to Downstream Tasks

There are two main strategies.

4.13.1 Strategy 1 — BERT as a Feature Extractor (Body Frozen)

1. Feed the input through pre-trained BERT.
2. Take the final hidden state at `[CLS]` as a *frozen* sentence vector.
3. Train any classical ML model (logistic regression, Naive Bayes, a small NN) on top of that frozen vector.

Pros. Fast, no GPU needed for the head, simple. **Cons.** Lower ceiling than fine-tuning.

4.13.2 Strategy 2 — Fine-tuning the Whole Model

1. Add a small randomly-initialised task-specific head (often just a linear layer).
2. Train *end-to-end* on the labelled task, with a small learning rate so the pre-trained weights drift slowly.
3. No masking of input tokens during fine-tuning — the model has already learned to handle non-`[MASK]` positions thanks to the 80/10/10 split.

Pros. Higher accuracy with fewer labelled examples. **Cons.** Needs GPU to fine-tune; updates millions of parameters.

4.13.3 Worked example: extractive question answering (SQuAD-style)

Task. Given a paragraph and a question, find the *span* (contiguous substring of the paragraph) that answers the question.

Input format.

`[CLS]` question tokens `[SEP]` paragraph tokens `[SEP]`

Output head. Two learnable vectors $S, E \in \mathbb{R}^{d_{\text{model}}}$ (start and end). For each paragraph position i with hidden state h_i:

$$s_i = \frac{\exp(S \cdot h_i)}{\sum_j \exp(S \cdot h_j)} \quad \text{(probability that } i \text{ is the start token)}, \tag{28}$$

$$e_i = \frac{\exp(E \cdot h_i)}{\sum_j \exp(E \cdot h_j)} \quad \text{(probability that } i \text{ is the end token)}. \tag{29}$$

At inference, the predicted answer span is $[i^*, j^*]$ with $i^* = \arg\max_i s_i$, $j^* = \arg\max_{j \geq i^*} e_j$, subject to $j \geq i$.

Edge case. If $j^* < i^*$ (i.e., the most likely end is before the most likely start), return the empty string — this typically signals "the answer is not in the passage".

Intuition

This is a beautifully simple example of how to adapt BERT: *add two learnable vectors* S, E (a couple of thousand parameters), inherit the rest from pre-training, fine-tune. With SQuAD-1.1, BERT-large reaches superhuman F1, beating many task-specific architectures designed before it.

4.14 GPT vs BERT — The Big Comparison

	GPT-1	**BERT-base**
Architecture	Decoder-only Transformer	Encoder-only Transformer
Attention	Masked (causal) self-attention	Bidirectional self-attention
Pre-training objective	Causal LM (predict next token)	MLM + NSP
Context direction	left-to-right only	both directions
Output for classification	last token hidden state	`[CLS]` hidden state
Best for	generation, dialogue, completion	understanding, classification, span tasks
Parameters	$\approx$ 117M	$\approx$ 110M
Pre-training data	BookCorpus (800M words)	BookCorpus + Wikipedia (3300M words)
Year	June 2018	October 2018

Remember This

When to use which?

- Need to *generate* text? GPT-style (decoder-only).
- Need a *sentence embedding* for classification or retrieval? BERT-style (encoder-only).
- Need to *transform* one sequence into another (translation, summarisation)? Encoder-decoder (T5, BART) is a natural fit, though modern decoder-only LLMs handle these too.

4.15 Common Mistakes & Quiz Traps — Week 4

Common Mistake

Mistake 1. "Greedy search always produces the most probable sequence."
Fix. It produces the sequence that is *locally* most probable at each step. The globally most probable sequence may require an early choice that is not locally maximal.

Common Mistake

Mistake 2. "Beam search with $k = 1$ gives diverse outputs."
Fix. $k = 1$ *is* greedy search — fully deterministic.

Common Mistake

Mistake 3. "Top-$p = 1$ samples from the full distribution."
Fix. Top-$p = 1$ keeps all tokens whose cumulative probability is below 1. Since all probabilities sum to 1, this keeps the entire vocabulary — equivalent to plain ancestral sampling (without any truncation).

Common Mistake

Mistake 4. "BERT uses causal masking like GPT, but with a different vocabulary."
Fix. BERT uses *bidirectional* attention. The only "mask" in BERT is the `[MASK]` input token, not an attention mask.

Quiz Trap

Trap. "If we mask 100% of tokens, MLM would be ideal because it gives the strongest gradient."
Reality. With 100% masking, the model sees no input context and is asked to "predict everything from nothing" — a useless task. Empirically, $\sim 15\%$ is the sweet spot.

Quiz Trap

Trap. "Lower temperature always improves text quality."
Reality. Low T kills creativity and induces repetition. High T injects noise and incoherence. The right T depends on the task; ChatGPT defaults to around 1 and lowers it for code/factual queries.

4.16 Memory Tricks — Week 4

Memory Trick

"E-G-B-K-T-N-C" — the decoding ladder from most to most flexible: **E**xhaustive, **G**reedy, **B**eam, top-**k**, **T**emperature, **N**ucleus (top-p), **C**ontrastive.

Memory Trick

"T down = peaked, T up = flat".
$T \to 0$ behaves like greedy (everything goes to the argmax); $T \to \infty$ becomes uniform random.

Memory Trick

"15% : 80 / 10 / 10".
Mask 15% of tokens; of those, 80% become `[MASK]`, 10% become random tokens, 10% are unchanged.

Memory Trick

"BERT = CLS + SEP + MASK"
Three special tokens: `[CLS]` for sentence-level summary, `[SEP]` for sentence boundary, `[MASK]` for the prediction target.

4.17 Quick Revision — Week 4 in One Page

Quick Revision

Decoding strategies (deterministic).
- **Exhaustive:** optimal but $\mathcal{O}(|\mathcal{V}|^T)$, infeasible.
- **Greedy:** argmax at every step. $k = 1$ beam search.
- **Beam:** keep top-k partial sequences. Length-normalise to prevent bias toward short outputs.

Decoding strategies (stochastic).
- **Top-k:** sample from top-k tokens after renormalising.
- **Temperature T:** softmax(u/T). Lower = sharper, higher = flatter.
- **Top-p (nucleus):** keep smallest set with cumulative probability $\geq p$. Dynamic, adaptive.

BERT.
- Encoder-only Transformer; bidirectional self-attention.
- **MLM** objective: mask 15% of tokens (80 / 10 / 10 split); predict originals.
- **NSP** objective: predict if sentence B follows sentence A.
- Three embeddings summed at input: token + segment (E_A/E_B) + position.
- Specials: `[CLS]` (classification), `[SEP]` (separator).
- **BERT-base:** 12 layers, $H = 768$, $h = 12$, $\sim$ 110M parameters.
- Downstream: feature-extractor or fine-tune; QA uses learned start/end vectors S, E.

Both GPT and BERT are $\sim$ 110M parameters but with opposite design philosophies (generation vs. understanding).

4.18 Practice Questions — Week 4

Multiple Choice

1. **(Q1)** Greedy decoding is equivalent to beam search with:

   (A) $k = 0$ (B) $k = 1$ (C) $k = |\mathcal{V}|$ (D) any odd k.

   Solution. (B). With $k = 1$, only the single best partial sequence is kept — which is exactly greedy.

2. **(Q2)** For open-ended text generation, the most commonly recommended decoding setting is:

   (A) Beam search with large k,

   (B) Greedy decoding,

   (C) Temperature ≈ 1 with top-$p \in [0.9, 0.95]$,

(D) Random sampling with $T = 100$.

Solution. (C). Open-ended generation benefits from controlled stochasticity; (A) gives bland generic text, (B) repeats, (D) is incoherent.

3. **(Q3)** The percentage of tokens masked in BERT's MLM pre-training is:

   (A) 5% (B) 15% (C) 30% (D) 50%.

   Solution. (B).

4. **(Q4)** In BERT, the `[CLS]` token's final hidden state is used for:

   (A) predicting the masked tokens,

   (B) sentence-level classification (e.g., NSP, downstream sentiment),

   (C) rotational position encoding,

   (D) separating sentence A from sentence B.

   Solution. (B). (D) is the `[SEP]` token's job.

5. **(Q5)** Of the masked tokens in BERT MLM, the fraction replaced with a random vocabulary token is:

   (A) 80% (B) 10% (C) 50% (D) 1%.

   Solution. (B). Recall the 80/10/10 split: 80% `[MASK]`, 10% random, 10% unchanged.

6. **(Q6)** Which of these is *deterministic* (same output for same prompt with no randomness)?

   (A) Top-k sampling,

   (B) Nucleus (top-p) sampling,

   (C) Beam search,

   (D) Temperature sampling.

   Solution. (C). Beam search is deterministic. (A), (B), (D) all introduce randomness through sampling.

Conceptual / Short Answer

7. **(Q7)** Explain in 2–3 sentences why beam search prefers shorter sequences unless length normalisation is applied.

   Answer. Each multiplication of a conditional probability (≤ 1) shrinks the joint. Longer sequences accumulate more such factors and end up with smaller joint probabilities — so beam search prematurely terminates. Length normalisation (dividing the log-score by sequence length or a length penalty ℓ^{α}) corrects this bias.

8. **(Q8)** Why is top-p (nucleus) usually preferred over top-k?

   Answer. Because top-p's set size is *adaptive*: for flat distributions it keeps many tokens (preserving creativity), and for peaked distributions it keeps few (preserving coherence). Top-k uses the same k for both regimes and fails on at least one of them.

9. **(Q9)** The 80/10/10 split in BERT's MLM exists because:

   Answer. Always replacing with `[MASK]` would cause a train/inference mismatch: the model would only learn to predict at `[MASK]` positions, but downstream tasks have no `[MASK]` tokens.

The 10% random and 10% unchanged tokens force the model to make predictions even at "normal" positions, reducing the gap between pre-training and fine-tuning.

10. **(Q10)** Could we use a decoder-only model (like GPT) for fill-in-the-blank? Why or why not?

    Answer. Not natively. GPT's causal mask blocks attention to the future, so at the blank position the model cannot use right-side context — it only sees the left. For fill-in-the-blank we need bidirectional context, which is exactly what BERT provides.

11. **(Q11)** For extractive QA with BERT, why do we need *two* learnable vectors S and E rather than one?

    Answer. We need to identify two distinct positions in the paragraph: the start of the answer span and the end. The dot products $S \cdot h_i$ and $E \cdot h_i$ let us score each position separately as a start candidate or end candidate. Both distributions are softmaxed across paragraph positions independently.

Numerical / Derivation

12. **(Q12)** A model has output probabilities $P = (0.5, 0.3, 0.15, 0.05)$ for the four candidate next tokens (in descending order). Compute the kept set for (a) top-k with $k = 2$, and (b) top-p with $p = 0.7$.

    Solution. (a) Top-2 $= \{0.5, 0.3\}$, renormalised to $\{0.625, 0.375\}$. (b) Cumulative: 0.5, 0.8, 0.95, 1.0. The first cumulative sum to exceed 0.7 is 0.8, so the nucleus is the first two tokens $\{0.5, 0.3\}$, renormalised to $\{0.625, 0.375\}$. In this case top-2 and top-p at $p = 0.7$ coincide.

13. **(Q13)** Verify the BERT-base parameter count starting from $H = 768$, $h = 12$, $n = 12$ layers, $|\mathcal{V}| = 30{,}000$, max length 512.

    Solution.

    - Embeddings: $30{,}000 \cdot 768 + 512 \cdot 768 + 2 \cdot 768 \approx 23.4$ M.
    - Attention per layer: $3 \cdot 768 \cdot 64 \cdot 12 + 768 \cdot 768 \approx 1.77 + 0.59 = 2.36$ M.
    - FFN per layer: $2 \cdot 768 \cdot 3072 + 3072 + 768 \approx 4.72$ M.
    - Total per layer: ≈ 7.08 M; for 12 layers: ≈ 85 M.
    - Grand total: $23.4 + 85 \approx 108$ M. Paper reports ≈ 110M (the difference is LayerNorm and MLM-head parameters).

14. **(Q14)** Suppose at $t = 1$ the logits are $u = (4.0, 3.0, 2.0, 1.0)$. Compute the probabilities under (a) $T = 1$ and (b) $T = 0.5$.

    Solution. (a) $\exp(u) = (54.6, 20.1, 7.4, 2.7)$, sum ≈ 84.8. Probabilities $\approx (0.644, 0.237, 0.087, 0.032)$. (b) $u/T = (8, 6, 4, 2)$, exp $\approx (2981, 403, 54.6, 7.4)$, sum ≈ 3446. Probabilities $\approx (0.865, 0.117, 0.016, 0.002)$. **Note how lower T has sharpened the distribution onto the top choice.**

15. **(Q15)** A beam search with $k = 4$, $|\mathcal{V}| = 50{,}000$, and length $T = 20$. Compare its cost (in score evaluations) with greedy.

    Solution.

    - Greedy: $|\mathcal{V}| \cdot T = 50{,}000 \cdot 20 = 10^6$ evaluations.
    - Beam: $k \cdot |\mathcal{V}| \cdot T = 4 \cdot 50{,}000 \cdot 20 = 4 \times 10^6$.
    - Beam costs $4\times$ more in score computation, plus k parallel forward passes through the model.

5 Week 5 — Tokenizers: BPE, WordPiece, and SentencePiece

Chapter goals. By the end of this chapter you will be able to:
1. Place tokenization in the overall LLM pipeline.
2. Articulate the trade-offs between character-, word-, and sub-word level tokenisation.
3. Implement (or trace by hand) the **Byte-Pair Encoding (BPE)** merge algorithm.
4. Distinguish **WordPiece** from BPE via the likelihood-based score.
5. Derive **SentencePiece** / **Unigram LM** tokenisation and walk through a Viterbi decoding example.
6. Choose the right tokenizer for a given language and task.

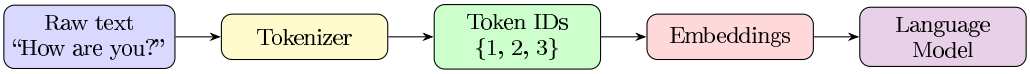
5.1 Why Tokenization Matters

Recall the language model objective:

$$P(x_1, x_2, \ldots, x_T) \;=\; P(x_1) \prod_{i=2}^{T} P(x_i \mid x_1, \ldots, x_{i-1}).$$

For this to be well-defined, we need to know what an x_i *is*: a character? a word? a sub-word? *Tokenization* is the procedure that turns raw text into discrete tokens that the model embeds.

5.1.1 The pipeline



The tokenizer sits at the very front (and also at the very back, where token IDs are converted back to text). The choice of tokenizer affects vocabulary size, embedding matrix size, OOV behaviour, and multilingual capability.

5.1.2 Five challenges in building a vocabulary

- **Vocabulary size.** Larger $|\mathcal{V}|$ ⇒ larger embedding matrix ($|\mathcal{V}| \cdot d_{\text{model}}$) and slower softmax over the vocabulary. What is the sweet spot?
- **Out-of-vocabulary (OOV) words.** If we cap $|\mathcal{V}|$ at 50K, what happens to "Sesquipedalophobia" or a person's surname we have never seen?
- **Misspellings.** A web-scraped corpus contains "thier" alongside "their". We should not bloat the vocab with typos.
- **Open vocabulary problem.** Agglutinative languages (Turkish, Finnish, Tamil) freely build new words by combining morphemes. Numbers and proper names are essentially infinite. The vocab is in principle unbounded.
- **Languages without word delimiters.** Japanese, Chinese, and Korean do not use whitespace between words.

Intuition

The dream tokenizer is **moderate-sized, robust to unknown words**, and **language-agnostic**. Sub-word tokenisation hits all three.

5.2 Three Granularities of Tokenization

Granularity	What is a token?	Pros / Cons	Algorithms
Character-level	Each Unicode character	Tiny vocabulary ($\sim$ 100); no OOV. But sequences become very long and each char carries little semantic info.	char-LM
Word-level	Each whitespace-separated word	Each token is meaningful; short sequences. But huge vocab ($>$ 250K); OOV problem; fails for Japanese.	whitespace, NLTK
Sub-word	Frequent words intact; rare words split into pieces	Moderate vocab; no OOV; language-agnostic	**BPE, WordPiece, SentencePiece**

Toy example. The sentence *"Hmm.., I know, I don't know"*:

- Whitespace tokenizer: `[Hmm.., I, know, I, don't, know]` – 6 tokens, vocab 4.
- Sub-word tokenizer: `[Hmm.., I, know, do, n't]` – the apostrophe-containing word "don't" is split into "do" + "n't" because "n't" is a recurring suffix. Vocab shrinks to 5 unique pieces (Hmm.., I, know, do, n't) covering more words.

Remember This

Sub-word tokenizers preserve frequent words intact and split rare words into pieces that themselves are frequent. This balances vocabulary size against sequence length. **Every modern LLM uses sub-word tokenization.**

5.3 Pre-processing: Normalisation and Pre-tokenisation

Before the sub-word algorithm runs, two clean-up steps happen:

1. **Normalisation.** Lowercase, strip accents, remove duplicate whitespace, strip HTML, fix Unicode. Example: "Hmm.., I know" $\rightarrow$ "hmm.., i know".
2. **Pre-tokenisation.** Split on whitespace (for space-using languages). The sub-word algorithm then operates on these pre-tokens.

Intuition

Pre-tokenisation is what we used to just call "tokenization". With the rise of sub-word algorithms, the term shifted: now "tokenisation" means the full pre-process + sub-word merge, and "pre-tokenisation" is the cheap whitespace split that comes first.

5.4 Byte-Pair Encoding (BPE)

BPE was originally a 1994 data-compression algorithm (Gage), repurposed for NLP by Sennrich et al. (2016) for neural machine translation. It is the dominant tokenizer for the GPT family.

Definition

Byte-Pair Encoding (BPE) starts from a character-level vocabulary, then iteratively merges the most frequent *adjacent pair* of tokens in the corpus into a single new token, repeating N times where N is a hyperparameter. The final vocabulary is the original characters plus the N learned merges.

5.4.1 Algorithm in pseudocode

```python
import re, collections

def get_stats(vocab):
    pairs = collections.defaultdict(int)
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols) - 1):
            pairs[symbols[i], symbols[i+1]] += freq
    return pairs

def merge_vocab(pair, v_in):
    v_out = {}
    bigram = re.escape(' '.join(pair))
    p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
    for word in v_in:
        w_out = p.sub(''.join(pair), word)
        v_out[w_out] = v_in[word]
    return v_out

vocab = {'l o w </w>': 5, 'l o w e r </w>': 2,
         'n e w e s t </w>': 6, 'w i d e s t </w>': 3}

num_merges = 10
for i in range(num_merges):
    pairs = get_stats(vocab)
    best = max(pairs, key=pairs.get)
    vocab = merge_vocab(best, vocab)
    print(best)
```

Listing 1: Reference BPE training loop.

5.4.2 Walking through an example

Corpus. *"knowing the name of something is different from knowing something. knowing something about everything isn't bad."*

Step 1 — Word frequencies (after appending word-boundary marker </w>). `</w>` tells the tokenizer where a word ends, so that "in" inside "knowing" is not confused with the standalone word "in".

Word	Freq	Word	Freq
k n o w i n g </w>	3	f r o m </w>	1
t h e </w>	1	s o m e t h i n g . </w>	1
n a m e </w>	1	a b o u t </w>	1
o f </w>	1	e v e r y t h i n g </w>	1
s o m e t h i n g </w>	2	i s n ' t </w>	1
i s </w>	1	b a d </w>	1
d i f f e r e n t </w>	1		

Step 2 — Initial character vocabulary. List each character with its frequency (count of occurrence across the entire corpus, weighted by word frequency). For example: `k` appears in "knowing" (freq 3), so count(`k`) = 3. Likewise count(`n`) = 13, count(`o`) = 9, and count(`</w>`) = 16 (sum of all word frequencies). Initial vocab size is 22.

Step 3 — Count adjacent pairs.

Pair	Freq	Pair	Freq
('k','n')	3	('i','n')	**7**
('n','o')	3	('n','g')	**7**
('o','w')	3	('g','</w>')	6
('w','i')	3	('t','h')	5
('h','e')	1	('e','</w>')	2

Step 4 — Merge the most frequent pair. Tie between `('i','n')` and `('n','g')`. Picking `('i','n')` (tie-break by lexicographic order):

- Add new token `in` with count 7.
- Decrement old characters: count(i) $= 10 - 7 = 3$, count(n) $= 13 - 7 = 6$.
- Update word representations: `k n o w in g </w>`.

Step 5 — Repeat. Recompute pair counts. The new most-frequent pair is `('in','g')` (still 7). Merge:

- Add token `ing` with count 7.
- Words containing "ing" now use this single token.

5.4.3 After many merges

After $\sim$ 45 merges, frequent units like `ing`, `the`, `ed`, `ly` become tokens. The vocabulary $\mathcal{V}$ contains both single characters *and* learned merges, in the order they were created. Rare words (e.g., "Sesquipedalophobia") still decompose into their constituent characters — no OOV.

5.4.4 BPE at inference time

Intuition

At test time, the input word is split into characters, and the algorithm applies the learned merges *in order of priority* (= order of creation). For instance, with $\mathcal{V} = \{a, b, c, \ldots, z, \texttt{lo}, \texttt{he}\}$ and the merge order "`lo` first, then `he`":

- Input "hello" $\rightarrow$ characters `[h, e, l, l, o]`.
- Apply "`lo`" merge $\rightarrow$ `[h, e, l, lo]`.
- Apply "`he`" merge $\rightarrow$ `[he, l, lo]`.
- Output: `he #l #lo` (`#` marks a non-initial sub-word).

5.4.5 BPE for non-segmented languages

BPE requires whitespace-separated words to apply pre-tokenisation. For Japanese / Korean / Chinese, you must first use a language-specific morphological segmenter (e.g., *Juman++* or *MeCab* for Japanese) to insert "word" boundaries, *then* apply BPE on top. This is awkward in multilingual settings, which motivates SentencePiece below.

Quiz Trap

Trap. "BPE always produces a unique tokenisation for a given word."
Reality. BPE is *greedy and deterministic*: given a fixed vocab and merge order, the output is unique. But *different* vocabularies (different merge orders) can yield different tokenisations of the same word. Researchers exploit this with *BPE-Dropout*, a stochastic variant.

5.5 WordPiece — BPE's Likelihood-Based Cousin

WordPiece was introduced by Schuster & Nakajima (2012) for Japanese / Korean speech recognition, and is the tokenizer used by **BERT.**

Definition

WordPiece is similar to BPE except that the merge criterion is *not* the raw pair frequency. Instead, the pair (α, β) with the highest **likelihood-based score** is merged:

$$\text{score}(\alpha, \beta) = \frac{\text{count}(\alpha, \beta)}{\text{count}(\alpha) \cdot \text{count}(\beta)}.$$

Formula Insight

Why this score makes sense. The numerator favours frequent pairs (as in BPE). The denominator *penalises* pairs whose individual tokens are already very frequent. Thus WordPiece is biased toward merging *rare* characters into larger sub-words, even if a frequent-character pair has a higher raw co-occurrence.
This is essentially the *pointwise mutual information* (PMI) statistic, beloved of classical NLP.

5.5.1 Worked WordPiece scoring on the same corpus

For the "knowing…" corpus, the score table becomes:

Pair	count(α)	count(β)	score
`(k, n)`	3	13	0.076
`(n, o)`	13	9	0.020
`(o, w)`	9	3	0.111
`(i, n)`	10	13	0.050
`(n, g)`	13	7	0.076
`(t, h)`	8	5	0.125
`(a, d)`	3	2	**0.160**

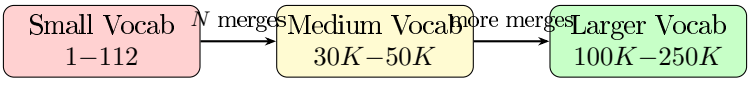
Observation. BPE would merge `(i,n)` first (frequency 7), but WordPiece merges `(a,d)` first (score 0.16) because `a` and `d` are both rare, making their co-occurrence statistically more "meaningful".

5.5.2 WordPiece notation

WordPiece marks non-initial sub-pieces with the prefix `##`. Thus "playing" might become `play ##ing`, where the double-hash signals "glue this to the previous piece without inserting a space". This is in contrast to BPE which uses the end-of-word marker `</w>`.

5.5.3 Vocab-size dial

Both BPE and WordPiece start from a small base ($\sim$ 100 characters) and *add merges* until a target vocab size (typically 30K–50K) is reached. The size is a hyperparameter.



5.6 SentencePiece / Unigram LM — The Probabilistic Approach

BPE and WordPiece both *merge* from below. SentencePiece (Kudo & Richardson, 2018), which implements the Unigram Language Model algorithm (Kudo, 2018), does the opposite: it starts with a large seed vocabulary and *prunes* downward, keeping the most useful sub-words.

Definition

Unigram LM Tokenization models each sub-word x_i as drawn i.i.d. from a distribution $p(x_i)$. The probability of a segmentation $\mathbf{x} = (x_1, \ldots, x_n)$ of a string X is

$$P(\mathbf{x}) = \prod_{i=1}^{n} p(x_i).$$

The optimal segmentation maximises this likelihood:

$$\mathbf{x}^* = \arg\max_{\mathbf{x} \in S(X)} P(\mathbf{x}),$$

where $S(X)$ is the set of all possible segmentations of X.

5.6.1 Multiple segmentations of a single word

Intuition

With vocab $\{h, e, l, o, \texttt{he}, \texttt{el}, \texttt{lo}, \texttt{ll}, \texttt{hell}, \texttt{ello}\}$, the word "hello" admits many segmentations:

- $\mathbf{x}_1 = (\texttt{he}, \texttt{ll}, \texttt{o})$
- $\mathbf{x}_2 = (\texttt{h}, \texttt{el}, \texttt{lo})$
- $\mathbf{x}_3 = (\texttt{he}, \texttt{l}, \texttt{lo})$
- $\mathbf{x}_4 = (\texttt{hell}, \texttt{o})$
- ...

The Unigram LM picks the segmentation maximising $\prod p(x_i)$. BPE, by contrast, picks *whichever segmentation its greedy merge order produces* — there is no probabilistic guarantee of optimality.

5.6.2 Worked example: segmenting "knowing"

Given some vocabulary that contains `k`, `now`, `ing`, `know`, `knowing` with counts proportional to:

- Candidate $\mathbf{x}_1 = (\texttt{k, now, ing})$: $p(\mathbf{x}_1) = p(k)\,p(\texttt{now})\,p(\texttt{ing}) = \frac{3}{163} \cdot \frac{3}{163} \cdot \frac{7}{163} = \frac{63}{4096}$.
- Candidate $\mathbf{x}_2 = (\texttt{know, ing})$: $p(\mathbf{x}_2) = p(\texttt{know})\,p(\texttt{ing}) = \frac{21}{256} \approx \frac{336}{4096}$.
- Candidate $\mathbf{x}_3 = (\texttt{knowing})$: $p(\mathbf{x}_3) = p(\texttt{knowing}) = \frac{1}{163} \approx \frac{768}{4096}$.

The winner is $\mathbf{x}_3$ – a *single* token for "knowing".

Remember This

The Unigram LM favours the segmentation with the *fewest sub-words* (because more pieces = more multiplicative penalties from probabilities < 1). This is a feature: it prefers "whole-word" tokens when they exist in the vocab.

5.6.3 Training algorithm (EM)

1. Construct a **seed vocabulary** $\mathcal{V}_0$ using BPE or an extended suffix-array algorithm. This $\mathcal{V}_0$ is intentionally *larger* than the final target.

2. **E-step:** Using current $p(x)$ estimates, segment the corpus with Viterbi decoding to find optimal segmentations.
3. **M-step:** Re-estimate $p(x)$ from the Viterbi segmentation frequencies.
4. Compute the corpus log-likelihood $\mathcal{L} = \sum_{X \in \mathcal{D}} \log \sum_{\mathbf{x} \in S(X)} P(\mathbf{x})$.
5. **Prune.** Remove the bottom $x\%$ of sub-words by likelihood contribution.
6. Repeat 2–5 until vocabulary reaches target size.

5.6.4 Viterbi decoding for the best segmentation

Setup. Given a unigram log-probability table:

Token	$\log p(x)$	Token	$\log p(x)$
b	-4.70	wh	-5.99
e	-2.70	er	-5.34
h	-3.34	where	-8.21
r	-3.36	by	-7.34
w	-4.29	he	-6.02

We segment "whereby". At each position t we track the best (highest log-likelihood) segmentation ending at t.

Position 1 ($t = 1$, prefix "w"): only one option, $\log p = -4.29$.

Position 2 ($t = 2$, prefix "wh"): two options.

- $(\texttt{w}, \texttt{h})$: $-4.29 + (-3.34) = -7.63$.
- $(\texttt{wh})$: -5.99.

Keep -5.99 (whole-word "wh"). *Note: by Viterbi's principle of optimality, we discard the "w, h" path going forward.*

Position 3 ($t = 3$, prefix "whe"):

- Extend best at $t = 2$ by 1 char: $(\texttt{wh}, \texttt{e}) : -5.99 + (-2.70) = -8.69$.
- Extend best at $t = 1$ by 2 chars: $(\texttt{w}, \texttt{he}) : -4.29 + (-6.02) = -10.31$.
- Whole 3-char "whe": not in vocab $\to -\infty$.

Keep -8.69 at $(\texttt{wh}, \texttt{e})$.

Position 5 ($t = 5$, prefix "where"):

- Whole 5-char "where" (in vocab): -8.21.
- Other combinations: all worse than -8.21.

Keep -8.21 at $(\texttt{where})$.

Position 7 ($t = 7$, prefix "whereby"):

- Extend best at $t = 5$ by "by" (2 chars): $-8.21 + (-7.34) = -15.55$.
- Extend best at $t = 6$ by "y" (1 char): *worse*.

Best final score: -15.55 **via segmentation** $(\texttt{where}, \texttt{by})$.

Backtrack the pointer chain: `where` at $t = 5$ came from start, then `by` extends to $t = 7$. So the answer is `[where, by]`.

Intuition

Why Viterbi instead of brute force? Brute force enumerates all $\mathcal{O}(2^{T-1})$ segmentations of a length-T string. Viterbi, by tracking only the best path ending at each position, runs in $\mathcal{O}(T^2)$. The same trick that makes HMM speech recognition tractable.

5.6.5 SentencePiece's killer feature: language-agnostic

SentencePiece treats the input as a *raw byte/character stream* (a "sentence piece"), without requiring pre-tokenisation. Whitespace itself becomes a token (usually represented as ▁, the meta-symbol). This makes SentencePiece **ideal for multilingual models** and especially for languages without word delimiters (Japanese, Chinese, Korean).

Research / Industry Insight

Tokenizer used by...
- **GPT-2, GPT-3, GPT-4:** byte-level BPE (operates on UTF-8 bytes, not chars).
- **BERT, DistilBERT:** WordPiece.
- **T5, mT5, BART, ALBERT, XLM-R:** SentencePiece (Unigram LM).
- **LLaMA, Mistral:** SentencePiece (BPE algorithm under the hood).
- **Claude, Gemini:** proprietary, but most modern LLMs use BPE-style sub-word.

The dominant trend is BPE (often byte-level), because it handles *any* Unicode input gracefully without a separate normaliser.

5.6.6 Morfessor (brief mention)

Morfessor (Creutz & Lagus, 2002) uses Maximum A Posteriori (MAP) estimation rather than MLE, allowing linguistic prior knowledge to be injected. It also accepts a small amount of *annotated* morphological data during training. Useful for morphologically rich languages and large-vocabulary continuous speech recognition.

5.7 The Big Comparison: BPE vs WordPiece vs SentencePiece

	BPE	**WordPiece**	**SentencePiece / Unigram LM**
Direction	bottom-up merge	bottom-up merge	top-down prune
Merge criterion	raw pair freq.	PMI score	log-likelihood contribution
Stopping criterion	N merges (hyperparam)	target vocab size	target vocab size
Determinism	yes (greedy)	yes (greedy)	yes via Viterbi; supports stochastic via subword regularisation
Needs pre-tokenisation?	yes	yes	no – treats whitespace as token
Multilingual / non-segmented	weak (needs MeCab etc.)	weak	strong
Used by	GPT family, RoBERTa, LLaMA	BERT, DistilBERT, ELECTRA	T5, ALBERT, XLM-R, mBART
Notation for sub-pieces	`</w>` marks word end	`##piece`	`_piece` (_ = space)

5.8 Common Mistakes & Quiz Traps — Week 5

Common Mistake

Mistake 1. "BPE and WordPiece produce the same tokenisation."
Fix. They use the same merge *procedure* but different *criteria* (raw frequency vs. PMI score). The resulting vocabularies differ; the same word may be tokenised differently.

Common Mistake

Mistake 2. "Sub-word tokenisation eliminates OOV completely."
Fix. Sub-word eliminates OOV at the *word* level by falling back to characters. But if the test text contains characters not seen during training (e.g., emojis or rare Unicode), they are still OOV. Byte-level BPE fixes this by tokenising at the byte level.

Common Mistake

Mistake 3. "Unigram LM tokeniser is non-deterministic."
Fix. Standard Unigram LM uses Viterbi to pick the maximum-likelihood segmentation — deterministic. The *Subword Regularization* variant samples from the top-k segmentations probabilistically.

Quiz Trap

Trap. "A larger BPE vocabulary always makes the model better."
Reality. A larger vocab shortens sequences (good) but blows up the embedding matrix and the softmax (bad). It also creates more rarely-seen tokens, which the model struggles to learn good embeddings for. Empirically, 32K–64K is a strong sweet spot for monolingual; up to 250K for highly multilingual (mT5).

Quiz Trap

Trap. "SentencePiece is the same algorithm as Unigram LM."
Reality. SentencePiece is a *library* that implements both BPE and Unigram LM (the latter is the default). Confusingly, "SentencePiece tokenizer" in casual usage often means "SentencePiece's Unigram LM mode". But you can configure SentencePiece to do BPE too.

5.9 Memory Tricks — Week 5

Memory Trick

"BPE merges by FREQ, WordPiece by PMI, Unigram by LIKELIHOOD."
Three criteria, three algorithms. The merge-step formula is the only real difference between BPE and WordPiece.

Memory Trick

"Build up (BPE/WP) or chip away (SentencePiece)."
BPE and WordPiece grow the vocabulary from small to medium. SentencePiece / Unigram LM starts with a large seed and prunes downward.

Memory Trick

"## for BERT, $</w>$ for BPE, _ for SentencePiece."
Three different conventions for marking sub-word boundaries.

Memory Trick

"Sub-word = no OOV at the word level."
Even "Sesquipedalophobia" decomposes into pieces the model has seen. Out-of-vocabulary problems essentially vanish.

5.10 Practical Applications

- **Multilingual models** (mBERT, XLM-R, mT5) use SentencePiece with a single shared vocabulary across 100+ languages.
- **Code models** (Codex, StarCoder, Code Llama) use BPE adapted to code: common tokens are `def`, `return`, `import`, etc.
- **Domain-specific models** (BioBERT, FinBERT) sometimes extend the base vocabulary with domain-specific sub-words for better efficiency.
- **Tiktoken** (OpenAI's tokenizer): a hyper-optimised byte-level BPE implementation. Open-source.

5.11 Quick Revision — Week 5 in One Page

Quick Revision

Pipeline. text → normalise → pre-tokenise → *sub-word tokenize* → token IDs → embeddings → model.
Three granularities. character (tiny vocab, long seq) / word (big vocab, OOV) / *sub-word* (balanced, no OOV).
BPE. Start from chars; iteratively merge the most frequent pair; stop after N merges. Greedy and deterministic. Marker: `</w>`.
WordPiece. Same as BPE but merge by score $= \frac{\text{count}(\alpha\beta)}{\text{count}(\alpha)\cdot\text{count}(\beta)}$. Marker: `##`. Used by BERT.
Unigram LM / SentencePiece. Seed with large vocab; EM + Viterbi; prune low-likelihood sub-words. Treats whitespace as a token (_); language-agnostic; used by T5, LLaMA-family. Best segmentation maximises $\prod_i p(x_i)$.
Worked Viterbi on "whereby". Best segmentation = `(where, by)` with log-likelihood -15.55.
Modern landscape. GPT/LLaMA: BPE (often byte-level). BERT: WordPiece. T5/multilingual: SentencePiece. Vocab sizes 32K–256K depending on language coverage.

5.12 Practice Questions — Week 5

Multiple Choice

1. **(Q1)** The BPE algorithm at each step merges:

   (A) the lexicographically first pair;

   (B) the most frequent adjacent pair;

   (C) the pair with highest pointwise mutual information;

(D) the pair with lowest variance.

Solution. (B).

2. **(Q2)** The WordPiece score for a pair (α, β) is:

   (A) $\text{count}(\alpha) + \text{count}(\beta)$;

   (B) $\text{count}(\alpha)\,\text{count}(\beta)/\text{count}(\alpha, \beta)$;

   (C) $\text{count}(\alpha, \beta)/(\text{count}(\alpha)\,\text{count}(\beta))$;

   (D) the cross-entropy between α and β.

   Solution. (C).

3. **(Q3)** Which tokenizer is used by BERT?

   (A) BPE (B) WordPiece (C) SentencePiece Unigram (D) character-level.

   Solution. (B).

4. **(Q4)** Which tokenizer treats whitespace as a regular token and thus does not require pre-tokenisation?

   (A) BPE (B) WordPiece (C) SentencePiece (D) char-level.

   Solution. (C).

5. **(Q5)** Unigram LM tokenisation, given competing segmentations, typically prefers:

   (A) the segmentation with the most pieces;

   (B) the segmentation with the fewest pieces (when whole-word tokens exist);

   (C) the alphabetically first segmentation;

   (D) all segmentations equally.

   Solution. (B). More pieces = more < 1 multiplications = lower probability.

Conceptual / Short Answer

6. **(Q6)** Why is character-level tokenisation rarely used for LLMs?

   Answer. Each character carries little semantic information, sequences are much longer (so T grows and self-attention's T^2 cost explodes), and the model must learn to compose characters into meanings — adding burden. Sub-word strikes a much better balance.

7. **(Q7)** What does the `</w>` marker in BPE accomplish?

   Answer. It marks the end of a word so that a sub-word inside a word (e.g. "in" in "knowing") is distinct from the same letters appearing at the end of a word. Without `</w>`, "in" (preposition) and "-in" (suffix in "raisin") would collide.

8. **(Q8)** Explain in one sentence why BPE-Dropout was invented.

   Answer. BPE is deterministic; randomly dropping some merges during training exposes the model to multiple segmentations of the same word, acting as a data augmentation that improves robustness (Provilkov et al., 2020).

9. **(Q9)** A vocabulary contains `{a, b, c, ..., z, lo, he}`. Tokenise "hello" with BPE (assume `lo` was merged before `he`).

Answer. Start: `[h, e, l, l, o]`. Apply `lo`: `[h, e, l, lo]`. Apply `he`: `[he, l, lo]`. Output (with `#` sub-word marker): `he #l #lo`.

10. **(Q10)** Why is SentencePiece especially suited to multilingual models?

    Answer. It treats raw input as a byte/character stream (no language-specific pre-tokenisation), and learns a single shared sub-word vocabulary across all languages. The same model can then tokenise English, Japanese, Hindi, etc., without language-specific code paths.

Numerical / Worked

11. **(Q11)** Toy corpus: *"the cat sat on the mat"*. List BPE's first merge.

    Solution. Word freqs: `the`:2, `cat`:1, `sat`:1, `on`:1, `mat`:1. After splitting into chars (with `</w>`) and counting pairs, the most frequent pair is `(t, h)` appearing in "the" twice. So the first merge is `th`.

12. **(Q12)** For the WordPiece score table given in the lecture, identify the merge that BPE would do first and the one WordPiece would do first.

    Solution. BPE: `(i, n)` or `(n, g)` at freq 7 (tie-break). WordPiece: `(a, d)` at score 0.16.

13. **(Q13)** Given the toy probabilities below, find the best Unigram-LM segmentation of "bad":

$$\log p(\texttt{b}) = -3, \quad \log p(\texttt{a}) = -2, \quad \log p(\texttt{d}) = -3, \quad \log p(\texttt{bad}) = -7.$$

    Solution.

    - (`b, a, d`) : $-3 + (-2) + (-3) = -8$.
    - (`bad`) : -7.

    The whole-word `bad` ($-7 > -8$) wins.

14. **(Q14)** A BPE tokenizer trained for N merges produces a vocabulary of size $|\mathcal{V}_0| + N$, where $|\mathcal{V}_0|$ is the initial character vocab. Starting from a 200-character base and training for 49,800 merges, what is the final vocab size?

    Solution. $200 + 49{,}800 = 50{,}000$. This is the GPT-2-style 50K vocab.

15. **(Q15)** A model has $|\mathcal{V}| = 50{,}000$ and $d_{\text{model}} = 4096$ (LLaMA-7B scale). How many parameters does the embedding layer alone contribute?

    Solution. $50{,}000 \cdot 4096 = 204{,}800{,}000 \approx 205$ M. At this scale, the embedding layer is huge — and weight tying with the output projection cuts the count in half (since otherwise the LM head would add another 205M).

6 Week 7 — Encoder-Decoder LLMs: BART, GPT-2, T5 and the Choices that Affect Performance

Chapter goals. By the end of this chapter you will be able to:

1. Explain **BART** — the denoising auto-encoder that combines BERT and GPT.
2. Articulate the **GPT-2** story: scaling + zero-shot prompting.
3. Reformulate *every* NLP task as "text-in / text-out" using **T5**.
4. Compare architectures (encoder-only / decoder-only / encoder-decoder) and objectives (CLM / MLM / Prefix-LM / span denoising) on equal compute.
5. Choose corruption strategies, rates, and span lengths based on the T5 study.
6. Decide how to spend extra compute: bigger model, longer training, larger batch, or ensembling.
7. Apply *adapter layers*, *gradual unfreezing*, and *multi-task learning* for parameter-efficient fine-tuning.

6.1 From BERT to BART — Best of Both Worlds

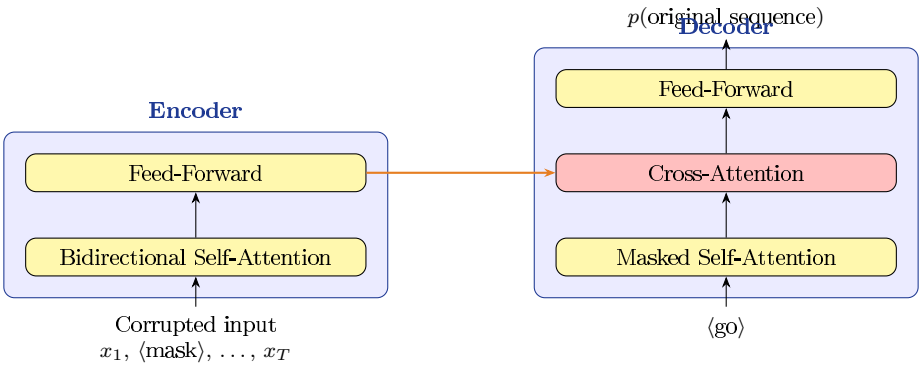
So far, two paradigms have ruled:

- **BERT** (encoder-only): bidirectional, great at *comprehension* (QA, classification, NER).
- **GPT** (decoder-only): unidirectional, great at *generation*.

Intuition

What if we wanted a single model that is *both* good at understanding the input *and* producing fluent output? Translation, summarization, dialogue — all need both. The answer is to bring back the original **encoder-decoder** architecture, this time pre-trained with a *denoising* objective. This model is **BART** (Bidirectional Auto-Regressive Transformer, Lewis et al., 2019).

6.1.1 BART architecture in one diagram



6.1.2 The BART recipe

- **Architecture.** A vanilla encoder-decoder Transformer with GELU activations.
- **Input.** The original text is *corrupted* in multiple ways:
  - Random token masking (*a la* BERT).
  - Token deletion.
  - Span masking (cover a contiguous span with one `[MASK]`).
  - Sentence permutation (shuffle sentence order).

  - Document rotation (start the document at a random position).
- **Target.** The decoder predicts the *original* (uncorrupted) sequence token-by-token, computing the standard CLM loss over the entire target sequence.

Remember This

BART = denoising autoencoder over text. BERT predicts *only* the masked tokens; BART predicts the *whole sequence*. The latter is more general and yields a model strong at both understanding (encoder side) and generation (decoder side).

6.2 GPT-2 — Prompting and the Scaling Story

GPT-1 (Week 3) demonstrated pre-train + fine-tune. But:

- Fine-tuning still costs thousands of labelled examples *per task*.
- For each task, we keep a *separate* fine-tuned copy of the model.
- This does not match how humans learn — we generalise from a few examples or even from instructions alone.

Intuition

What if a sufficiently large model, exposed to enough text, could perform downstream tasks *just from a textual instruction in its prompt* — with no labelled data, no fine-tuning, no gradient updates? This is the **zero-shot transfer** hypothesis that motivated GPT-2 (Radford et al., 2019, *Language Models are Unsupervised Multitask Learners*).

6.2.1 The single-equation insight

Formula Insight

Standard supervised learning models $p(\text{output} \mid \text{input})$, separately per task.
Multi-task learning via prompting models a *single conditional*:

$$p(\text{output} \mid \text{input}, \text{task}),$$

where "task" is just a few words *prepended* (or appended) to the input sequence. The model is trained as a vanilla LM, so the same forward pass produces task-aware outputs.
Example. Prepend "`TL;DR:`" to a long article. The model continues with a summary — because the GPT-2 training data contained many examples of "`TL;DR:`" followed by a summary on Reddit, news, blogs.

6.2.2 Recipe for GPT-2

Hyperparameter	GPT-2
Sizes trained	117M, 345M, 762M, 1.5B parameters
Layers (largest)	48
d_{model} (largest)	1600
Context length	1024
Batch size	512
Vocabulary	Byte-level BPE, 50,257 tokens
Dataset	WebText: 40 GB, 8M Reddit-linked pages
Objective	Causal LM

6.2.3 Zero-shot performance

GPT-2 1.5B, with *no fine-tuning*, outperformed many dedicated language models on their own benchmarks (WikiText perplexity, LAMBADA accuracy, 1BW perplexity, text8 BPC) — simply by virtue of having seen enough text. On more complex tasks (QA, summarization, translation), it was *not* state-of-the-art, but the trend was clear: scaling helps.

Remember This

The big lesson of GPT-2. A 10×-bigger language model, trained on 40×-more web text, learns multitask abilities *implicitly*. "Large language models are unsupervised multitask learners." This insight *directly* motivated GPT-3 (175B) and everything that came after.

Quiz Trap

Trap. "GPT-2 introduced fine-tuning for prompting."
Reality. GPT-2 demonstrated *zero-shot* prompting — no fine-tuning, no labelled examples. The prompt itself encodes the task. (Few-shot prompting / in-context learning came with GPT-3.)

6.3 The T5 Question — Which Choices Actually Matter?

Intuition

GPT-1 (∼117M, CLM, BookCorpus, decoder-only) and BERT-base (∼110M, MLM, BookCorpus+Wiki, encoder-only) perform very differently on downstream tasks. *Why*?

- Architecture (decoder vs. encoder)?
- Objective (CLM vs. MLM)?
- Dataset (size, quality)?
- Model size?
- Training time?
- Fine-tuning protocol?

T5 (Raffel et al., 2020, *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*) was the heroic effort to systematically vary each axis and measure the effect.

6.4 T5: Everything is Text-in / Text-out

Definition

The **T5 framework** casts *every* NLP task as a string-to-string mapping, with a *task-specific prefix* that tells the model what to do.

Input string	Output string
`translate English to Tamil: I enjoyed the movie`	`Naan padathai rasithen`
`summarize: state authorities dispatched emergency crews tuesday...`	`six people hospitalized after a storm in attala county.`
`stsb sentence1: The rhino grazed on the grass. sentence2: A rhino is grazing in a field.`	`3.8`

The genius. Even tasks with numerical outputs (e.g., the STS-B regression between 0–5) become string outputs. Even classification (sentiment) becomes generation of the label word. The same encoder-decoder Transformer handles them all.

6.4.1 T5 dataset — C4

Colossal Clean Crawled Corpus (C4) was built specifically for T5:

- Source: Common Crawl (raw web).
- Cleaning: language detection, deduplication, removal of bad words, removal of pages with placeholder text, etc.
- Size: **750 GB**, $\approx$ 300 M documents, $\approx$ 156 B tokens.
- Tokenizer: SentencePiece, vocab 32,000, mostly English.
- This is $\sim 52\times$ bigger than the BERT pre-training corpus (BookCorpus + Wikipedia $\approx$ 3.3B tokens).

6.5 The T5 Baseline

Baseline architecture. Encoder-decoder, 12 encoder layers + 12 decoder layers, $d_{\text{model}} = 768$, 220M parameters total (roughly twice BERT-base because of the decoder). This is the model against which everything is compared.

6.5.1 Span-corruption denoising objective

Input corruption. Random spans of tokens are replaced by sentinel tokens $\langle X\rangle, \langle Y\rangle, \langle Z\rangle, \ldots$. The model is trained to predict, in order, each sentinel followed by the original span.

Example.

- Original: *"What sets this mission apart is the pivotal role of artificial intelligence (AI) in guiding the spacecraft during its critical descent to the moon's surface."*
- Input: *"What sets this $\langle X\rangle$ is the pivotal role of artificial $\langle Y\rangle$ (AI) in guiding the spacecraft during its $\langle Z\rangle$ to the moon's surface."*
- Target: *"$\langle X\rangle$ mission apart $\langle Y\rangle$ intelligence $\langle Z\rangle$ critical descent $\langle W\rangle$"*.

Formula Insight

Why span-corruption beats BERT-style single-token masking.

- Replacing a span with one sentinel *shortens* the encoder input, *reducing compute* per step.
- The decoder predicts only the corrupted spans, not all tokens — shorter target, faster training.
- Forces the model to predict *groups* of tokens together, learning multi-token coherence.

6.5.2 Baseline training hyperparameters

- Batch size $B = 128$.
- Context length $T = 512$. So $B \cdot T = 65{,}536 = 2^{16}$ tokens / batch.
- Pre-training steps: $2^{19} = 524{,}288$.
- Total pre-training tokens: $2^{19} \times 2^{16} = 2^{35} \approx 34$B tokens ($\sim 0.2\%$ of C4 — not even one epoch!).
- Optimizer: AdaFactor (memory-cheap version of Adam).
- LR schedule: inverse-square-root with 10,000 warmup steps.

6.5.3 Fine-tuning protocol

After pre-training, fine-tune for 2^{18} steps on each downstream task, evaluating checkpoints periodically and picking the best one. Datasets used: **GLUE** & **SuperGLUE** (NLU), **SQuAD** (QA), **CNN/DM** (summarization), **WMT16 En-Ro** (translation).

6.6 Experiment 1 — Architecture

6.6.1 Three families to compare

Architecture	Attention pattern	Pre-training objective options
Encoder-decoder	Encoder = full; Decoder = causal + cross	Span denoising (T5 baseline)
Decoder-only (LM)	Causal	CLM
Decoder-only (Prefix-LM)	Causal-with-prefix (input attended bidirectionally; output causally)	Prefix-LM or span denoising

6.6.2 Mask patterns visualised

- **Fully-visible**: every cell attends to every cell. Used in BERT encoder.
- **Causal**: token i sees only positions $\leq i$. Used in GPT.
- **Causal-with-prefix**: prefix portion is fully visible (bidirectional), the rest is causal. Used in Prefix-LM.

6.6.3 Fair comparison: equal P *or* equal M

Intuition

We cannot compare "BERT-base" and "GPT-base" fairly because they differ in *many* ways. The T5 study fixes *either*:

- Parameter count P (same number of trainable params); *or*
- Compute M (same number of FLOPs to process the same input).

An encoder-decoder with $L + L$ layers has $\approx 2P$ parameters but the *same* FLOPs M as a L-layer decoder-only model on the same input. So fixing M favours the encoder-decoder (more parameters for the same compute).

6.6.4 Results (qualitative summary)

- Sharing encoder and decoder weights ($\theta_{\text{Enc}} = \theta_{\text{Dec}}$) *slightly* hurts performance.
- Halving the number of layers ($6 + 6$ vs $12 + 12$) hurts performance.
- For fixed P and M, **encoder-decoder wins**, especially with span denoising.
- Decoder-only with Prefix-LM gets close to encoder-decoder.
- **Denoising objective always beats CLM** on downstream tasks at fixed parameter count.

Remember This

T5 conclusion on architecture. Encoder-decoder + span denoising is the best combination at equal compute. This is why T5 itself adopts this architecture.
However, in the LLM era, decoder-only models (LLaMA, GPT-3+) dominate because they scale more cheaply (no separate encoder) and exhibit emergent in-context learning abilities. The lesson: design choices have "right answers" relative to a fixed compute budget, not in absolute terms.

6.7 Experiment 2 — Unsupervised Objectives

T5 tested four high-level objective families using the input *"Thank you for inviting me to your party last week."*:

Objective	Input	Target
Prefix LM	"Thank you for inviting"	"me to your party last week."
BERT-style	"Thank you for $\langle M \rangle$ me to $\langle M \rangle$ party *apple* week."	(original sentence)
Deshuffling	"party me for your to . last fun you inviting week Thank"	(original sentence)
MASS-style	"Thank you $\langle M \rangle\langle M \rangle$ me to your party $\langle M \rangle$ week."	(original sentence)
Random spans (T5 default)	"Thank you $\langle X \rangle$ to $\langle Y \rangle$ week ."	"$\langle X \rangle$ for inviting me $\langle Y \rangle$ your party last $\langle Z \rangle$"

Key findings.

- BERT-style is overall better than Prefix-LM and deshuffling.
- Deshuffling is the *worst* — shuffling destroys too much linguistic structure.
- Both Prefix-LM and BERT-style *help translation* more than the others.

6.7.1 Corruption strategies

Within BERT-style, four sub-strategies were tested:

- **Mask**: replace with $\langle M \rangle$.
- **Replace tokens (T5 default)**: replace span with a single sentinel.
- **Drop tokens**: remove them entirely, no sentinel.

All three perform comparably. *Replacing* (span-corruption) is the T5 default because it shortens both input and target ⇒ faster training.

6.7.2 Corruption rate and span length

- **Rate**: 10%, 15%, 25%, 50% all perform comparably. T5 picks 15%.
- **Span length**: 2, 3, 5, 10 tokens. Span 3 wins narrowly on non-translation tasks; span 10 underperforms on two tasks.

Remember This

T5's chosen objective. Span corruption with 15% rate, sentinel replacement, average span length 3. This is the de facto recipe for encoder-decoder denoising.

6.8 Experiment 3 — Dataset Size and Quality

Dataset	Size	Notes
C4 (default)	750 GB	Clean, filtered, deduplicated
Unfiltered-C4	∼ 6 TB	Only language detection; quality drops
RealNews-like	35 GB	News-only filter
WebText-like	17 GB	Reddit-linked pages
Wikipedia	16 GB	Encyclopedia text only
Wikipedia + Book-Corpus	20 GB	The classic BERT corpus

Findings.

- Unfiltered C4 performs *worse* than C4 despite being 8× larger. **Quality ≫ raw size.**
- Domain-specific datasets (RealNews-like, Wikipedia) help on related tasks *even when smaller*.
- Repeating training samples (small dataset, many epochs) leads to memorisation and degrades downstream performance.

Quiz Trap

Trap. "Bigger pre-training corpus is always better."
Reality. Quality matters more than raw size, especially when filtering is weak. *Diverse, clean, in-domain* is the winning combination.

6.8.1 Memorisation through repetition

If we shrink the training set by 2^k but train for $2^k\times$ more epochs (so the model sees the same total tokens), at high k the model starts to memorise. The training loss plummets while the downstream performance worsens — a classic overfitting signature. **Diversity is essential.**

6.9 Experiment 4 — Fine-Tuning Strategies

Full fine-tuning updates all 220M parameters per downstream task. Expensive! Two alternatives:

6.9.1 Adapter layers

Definition

An **adapter** is a small bottleneck network inserted after each FFN block in each layer. It has the structure

$$x \;\to\; \text{Linear}(d_{\text{model}} \to d) \;\to\; \text{ReLU} \;\to\; \text{Linear}(d \to d_{\text{model}}) \;\to\; x + (\text{residual}).$$

During fine-tuning, *only* adapter parameters and layer-norm parameters are updated; the rest of the model is frozen.

Parameter cost. The adapter contributes only 0.5% to 8% of the layer's parameters (depending on d).

Result. Lower d is OK for low-resource tasks (SQuAD with limited labels); high-resource tasks (translation) need higher d for reasonable performance.

6.9.2 Gradual unfreezing

Freeze the entire model; unfreeze layers one at a time, starting from the *top*. For 12 encoder + 12 decoder layers and 2^{18} fine-tuning steps, unfreeze layer L_k of both encoder and decoder simultaneously every $2^{18}/12 \approx 21{,}845$ steps.

Result. Slightly worse performance than full fine-tuning, but speeds up training. Better schedules likely possible.

Research / Industry Insight

Modern descendants. Both ideas have evolved:

- Adapters → **LoRA** (Low-Rank Adaptation, Hu et al., 2021): inject learnable low-rank matrices instead of full bottleneck layers. The dominant PEFT method today.
- Gradual unfreezing → used in ULMFiT (Howard & Ruder, 2018) and in "layer-wise learning-rate decay" schedules common in modern fine-tuning.

6.10 Experiment 5 — Multi-Task Learning

Why fine-tune separately per task? Why not train a *single* model on *all* datasets jointly?

6.10.1 Examples-proportional mixing

Intuition

The naive approach concatenates all datasets and trains on the union. But task dataset sizes vary by orders of magnitude (C4 has 156B tokens; SQuAD has ∼ 100K). A naive concatenation would have the model see C4 for 99.999% of the time and never learn the smaller tasks.

Definition

Examples-proportional mixing. Given N tasks with dataset sizes $e_1, \ldots, e_N$ and an artificial cap K, sample task m during training with probability

$$r_m = \frac{\min(e_m, K)}{\sum_n \min(e_n, K)}.$$

The cap K prevents the largest dataset from dominating.

6.10.2 Temperature mixing

To further control imbalance, apply a temperature T to the sampling probabilities:

$$r_m^{(T)} \propto r_m^{1/T}.$$

Higher $T \rightarrow$ more uniform task sampling. Lower $T \rightarrow$ more skewed (larger datasets favoured).

6.10.3 Results

- Pure multi-task training (no separate pre-training) *degrades* performance relative to pre-train-then-fine-tune.
- **Multi-task pre-training + per-task fine-tuning** reaches the baseline.
- "Leave-one-out" multi-task pre-training (hold out one task, then fine-tune on it) is only *slightly* worse than full multi-task — suggesting good task generalisation.

Remember This

T5's verdict. Plain unsupervised pre-training (denoising on C4) + per-task fine-tuning still wins, because of the massive scale of unlabelled data. Multi-task learning is useful in special cases (e.g., low-resource translation).

6.11 Experiment 6 — How to Spend Extra Compute?

Given a budget that is 4× the baseline, where should it go?

- **Train longer.** $t \rightarrow 4t$ steps. The model sees more tokens.
- **Bigger batch size.** 128 → 512. Same training time, more tokens seen.
- **Bigger model.** 2× parameters; $2t$ steps so total compute is the same.
- **Ensembling.** Pre-train and fine-tune 4 independent models; aggregate.

Findings (qualitative).

- Longer training → consistent improvement.
- Bigger batch → consistent improvement.
- Bigger model with fewer training tokens → better in *many* tasks, especially translation. But increases inference cost.
- Ensembling → good for translation, costs 4× inference time.

Intuition

The interview answer. There is no universally "best" way to scale — it's a trade-off. If you have a deployment with cheap inference, scale parameters. If inference is expensive, train longer with a smaller model. If you can afford 4× inference, ensemble for the last fractions of a BLEU point.

6.12 The Final T5 Models

Armed with all this knowledge, the T5 authors trained five models:

Model	params	layers	d_{model}	d_{ff}	d_{kv}	heads
T5-Small	60M	6	512	2048	64	8
T5-Base	220M	12	768	3072	64	12
T5-Large	770M	24	1024	4096	64	16
T5-3B	3B	24	1024	16384	128	32
T5-11B	11B	24	1024	65536	128	128

All pre-trained for $2^{20} = 1$M steps, batch size $2048 = 2^{11}$. Total: ≈ 1 trillion tokens ($32\times$ the baseline). **T5-11B achieved SOTA on many benchmarks at the time.**

6.13 Common Mistakes & Quiz Traps — Week 7

Common Mistake

Mistake 1. "BART = BERT + GPT layers stacked together."
Fix. BART is a vanilla encoder-decoder Transformer; the names BART, BERT and GPT refer to different pre-training paradigms, not to neural blocks. BART's contribution is the *multi-modal corruption* pre-training scheme, not a new architecture.

Common Mistake

Mistake 2. "T5 is decoder-only."
Fix. T5 is encoder-decoder. The largest T5 model is 11B parameters *evenly split* between encoder and decoder.

Common Mistake

Mistake 3. "GPT-2's zero-shot performance beats fine-tuned BERT on most tasks."
Fix. GPT-2 zero-shot is impressive but *below* task-specific fine-tuned models on most benchmarks. Its value is the proof-of-concept that a single LM can do many tasks without fine-tuning — a paradigm shift, not an immediate SOTA win.

Quiz Trap

Trap. "Span corruption requires a separate Transformer for each masked span."
Reality. A single forward pass produces all span predictions, marked by sentinels $\langle X\rangle, \langle Y\rangle, \langle Z\rangle$. The decoder learns to emit each sentinel followed by its span content in order.

Quiz Trap

Trap. "Examples-proportional mixing weights tasks by raw size."
Reality. It *caps* the sizes at K, so a 156B-token dataset is treated the same as a K-token cap. Without the cap, the largest dataset would dominate training.

6.14 Memory Tricks — Week 7

Memory Trick

"BART = BERT + GPT."
Bidirectional encoder + **A**uto**R**egressive decoder, joined by cross-attention. Pre-trained with multi-modal corruption.

Memory Trick

"GPT-2's superpower: prompting."
Pre-train a big LM; prepend a task description; the LM generates the answer. Zero labelled examples. The "Language Models are Unsupervised Multitask Learners" paper.

Memory Trick

"T5: text in, text out."
Every task – classification, regression, QA, translation, summarization – is reduced to string-to-string. The task name is part of the input.

Memory Trick

"Span = 3, rate = 15%, sentinel replacement, EncDec."
The T5 winning recipe distilled to one line.

6.15 Comparison Table: BART vs GPT-2 vs T5

	BART	**GPT-2**	**T5**
Year	2019	2019	2020
Architecture	Encoder-decoder	Decoder-only	Encoder-decoder
Objective	Multi-modal denoising	CLM	Span corruption denoising
Dataset	160 GB (CC, books, news, stories)	WebText 40 GB	C4, 750 GB
Vocab	BPE	Byte-level BPE 50,257	SentencePiece 32,000
Largest size	400M	1.5B	11B
Strength	Generation + comprehension	Zero-shot generation	Unified multi-task

6.16 Quick Revision — Week 7 in One Page

Quick Revision

BART. Encoder-decoder pre-trained as a *denoising autoencoder*: encoder reads a corrupted sentence; decoder reconstructs the original. Multiple corruption modes (mask, delete, span, shuffle, rotate).
GPT-2. Scaling laws meet zero-shot prompting. 1.5B parameters, WebText 40GB, byte-level BPE. Showed that *language modelling* alone produces an "unsupervised multitask learner".
T5 framework. `text in` $\rightarrow$ `text out` for every task. Task prefixes in the input.
T5 systematic study findings.
- **Architecture**: Encoder-decoder + denoising > Prefix-LM > decoder-only LM (at fixed P, M).
- **Objective**: span corruption > token masking; deshuffling worst.
- **Corruption rate**: 15% wins; 10%–50% all comparable.
- **Span length**: 3 tokens optimal on average.
- **Dataset**: *quality* > *raw size*. Repetition causes memorisation.
- **Compute**: longer training, bigger batch, bigger model all help, with trade-offs.

T5-11B. 11B parameters, 1T tokens, denoising span-3 / 15% / sentinel replacement, encoder-decoder. SOTA on GLUE, SuperGLUE, SQuAD, CNN/DM at release.
Adapter layers. Bottleneck $d_{\text{model}} \rightarrow d \rightarrow d_{\text{model}}$ inserted after FFN. Updates 0.5–8% of parameters during fine-tuning. Modern descendant: LoRA.
Multi-task learning. Use examples-proportional mixing with cap K and optional temperature T. Multi-task pre-train + fine-tune $\approx$ baseline; pure multi-task training < baseline.

6.17 Practice Questions — Week 7

Multiple Choice

1. **(Q1)** BART's pre-training objective is best described as:

   (A) Masked Language Modelling on the encoder only.

   (B) Causal Language Modelling on the decoder only.

   (C) Reconstructing the original text after multiple types of corruption.

   (D) Sentence-level binary classification.

   Solution. (C). BART is a denoising autoencoder over text.

2. **(Q2)** GPT-2 demonstrated that:

   (A) Bigger BERT models always outperform GPT.

   (B) A sufficiently large LM can perform multiple tasks zero-shot via prompts.

   (C) Cross-attention is essential for translation.

   (D) Causal masking should be removed.

   Solution. (B).

3. **(Q3)** In T5's span corruption, a *sentinel* token represents:

   (A) the end of a sentence,

(B) a placeholder for a contiguous span of masked tokens,

(C) the special task prefix,

(D) the padding token.

Solution. (B).

4. **(Q4)** Adapter layers reduce fine-tuning cost by:

   (A) replacing the FFN entirely with the adapter,

   (B) updating only adapter and layer-norm parameters while freezing the rest,

   (C) halving the number of layers,

   (D) using dropout.

   Solution. (B).

5. **(Q5)** The C4 dataset is approximately:

   (A) 750 MB (B) 7.5 GB (C) 750 GB (D) 7.5 TB.

   Solution. (C).

6. **(Q6)** Examples-proportional mixing with cap K ensures that:

   (A) all tasks have exactly equal sampling probability,

   (B) very large datasets do not dominate training,

   (C) smaller datasets get more weight than larger ones,

   (D) the model never sees any task more than once.

   Solution. (B). The cap clips the effective dataset size and equalises exposure across tasks.

Conceptual / Short Answer

7. **(Q7)** Why is span-corruption denoising computationally cheaper than BERT-style token masking?

   Answer. BERT masks individual tokens but the entire input length is unchanged. Span corruption replaces a multi-token span with a *single* sentinel, *shortening* both the encoder input and the decoder target. Shorter sequences = fewer FLOPs per step.

8. **(Q8)** Explain in one sentence why "deshuffling" performs worst among the unsupervised objectives tested in T5.

   Answer. Deshuffling destroys local linguistic structure (word order, syntax, phrase boundaries) so completely that the model spends most of its capacity learning to undo the shuffle rather than learning useful representations.

9. **(Q9)** An encoder-decoder model with $L = 12$ layers in each side has roughly twice the parameters of a decoder-only model with $L = 12$ layers, yet has the *same* FLOPs for the same input. Why?

   Answer. Each token traverses the same number of self-attention + FFN computations in both architectures, but the encoder-decoder has *disjoint weight matrices* for each of its $L + L$ layers, doubling parameter count. FLOPs depend on layers per forward pass, not on the total number of parameters.

10. **(Q10)** Why does pure multi-task learning (no separate pre-training) often underperform pre-train-then-fine-tune?

    Answer. Pure multi-task learning sees only labelled task data, which is small. Pre-train-then-fine-tune sees enormous quantities of unlabelled data first (modelling language in general), then specialises with much smaller labelled data. The unlabelled pre-training step gives the model far more knowledge to draw from.

11. **(Q11)** In Prefix-LM, why are the prefix tokens fully visible to each other while the suffix tokens are causally masked?

    Answer. The prefix encodes the *input* (which is fully given), so bidirectional attention helps the model build a rich representation. The suffix is the *output* being generated, so causal masking enforces autoregressive generation.

Numerical

12. **(Q12)** T5's baseline pre-training: batch size 128, context length 512, 2^{19} steps. Compute the total tokens seen.

    Solution. $128 \times 512 \times 2^{19} = 2^7 \times 2^9 \times 2^{19} = 2^{35} \approx 34$ B tokens.

13. **(Q13)** T5's final 11B model: batch 2048, context 512, 2^{20} steps. How many tokens does it see in total, and how many times is that the baseline?

    Solution. $2048 \times 512 \times 2^{20} = 2^{11} \times 2^9 \times 2^{20} = 2^{40} \approx 1.1$T tokens. That is $2^{40}/2^{35} = 2^5 = 32\times$ the baseline.

14. **(Q14)** An adapter has $d_{\text{model}} = 768$ and bottleneck $d = 64$. What is the parameter count of one adapter (ignoring biases)?

    Solution. Two linear layers: $768 \times 64 + 64 \times 768 = 2 \cdot 768 \cdot 64 = 98{,}304 \approx 0.1$M. With 24 adapters in a 12-layer enc-dec (12 encoder + 12 decoder), total ≈ 2.4M parameters — about 1% of a 220M model.

15. **(Q15)** Examples-proportional mixing with three tasks of sizes $e_1 = 100$K, $e_2 = 10$M, $e_3 = 10$B and cap $K = 1$M. Compute the sampling probability for each task.

    Solution. Clipped sizes: $\min(e_1, K) = 100$K, $\min(e_2, K) = 1$M, $\min(e_3, K) = 1$M. Sum $= 2.1$M.

    - $r_1 = 100\text{K}/2.1\text{M} \approx 0.048$ (4.8%).
    - $r_2 = 1\text{M}/2.1\text{M} \approx 0.476$ (47.6%).
    - $r_3 = 1\text{M}/2.1\text{M} \approx 0.476$ (47.6%).

    Without the cap, r_3 would have been $\approx 99.99\%$. The cap brings r_2 and r_3 to equal footing while still giving the smallest task (r_1) at least 4.8% exposure.

7 Week 8 — The LLM Landscape, Scaling Laws, and Data Pipelines

Chapter goals. By the end of this chapter you will be able to:

1. Map the post-T5 LLM landscape (GPT-3 / Jurassic / Gopher / GLaM / OPT / LLaMA / InstructGPT / ChatGPT).
2. Read the "unifying diagram" showing all the design choices a modern LLM configures (position encoding, attention, normalisation, activation, FFN type, objective, optimiser).
3. Apply the **Kaplan scaling law** $L(N, D) = [(N_c/N)^{\alpha_N/\alpha_D} + D_c/D]^{\alpha_D}$ to estimate test loss.
4. Articulate the three axes of pre-training data: scale, quality, diversity.
5. Trace the end-to-end pre-processing pipeline (CCNet → C4 → RefinedWeb) including language detection, quality filtering, PII removal, deduplication.
6. Compare the major datasets (C4, The Pile, RefinedWeb, RedPajama, DOLMA, mC4, Stack, ROOTS, Sangraha).
7. Explain why deduplication matters and what the empirical evidence shows.

Part A — The LLM Landscape After T5

7.1 The Post-T5 Race

Right after T5 (2020), the field exploded with model releases. A quick chronology:

Model	Params	Architecture	Key idea / claim to fame
GPT-3 (2020)	175 B	Decoder-only	Few-shot "in-context" learning emerges at scale.
Jurassic-1	178 B	Decoder-only	AI21 Labs' commercial GPT-3 alternative.
GPT-J	6 B	Decoder-only	First strong *open-source* GPT clone.
Gopher (DeepMind)	280 B	Decoder-only	MassiveText custom corpus; very strong on KB QA.
GLaM	1.2 T	Mixture-of-Experts	Sparse routing; only ∼ 97B params active at a time.
PaLM	540 B	Decoder-only	RoPE position; trained on 780B tokens.
OPT (Meta)	175 B	Decoder-only	Open-source replication of GPT-3.
InstructGPT	1.3–175 B	Decoder-only	RLHF-aligned dialogue model.
ChatGPT (2022)	—	Decoder-only	InstructGPT-style alignment + chat UI.
LLaMA / 2 / 3	7 B–70 B	Decoder-only	High-quality open-source training.

Intuition

Two big trends crystallised.

- **Architecture:** Decoder-only became the dominant choice (cheap, scalable, in-context learning emerges).
- **Training paradigm:** "Pre-train → instruction-tune → RLHF align" replaced "pre-train → fine-tune". We will study these alignment steps in Week 11+.

7.2 The Unifying Diagram — Choices Inside Every LLM

Even if you commit to "decoder-only Transformer", many choices remain. The lecturer introduced a unifying diagram covering them all. We summarise the design space here:

Component	Options
Position encoding	Absolute (GPT-1, BERT) / Relative (T5) / **RoPE** (LLaMA, PaLM) / NoPE / ALiBi (BLOOM, MPT).
Attention variant	Full / sparse / multi-head (MHA) / multi-query (MQA) / grouped-query (GQA) / FlashAttention / PagedAttention.
Normalisation	LayerNorm (GPT-1) / **RMSNorm** (LLaMA) / DeepNorm. Placement: PostNorm / **PreNorm** / Sandwich.
Activation in FFN	ReLU / GELU / Swish / **SwiGLU** (LLaMA) / GeGLU (PaLM).
FFN type	Dense FFNN / **Mixture-of-Experts (MoE)** (GLaM, Mixtral).
Objective	CLM (GPT) / MLM (BERT) / Denoising (T5/BART) / Prefix-LM.
Optimiser	Adam / AdamW (most common) / AdaFactor (T5) / LION.

Remember This

The modern "LLaMA-style" recipe: **Decoder-only, RoPE, GQA, RMSNorm, PreNorm, SwiGLU, dense FFN, CLM, AdamW.** Most open-weight LLMs in 2024 (LLaMA-3, Mistral, Qwen, Yi) share this combination.

Research / Industry Insight

Why each choice in the modern stack?

- **RoPE**: rotates query/key vectors by an angle proportional to position. Naturally generalises to longer contexts at inference.
- **GQA**: groups KV heads to reduce KV-cache memory during generation (LLaMA-2 onwards).
- **RMSNorm**: simpler than LayerNorm (no mean centering); $\sim 7\%$ faster.
- **PreNorm**: stable training without learning-rate warmup.
- **SwiGLU**: $(\text{Swish}(xW_1) \odot xW_2)W_3$. Empirically better than ReLU/GELU at the cost of one extra linear layer.

Part B — Scaling Laws and Data

7.3 The Kaplan Scaling Law

Kaplan et al. (2020, OpenAI) empirically fit a *power law* relating test loss to the number of *non-embedding* parameters N and the number of training tokens D:

Formula Insight

Kaplan scaling law.

$$\boxed{L(N, D) = \left[\left(\frac{N_c}{N}\right)^{\alpha_N/\alpha_D} + \frac{D_c}{D}\right]^{\alpha_D},}$$

with empirical constants

$N_c = 8.8{\times}10^{13}$ (88T params), $D_c = 5.4{\times}10^{13}$ (54T tokens), $\alpha_N = 0.076$, $\alpha_D = 0.095$.

7.3.1 What the formula tells us

- As $N \to \infty$ *and* $D \to \infty$, $L \to 0$ (in theory).
- As N alone grows (with D fixed), the first term in brackets vanishes asymptotically as $N^{-\alpha_N/\alpha_D}$.
- As D alone grows (with N fixed), the second term decays as $D^{-\alpha_D} \Rightarrow L \sim D^{-\alpha_D \cdot \alpha_D}$. (*Wait: more carefully, when D is the bottleneck, $L \sim D^{-\alpha_D \cdot 1} \cdot const$. Either way, the decay is power-law in D.*)

7.3.2 Worked example — estimating test loss

Let $N = 100\text{M} = 10^8$ non-embedding params and $D = 10\text{M} = 10^7$ tokens.

$$\begin{aligned}
\frac{N_c}{N} &= \frac{8.8 \times 10^{13}}{10^8} = 8.8 \times 10^5, \\
\left(\frac{N_c}{N}\right)^{\alpha_N/\alpha_D} &= (8.8 \times 10^5)^{0.076/0.095} = (8.8 \times 10^5)^{0.8} \\
&\approx \exp(0.8 \cdot \ln(8.8 \times 10^5)) \approx \exp(0.8 \cdot 13.69) \approx \exp(10.95) \approx 56{,}962. \\
\frac{D_c}{D} &= \frac{5.4 \times 10^{13}}{10^7} = 5.4 \times 10^6, \\
L &\approx (56{,}962 + 5.4 \times 10^6)^{0.095} \approx (5.46 \times 10^6)^{0.095} \approx 4.36.
\end{aligned}$$

Interpretation. With only 10M tokens, the data is the bottleneck (the D_c/D term dominates). Adding more parameters has diminishing returns.

7.3.3 What happens if we add more data?

If we increase D by $10\times$ to 10^8 tokens:

$$\frac{D_c}{D} = 5.4 \times 10^5, \quad L \approx (56{,}962 + 5.4 \times 10^5)^{0.095} \approx (5.97 \times 10^5)^{0.095} \approx 3.54.$$

A clear drop. **More data lowers test loss; the model is no longer data-limited.**

Intuition

The Kaplan law motivated the entire "scale at all costs" arc of 2020–2022: **bigger model + more data = lower loss.** Subsequent work by Hoffmann et al. (2022, the *Chinchilla* paper) corrected the balance: for a fixed compute budget, parameters and tokens should scale *at roughly the same rate*, not parameters dominantly. LLaMA-1 (7B / 1.4T tokens) and beyond follow Chinchilla guidance.

Quiz Trap

Trap. "Scaling laws say bigger is always better."
Reality. They predict *lower loss at higher compute*, but loss is not the only metric. They say nothing about alignment, factuality, instruction following, or reasoning. The biggest model that follows scaling laws may still hallucinate.

7.4 The Three Axes of Data: Scale, Quality, Diversity

Intuition

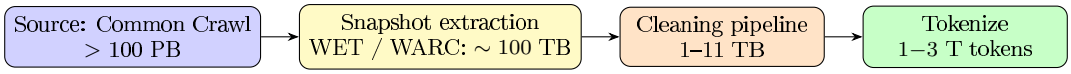
You can ship more data along three independent directions:
- **Scale**: raw token count.
- **Quality**: how clean, well-formed, and informative is each token.
- **Diversity**: how broad the topical / linguistic coverage is.

All three matter; the right mix depends on the goal.

7.4.1 Datasets over the years (the trend)

Dataset	Year	Size	Notes
BookCorpus	2015	5 GB	Used by BERT, GPT-1.
Wikipedia (English)	—	21 GB	Used by BERT, T5 mix.
WebText (closed)	2019	40 GB	GPT-2 training set.
RealNews	2019	110 GB	News pre-training.
C4	2020	750 GB / 156B tokens	T5 baseline.
The Pile	2020	800 GB / 170B tokens	22 diverse sources.
ROOTS	2022	1.6 TB / 341B tokens	Multilingual; BLOOM.
RefinedWeb (Falcon)	2023	2.6 TB / 5T tokens (600B public)	Web only, high quality.
RedPajama-v1 / v2	2023	5 TB / 30T tokens	Open replica of LLaMA mixture.
DOLMA	2024	11 TB / 3T tokens	Open with documentation.
Sangraha	2024	— / 251B tokens	Multilingual incl. Indian languages.

7.4.2 The pipeline at a glance



The cleaning pipeline shrinks the data by 10× to 100× but is what turns garbage into a usable training corpus.

7.5 Common Crawl — The Raw Source

Common Crawl (CC) releases monthly snapshots of the web with 3–5 billion pages. Total accumulated data: > 100 petabytes. Available in three formats:

Format	Size	Content
WARC	86 TiB / snapshot	Raw HTML pages (full markup + headers).
WAT	21 TiB / snapshot	Web ARCHIVE Transformation: metadata + links.
WET	8.7 TiB / snapshot	WARC-Encapsulated Text: extracted plain text.

Most pipelines work on **WET** (cheap and clean enough). RefinedWeb and Pile-CC work on **WARC** for the highest-quality text extraction.

7.6 The Cleaning Pipeline — Five Stages

All modern pipelines share a five-step backbone:

1. **Language detection.** Identify and keep only the target language(s). Tools: `langdetect`, `pycld2`, `FastText`, `IndicLID`.
2. **Quality filtering.** Score pages by quality (using heuristics or a small LM trained on a "gold" reference corpus like Wikipedia). Drop low-quality pages.
3. **Toxicity / safety filter.** Detect bad words, hate speech, stereotypes. Tools: bad-word lists, Perspective AI, custom classifiers.
4. **PII removal.** Detect and redact email addresses, phone numbers, credit-card numbers, etc. Usually regex + heuristics.
5. **Deduplication.** Remove duplicate lines / paragraphs / documents. Critical for performance.

7.6.1 Exact vs Fuzzy Deduplication

Type	Catches	Typical algorithm
Exact	byte-identical content	Suffix array, hash table
Fuzzy (near)	paraphrased / lightly edited copies	**MinHash** + **LSH**, SimHash, Bloom filter

Example.

- *Webpage 1:* "The orbit of Aditya-L1 is a periodic Halo orbit."
- *Webpage 2:* "The orbit of Aditya-L1 is a periodic Halo orbit." *(exact)*
- *Webpage 3:* "Aditya-L1's orbit is a periodic Halo orbit located ~1.5M km from Earth." *(near-duplicate)*

Exact dedup catches webpage 2. Fuzzy dedup (e.g., MinHash with LSH) is needed for webpage 3.

7.7 Three Famous Pipelines

7.7.1 Pipeline 1: fastText (the basic version)

- Language detection → deduplication (using hashing).
- Only covers two of the five steps.
- No quality filter, no PII removal → filtered text still contains toxic and low-quality content.

7.7.2 Pipeline 2: CCNet (Wenzek et al., 2019)

CCNet was designed for *high-quality monolingual* pre-training data, particularly useful for low-resource languages where Wikipedia alone is too small (Hindi has only 1.2M Wikipedia pages; Tamil 0.5M; vs. English's 59M).

Steps.

1. Language detection.
2. Paragraph-level deduplication (using hashing).
3. Quality filtering via an in-domain LM (e.g., trained on Wikipedia): score each paragraph; sort into head / middle / tail buckets; drop the tail.

Result. Documents extracted for 130 languages from a single Feb-2019 CC snapshot. Hindi: $\sim$ 550B tokens. Tamil: $\sim$ 1.5B tokens. Vastly more than the Wikipedia-only baseline.

Intuition

The CCNet "head / middle / tail" trick is clever: split a webpage into three parts (top, middle, bottom of page) and score each separately. Real content tends to be in the middle; menus / ads / footers in the head and tail.

7.7.3 Pipeline 3: C4 (T5)

C4 extends CCNet with additional filtering heuristics specifically tuned for English:

- Remove pages containing "`lorem ipsum`" (placeholder text from web design templates).
- Remove pages containing curly brackets `{ }` (likely source code).
- Remove lines that do not end with terminal punctuation marks (likely menu items, headers).
- Remove pages containing any word from a manual bad-words list.
- Final exact deduplication.

Result. From a 20 TB CC snapshot, English subset is 6 TB; after all C4 filters, the final C4 is **750 GB**.

7.7.4 Worked example: the "Lemon" page

The lecturer's recurring example: a webpage about lemons that has:

- Useful encyclopedia-style content (good!).
- "Please enable JavaScript to use our site." (boilerplate, drop).
- "Dried Lemons, \$3.59/pound." (e-commerce noise, drop).
- "`function Ball(r) { ... }`" (JS source code, drop because curly bracket).
- "Lorem ipsum dolor sit amet..." (placeholder, drop entire page).

After C4 filtering, only the encyclopedia paragraphs remain.

7.8 Pipeline 4: RefinedWeb — "Web Only is Enough"

The RefinedWeb pipeline (Falcon, 2023) made a bold claim: *high-quality web data alone, in sufficient quantity, beats curated mixtures.*

Three-stage attrition.

1. Trafilatura + Language ID: -50% of CC.
2. Quality filtering: -24% of remainder.
3. Deduplication (both fuzzy and exact): -12% of remainder.

Net effect: $\sim 90\%$ of the original CC snapshot is discarded.

Remember This

The lesson of RefinedWeb. Aggressive filtering is more important than the source. You can match or beat "curated" corpora (Wiki + books + papers + code) using web data alone, provided you filter ruthlessly.

7.8.1 Pipeline 5: SETU / Sangraha (multilingual, India-focused)

For Indian languages, the AI4Bharat group built the **SETU pipeline**:

- **Sources:** curated web (manually verified), YouTube videos (ASR), digitised PDFs (OCR).
- **Language detection:** IndicLID + CLD3 + NLLB.
- **Text extraction:** Trafilatura.
- **Deduplication:** Fuzzy via MinHash.

Result: Sangraha-Verified — a $\sim$ 251B-token multilingual corpus including 22 Indian languages.

Research / Industry Insight

Why ASR and OCR matter for low-resource languages. The web has very limited Tamil / Marathi / Odia text but plenty of audio (videos) and scanned books. ASR and OCR can multiply the available text by an order of magnitude — crucial when Wikipedia alone is too small for pre-training.

7.9 Why Deduplication Actually Matters

Intuition

A single 61-word sentence was found to be repeated **60,000 times** in C4. This is not a quirk — duplication at this scale fundamentally changes the model's behaviour. Three concrete problems:

1. **Memorisation.** The model learns to reproduce the exact passage rather than generalise.
2. **Privacy risk.** Personal info appearing in duplicates is more likely to be regurgitated at inference.
3. **Inflated test scores.** If a duplicate also appears in the test set, benchmarks are contaminated.

7.9.1 Empirical evidence

- *Quantifying Memorization* (Carlini et al., 2022): a sequence that appears 10× in training is reproduced verbatim 1000× more often than a sequence that appears once.
- *Learning from Repeated Data* (Hernandez et al., 2022): repeating 0.1% of data 100 times can degrade an 800M-param model down to the level of a 400M model.
- *The Pile authors themselves* admit: "We found the Pile was particularly full of duplicate documents; future researchers should perform additional deduplication."

7.9.2 Why perfect dedup is hard

- Fuzzy duplicates (paraphrased copies) require approximate similarity computation over trillions of pairs.
- Near-duplicates can be subtle (e.g., "the orbit is…" vs "the orbit was reported to be…").
- Compute cost of pairwise comparison is $O(n^2)$, infeasible at scale. Hence MinHash + Locality-Sensitive Hashing (LSH) as a near-linear approximation.

7.10 Major Datasets at a Glance

Dataset	Lang	# Tokens	Curated?	Sources / Diversity
C4	English	∼ 156 B	No	Webpage only
The Pile	English	∼ 170 B	Yes	22 sources: web, books, code, arxiv, PubMed, ...
Stack	Code	> 1 T	Yes	380 programming languages
RefinedWeb (Falcon)	English	5 T (600B public)	No	Webpage only
RedPajama-v1 / v2	English / Multi	1.2 / 30 T	Yes	Web, books, arxiv, wiki, StackExchange
DOLMA	English	3 T	Yes	Web, books, wiki, Stack, STEM
mC4	Multi	∼ 418 B	No	Webpage, 101 languages
ROOTS	Multi	∼ 341 B	Yes	Natural + programming languages (BLOOM)
Sangraha	Multi	251 B	Yes	Web, video, scanned PDF, synthetic

7.10.1 Two philosophies

- **The Pile / DOLMA philosophy:** maximise *diversity* with many curated sources (web + books + code + papers + dialogue).
- **RefinedWeb / Falcon philosophy:** maximise *web-only quality*; trust that filtered web data covers all important domains implicitly.

7.11 Data Mixing — Sampling Across Sources

For a multi-source corpus, the model's training distribution is a *weighted mix* of the sources. Example mixture for a hypothetical 1T-token training run:

Source	Sample weight
C4	40%
OpenWebText	15%
Books	10%
Wikipedia	10%
RealNews	10%
Code (GitHub / Stack)	10%
Stories	5%

Decisions you make at this stage.

- How much code? (LLaMA-1: ∼ 5%; DeepSeek-Coder: 87%).
- How much non-English? (LLaMA-2: ∼ 8%; mBLOOM: ∼ 50%).
- How much conversation data? (LaMDA gives this large weight, since it is conversation-focused.)
- How much synthetic data? (Phi-1 / 2 / 3 use heavily synthetic; controversial.)

Remember This

The mixture is a hyperparameter. Just like learning rate or batch size, the data-mixing ratios materially affect downstream performance. Many recent papers (DoReMi, Sheared LLaMA) train small "proxy" models to search for the best mixture *before* committing to the expensive large training run.

7.12 Open Questions

The lecturer raised four open questions, all live research topics:

- **Are we hitting a data wall?** High-quality English text is finite; some estimates put exhaustion around 2026.
- **Synthetic data:** can GPT-4 generate training data for GPT-5? Early evidence (Phi-3, Self-Instruct, distillation) suggests yes, with caveats about model collapse.
- **Small but high-quality datasets:** "less is more" — can 30B–100B tokens of *very high quality* match 1T tokens of average web data?
- **Code + text mixing:** does code data improve general reasoning? (Many teams report yes.)

7.13 Common Mistakes & Quiz Traps — Week 8

Common Mistake

Mistake 1. "The Kaplan scaling law tells us how big the model should be."
Fix. Kaplan gives a *loss surface* as a function of (N, D). To find the optimal (N, D) for a given *compute budget* $C = 6ND$ FLOPs, you also need Chinchilla's joint analysis. Kaplan alone predicts test loss; the optimum point requires both papers.

Common Mistake

Mistake 2. "WARC, WAT, WET are different processed versions of CC."
Fix. They are different *file formats* of the *same* CC snapshot: WARC = raw HTML, WAT = metadata extracted, WET = plain text extracted. WET is the cheapest input to a text pipeline.

Common Mistake

Mistake 3. "Deduplication is optional — the model averages out duplicates."
Fix. Duplicates are *amplified* during training because gradient updates on the same content compound. Even 0.1% duplicate rate at 100× frequency can halve effective model capacity.

Quiz Trap

Trap. "Curated mixtures (Pile/DOLMA) always beat web-only datasets."
Reality. RefinedWeb / Falcon showed that aggressive filtering on web data alone can match or beat curated mixtures. The frontier of large-corpus design is "ruthless filtering of cheap data," not necessarily "hand-curated sources".

Quiz Trap

Trap. "Quality filtering with a Wikipedia-based classifier is bias-free."
Reality. Quality classifiers trained on "gold" references (Wikipedia, Reddit) implicitly bias the corpus toward those demographics' English. This is why some teams (Gopher) deliberately *do not* train such classifiers.

7.14 Memory Tricks — Week 8

Memory Trick

"LLaMA-style: RoPE + RMSNorm + SwiGLU + PreNorm + GQA."
The dominant modern decoder-only recipe in five components.

Memory Trick

"Three axes: Scale, Quality, Diversity."
All three matter; cheap data is OK if you filter; small & clean can beat huge & messy.

Memory Trick

"CC → WET → language → quality → PII → dedup → pre-training."
The canonical six-step pipeline. Memorise the order.

Memory Trick

"MinHash + LSH for fuzzy dedup."
The standard tool for near-duplicate detection at scale. Exact dedup uses suffix arrays or hash tables.

7.15 Quick Revision — Week 8 in One Page

Quick Revision

LLM landscape post-T5. Decoder-only models won, with sizes from 6B (GPT-J) to 1.2T (GLaM MoE). Modern recipe: *RoPE + GQA + RMSNorm + PreNorm + SwiGLU + AdamW + CLM.*
Kaplan scaling law. $L(N, D) = [(N_c/N)^{\alpha_N/\alpha_D} + D_c/D]^{\alpha_D}$ with $N_c = 8.8 \times 10^{13}$, $D_c = 5.4 \times 10^{13}$, $\alpha_N = 0.076$, $\alpha_D = 0.095$. More N and $D \Rightarrow$ lower L.
Three axes of data. Scale (token count), Quality (filtering), Diversity (sources).
Common Crawl. $>$ 100 PB total. Released monthly. Three formats: **WARC** (raw HTML), **WAT** (metadata), **WET** (plain text).
Pipeline (5 stages). Language detection $\rightarrow$ quality filter $\rightarrow$ toxicity filter $\rightarrow$ PII removal $\rightarrow$ deduplication.
CCNet $\rightarrow$ C4 $\rightarrow$ RefinedWeb. Each adds more aggressive filtering. RefinedWeb's verdict: high-quality web data alone, filtered to $\sim 10\%$ of source, is sufficient.
Deduplication. Exact (suffix array, hash) vs fuzzy (MinHash + LSH, SimHash). A 61-word sentence repeated 60K times in C4. Repetition hurts: 0.1% data $\times$ 100$\times$ can halve model effective capacity.
Major datasets. C4 (156B), The Pile (170B), RefinedWeb (5T total, 600B public), RedPajama-v1/v2 (1.2/30T), DOLMA (3T), Stack (1T code), mC4 (418B multi), ROOTS (341B multi), Sangraha (251B Indic).
Mixing. Sample tokens from multiple sources with predefined weights (e.g., 40% C4, 15% web, 10% Wikipedia, 10% news, etc.).

7.16 Practice Questions — Week 8

Multiple Choice

1. **(Q1)** In the Kaplan scaling law, when $N \rightarrow \infty$ and D is fixed, test loss approaches:

   (A) zero,

   (B) a non-zero floor that depends only on D,

   (C) one,

   (D) infinity.

   Solution. (B). When $N \rightarrow \infty$, the $(N_c/N)^{\alpha_N/\alpha_D}$ term vanishes and $L \rightarrow (D_c/D)^{\alpha_D}$, a constant depending on D.

2. **(Q2)** CCNet's quality filtering uses:

   (A) a hand-coded list of bad words,

   (B) a small LM trained on a reference domain (e.g., Wikipedia),

   (C) a Bayesian network,

   (D) exact-match deduplication.

   Solution. (B). The LM scores how "Wikipedia-like" each paragraph is.

3. **(Q3)** Which CC format gives raw HTML pages?

   (A) WET (B) WAT (C) WARC (D) WPC.

   Solution. (C). WARC = Web ARChive. WET is plain text; WAT is metadata.

4. **(Q4)** The C4 filter "drop lines that don't end with a terminal punctuation mark" primarily removes:

   (A) source code,

   (B) news articles,

   (C) menu items and headers,

   (D) Wikipedia content.

   Solution. (C). Real prose ends in periods, exclamations, or question marks; menu items and headers usually do not.

5. **(Q5)** Fuzzy deduplication is typically done with:

   (A) exact hash tables,

   (B) MinHash with Locality-Sensitive Hashing,

   (C) SQL primary keys,

   (D) byte-level comparison.

   Solution. (B). MinHash + LSH provides near-linear approximate near-duplicate detection.

6. **(Q6)** RefinedWeb's empirical claim is:

   (A) Curated mixtures always beat web-only data.

   (B) Heavily filtered web-only data can match or beat curated mixtures.

   (C) Web data is too noisy for LLM pre-training.

   (D) Deduplication is unnecessary.

   Solution. (B). The Falcon team's headline contribution.

Conceptual / Short Answer

7. **(Q7)** Why does the Kaplan law use *non-embedding* parameters N rather than total parameters?

   Answer. Embedding parameters scale with vocabulary size $|\mathcal{V}|$, not with model capacity. A 50K-vocab embedding adds $\sim$ 30M parameters that do not help the model "compute" more — they just look up vectors. Kaplan's law measures the loss as a function of *computational* capacity, so embeddings are excluded.

8. **(Q8)** Explain in 2–3 sentences why CCNet's "head / middle / tail" quality scoring works.

   Answer. CCNet splits each webpage into three vertical portions and scores each separately. Real article content tends to sit in the middle; the head (banner, menu, header) and tail (footer, related links, comments) are typically boilerplate. Scoring per portion lets the pipeline keep the good middle and discard the noisy edges.

9. **(Q9)** What is the difference between MinHash and SimHash for fuzzy deduplication?

   Answer. *MinHash* approximates Jaccard similarity between sets (e.g., of shingles); two documents have a small MinHash signature distance iff their sets overlap significantly. *SimHash* produces a fingerprint such that the Hamming distance between two fingerprints approximates the cosine distance between the documents. Both are sub-linear via LSH bucketing.

10. **(Q10)** Why is "perfect" deduplication hard at the trillion-token scale?

    Answer. Pairwise comparison of n documents is $O(n^2)$, infeasible for $n \sim 10^{10}$ documents. Approximate techniques (MinHash, SimHash) trade exact detection for sub-linear cost. Tuning the LSH threshold creates a precision/recall trade-off — some near-duplicates always slip through.

11. **(Q11)** Suppose you only have 100 GB of high-quality text data and want to train a $\sim$ 10B model. Should you (a) repeat the data many epochs, or (b) shrink to a smaller model?

    Answer. Both are suboptimal. (a) repeating data causes memorisation and degrades downstream performance (Hernandez et al., 2022). (b) shrinking the model is more consistent with Chinchilla scaling — a $\sim$ 1B model trained for $\sim$ 20B tokens (i.e., 200 epochs of 100GB, $\approx$ same total compute) is overfit. The right answer is to *augment the data* (more sources, synthetic generation, web crawling) so a 10B model can be properly trained on $\sim$ 200B unique tokens.

Numerical

12. **(Q12)** Compute the Kaplan test loss for $N = 1\text{B}$ and $D = 100\text{B}$ tokens.

    Solution.

$$\begin{aligned}
\frac{N_c}{N} &= 8.8 \times 10^{13}/10^9 = 8.8 \times 10^4, \\
(N_c/N)^{0.076/0.095} &= (8.8 \times 10^4)^{0.8} \approx \exp(0.8 \ln(8.8 \times 10^4)) \approx \exp(0.8 \cdot 11.39) \approx \exp(9.11) \approx 9{,}030. \\
\frac{D_c}{D} &= 5.4 \times 10^{13}/10^{11} = 540, \\
L &\approx (9{,}030 + 540)^{0.095} \approx (9570)^{0.095} \approx 2.46.
\end{aligned}$$

13. **(Q13)** Common Crawl releases monthly snapshots of $\sim$ 3–5B pages. If WET is $\sim$ 8.7 TiB per snapshot, and a pipeline keeps $\sim$ 5% of pages after filtering and deduplication, estimate the cleaned size in TB.

    Solution. $0.05 \times 8.7 \approx 0.44$ TiB. To put it in tokens, roughly 4 bytes per token gives $0.44 \times 10^{12}/4 \approx 1.1 \times 10^{11} \approx 110$ B tokens per cleaned snapshot. For a 1T training set, we would need $\sim$ 9 snapshots' worth.

14. **(Q14)** A 1.5T-token dataset is to be sampled with weights C4: 40%, Wikipedia: 10%, Books: 10%, OpenWebText: 15%, Code: 15%, Other: 10%. If C4 actually has 200B unique tokens, how many epochs of C4 will the model see?

    Solution. Total C4 tokens sampled $= 0.4 \times 1.5\text{T} = 0.6\text{T}$. Epochs $= 0.6\text{T}/200\text{B} = 3$. The model sees C4 three times. This is fine if Chinchilla-balanced; concerning if more.

15. **(Q15)** Suppose 0.1% of a 100B-token corpus is duplicated 100 times. Compute the effective training token count.

    Solution. Duplicates: $0.001 \times 100\text{B} = 100\text{M}$ unique sequences each appearing 100 times in the corpus. Their total token count: $\sim 100\text{M} \times 100 = 10\text{B}$ tokens (assuming a fixed average length). Unique tokens elsewhere: $\sim 99.9\text{B}$. Effective unique training data: $\sim 99.9\text{B} + 100\text{M} \approx 100\text{B}$, but *wasted* compute on 10B tokens that contribute little new signal — equivalent to training a 2×-smaller model in the empirical Hernandez et al. study.

8 Week 11 — Long Sequences, Complexity, and Fast Attention

Chapter goals. By the end of this chapter you will be able to:
1. Derive the $O(T^2 d)$ time and $O(T^2)$ memory complexity of self-attention.
2. Articulate the difference between FLOP reduction and *wall-clock* speedup.
3. Sketch approximate-attention families: **local**, **sparse**, **global + local**, **block**, **LSH (Reformer)**, **low-rank (Linformer)**, **kernel (Performer)**.
4. Explain GPU memory hierarchy (HBM vs SRAM) and why MHA is memory-bound.
5. Trace the **Flash Attention** algorithm step by step (tiling + online safe-softmax).
6. Compute KV-cache size and apply MQA, GQA, Paged Attention to reduce it.

8.1 Why Long Sequences are Hard

Most of the architectures we've studied have context lengths of 512–1024 tokens. But the world wants more:

- **Summarising a 64,000-word book.**
- **Answering a question from a 1000-page manual.**
- **Audio (10s of speech at 8 kHz = 80,000 samples; images viewed as patch sequences).**

Model	Context length
LLaMA-2	4K
Mistral-7B	8K
GPT-4, Gemini 1.0	32K
MosaicML MPT	65K
Anthropic Claude	100K
GPT-4 Turbo, Gemini 1.5 Pro	128K
Claude 2.1	200K
Gemini 1.5 Pro (limited)	2 M (public) / 10 M (experimental)

Intuition

We've moved from kilo-tokens to mega-tokens in just a few years. **Why is this hard?** The self-attention matrix is $T \times T$. At $T = 100$K that is 10 billion entries per head per layer — 40 GB just for one attention map in FP32. The **quadratic cost in** T is the fundamental bottleneck.

8.2 Time Complexity of Self-Attention

Recall the self-attention computation:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right), \qquad O = AV,$$

with $Q, K, V \in \mathbb{R}^{T \times d}$.

8.2.1 Step 1 — $QK^\top$

One row $\langle q_i, k_j \rangle$ is a d-dim dot product: d multiplications and $d-1$ additions. For one row of the attention matrix (all T columns): $T \cdot d$ multiplications and $T \cdot (d-1)$ additions. For all T rows: $T^2 \cdot d$ operations.

Formula Insight

Cost of $QK^\top$: $O(T^2 d)$ operations. Memory for the result $Z \in \mathbb{R}^{T\times T}$: $O(T^2)$.

8.2.2 Step 2 — softmax

Each row of Z is normalised by computing T exponentials and a sum. Total: $O(T^2)$ operations (one constant-time operation per entry). Dominated by step 1.

8.2.3 Step 3 — AV

A is $T \times T$ and V is $T \times d$, so $AV \in \mathbb{R}^{T\times d}$. Cost: $O(T^2 d)$.

8.2.4 Bottom line

Remember This

The entire self-attention block is **quadratic in sequence length**:

$$\boxed{\text{Time: } O(T^2 d), \quad \text{Memory: } O(T^2).}$$

For $T \gg d$, the quadratic term in T dominates and is the bottleneck.

8.3 Memory Footprint of MHA

For a batch size B, sequence length T, n_h heads, and per-head width d (so $d_{\text{model}} = n_h \cdot d$):

Tensor	Shape	Memory (one sample)
Q, K, V **each**	$T \times n_h \cdot d$	$T \cdot d_{\text{model}}$
Q, K, V **together**	—	$3T \cdot d_{\text{model}}$
Attention matrix A **(per head)**	$T \times T$	T^2
All heads of A	—	$n_h \cdot T^2$

For a batch of B:

$$\text{MHA memory} = \underbrace{3BT\, d_{\text{model}}}_{Q,K,V} + \underbrace{B\, n_h\, T^2}_{\text{attention matrices}}.$$

Memory scales linearly in B and quadratically in T. For $T = 100K$, $n_h = 32$, $B = 1$: attention matrix alone needs $32 \cdot 10^{10} = 3.2 \times 10^{11}$ floats ≈ 1.3 TB in FP32. Infeasible.

8.4 The Fast-Attention Wishlist

We want to drop from $O(T^2 d)$ to $O(cTd)$ where $c \ll T$ — ideally a constant or a small function of T (e.g., $\log T$). There are five broad strategies:

Family	Idea	Examples
Local attention	each token attends to nearby window of size c	Strided / Dilated
Global + local	a few "special" tokens see everything; rest see locals	Longformer
Global + random + local	add random sparse pattern on top	BigBird
Block / permutation	partition the attention matrix; mix blocks via permutation	Sparse Transformer, Blockwise MHA
Hash bucketing	LSH groups similar queries; attend within bucket	Reformer
Low-rank	approximate $A \approx Q'(K')^\top$ with $K' \in \mathbb{R}^{k\times d}$, $k \ll T$	Linformer
Kernel methods	approximate $\exp(q\,k^\top/\sqrt{d})$ via random features	Performer

Intuition

A common observation behind *all* these methods: "the attention matrix is mostly **sparse** or **low-rank** in practice" — empirical studies of BERT show that neighbouring tokens matter most and many heads don't use all positions.

Part A — Sparsifying the Attention Matrix

8.5 Strided Local Attention

Definition

Strided local attention restricts each token to attend only to the c tokens in a window centered on itself: $\lfloor c/2 \rfloor$ to the left and $\lfloor c/2 \rfloor$ to the right.

Complexity. Each query has c active keys, so total: $O(c \cdot T \cdot d)$. With c constant, this is **linear in** T.

8.5.1 Dilated attention

Like dilated convolutions, dilated attention skips positions: with dilation r, position i attends to positions $i \pm r, i \pm 2r, \ldots$. The receptive field grows exponentially when stacked in deeper layers. Same $O(cTd)$ cost.

8.5.2 Layer-wise window growth

A common trick: small windows ($c = 32$) in lower layers, full attention in the topmost layer. Local features build up locally; the final layer integrates globally.

8.6 Global + Local Attention (Longformer-style)

Intuition

For classification, the `[CLS]` token needs to see *everything* for a good prediction. But every other token only really needs nearby context. So: **a few special tokens get global attention; the rest stay local.**

Complexity. Let g be the number of global tokens. Then total cost is

$$O\big((cT + gT)d\big) \;\approx\; O\big((c+g)Td\big),$$

still linear in T when c, g are constants.

8.6.1 BigBird — Global + Random + Local

BigBird (Zaheer et al., 2020) adds a third pattern: each token also attends to a few *random* tokens. Three patterns combined give a theoretically powerful sparse attention that approximates full attention well. Complexity:

$$O((cT + gT + rT)d) \;=\; O((c+g+r)Td).$$

Research / Industry Insight

Theoretical guarantee. BigBird is a *universal approximator* of full attention sequences — in the sense that for any continuous mapping, there exists a BigBird configuration that approximates it arbitrarily well. The random component is crucial for the proof.

8.7 Local Block Attention with Permutations

Split the sequence into n contiguous blocks. By default, each block attends only to itself (block-diagonal). To capture longer-range dependencies, we apply a *permutation* to choose which K-block each Q-block attends to.

Definition

Block-attention with permutation π.

$$\text{BlockAtt}(Q, K, V; \pi)_i \;=\; \text{softmax}\left(\frac{Q_i K_{\pi(i)}^\top}{\sqrt{d}}\right) V_{\pi(i)}.$$

The masking matrix M_{ij} is 1 if $\pi(\lfloor in/T \rfloor) = \lfloor jn/T \rfloor$, else 0. There are $n!$ possible permutations of n blocks.

Complexity. Each Q-block $(T/n \times d)$ multiplies with one K-block of the same size: $O((T/n)^2 \cdot d)$ per block, times n blocks: $O(T^2 d/n)$. Quadratic in T but with a $1/n$ factor.

8.7.1 Blockwise multi-head attention

Different heads can use different permutations. This way the model captures multiple long-range patterns in parallel. With $n = 16$ blocks and $T = 32$, only 6.25% of the attention matrix is non-zero (93% sparse).

Intuition

Why permutations work: they connect arbitrary blocks across the sequence. Stacking layers compounds these connections, effectively giving every token an indirect path to every other token — much like dilated convolutions or BigBird's random component.

Part B — Hashing and Low-Rank Methods

8.8 Locality-Sensitive Hashing Attention (Reformer)

Observation. For a given query q_i, only a small subset of keys $\{k_j\}$ produce large attention weights. *Can we find those without computing all T dot products?* Yes — using **Locality-Sensitive Hashing** (LSH).

Definition

LSH attention (Reformer, Kitaev et al. 2020) hashes each query and key to a bucket such that *nearby* vectors share a bucket with high probability. Within each bucket, the model computes attention only among bucketed tokens. Total complexity: $O(T \log T)$.

8.8.1 The catch: $Q = K$

For LSH to work, we need each query to hash similarly to its preferred keys. The cleanest way is to **share weights**: $W_Q = W_K$, hence $Q = K$. This restricts the model class but works well in practice.

8.8.2 The LSH hash function

Define b hash buckets. Sample a random projection matrix $R \in \mathbb{R}^{d \times b/2}$. For any vector $x \in \mathbb{R}^d$:

$$\text{hash}(x) \;=\; \arg\max_{\ell} \big[\, xR \;\|\; -xR \,\big]_\ell,$$

i.e., concatenate xR with $-xR$ to get a length-b vector, then take the argmax.

8.8.3 Worked example

Let $d = 64$, $b = 4$. Sample $R \in \mathbb{R}^{64 \times 2}$. For two vectors q_1 and k_2 that are *far apart*, the projections are likely opposite:

$$\begin{aligned} q_1 R &\approx (0.5, 0.25), \quad -q_1 R \approx (-0.5, -0.25) \\ \text{concat} &= (0.5, 0.25, -0.5, -0.25), \quad \text{argmax} = 0 \\ k_2 R &\approx (-0.25, -0.5), \quad -k_2 R \approx (0.25, 0.5) \\ \text{concat} &= (-0.25, -0.5, 0.25, 0.5), \quad \text{argmax} = 3 \end{aligned}$$

Different buckets $\Rightarrow$ not close.

For two vectors q and k that *are* close, repeating the hash with several random R matrices gives the same bucket most of the time.

8.8.4 LSH algorithm summary

1. Hash every query (= key) to a bucket.
2. Sort tokens by bucket index, breaking ties by sequence position.
3. Within each bucket, perform standard attention (causal masking is still applied for autoregressive use).
4. Repeat with n_R random projections to recover queries that fell into the wrong bucket; average.

8.9 Linformer — Self-Attention is Low Rank

Wang et al. (2020) empirically computed the singular values of attention matrices in RoBERTa and found a *long-tail* distribution: a few singular values dominate. **Self-attention matrices are effectively low-rank.**

Definition

Linformer introduces two *learnable* projection matrices $E, F \in \mathbb{R}^{k \times T}$ with $k \ll T$, then redefines attention as

$$A \;=\; \mathrm{softmax}\!\left(\frac{Q(EK)^\top}{\sqrt{d}}\right), \qquad O \;=\; AFV.$$

Now $A \in \mathbb{R}^{T \times k}$ instead of $T \times T$. Complexity: $O(kTd)$ — linear in T.

Intuition

E projects the T-dimensional key axis down to k dimensions. So instead of attending to T keys, each query attends to k "compressed" keys — one per row of EK. Empirically $k = 256$ suffices even for $T = 4096$.

8.10 Performer — Kernel Methods for Linear Attention

Observation. The softmax row-stochastic constraint can be rewritten:

$$A_{ij} \;=\; \frac{\exp(q_i^\top k_j/\sqrt{d})}{\sum_{j'} \exp(q_i^\top k_{j'}/\sqrt{d})} \;=\; \frac{\mathcal{K}(q_i, k_j)}{\sum_{j'} \mathcal{K}(q_i, k_{j'})},$$

where $\mathcal{K}$ is a (positive-definite) *kernel function*.

8.10.1 The key trick: random feature maps

Any positive-definite kernel can be approximated as

$$\mathcal{K}(q, k) \;=\; \mathbb{E}\Big[\phi(q)^\top \phi(k)\Big],$$

where $\phi : \mathbb{R}^d \to \mathbb{R}^r_+$ is a *random feature map*. For the softmax kernel, the canonical choice (Performer, Choromanski et al. 2021) is

$$\phi(x) \;=\; \frac{1}{\sqrt{r}} \exp\!\big(Wx - \tfrac{1}{2}\|x\|^2\big),$$

with $W \in \mathbb{R}^{r \times d}$ where each row is sampled from $\mathcal{N}(0_d, I_d)$ and then orthogonalised (Gram-Schmidt).

8.10.2 The linear-time payoff

With this approximation, AV becomes

$$AV \;\approx\; \mathrm{diag}(\phi(Q)\phi(K)^\top \mathbf{1})^{-1} \cdot \phi(Q)\big(\phi(K)^\top V\big).$$

The key is that $\phi(K)^\top V$ has shape $r \times d$ and can be computed once in $O(Trd)$. Multiplying $\phi(Q)$ by this is another $O(Trd)$. **Total:** $O(Trd)$**, linear in** T**.**

Formula Insight

Why this works. The matrix multiplication is associative: $(QK^\top)V = Q(K^\top V)$. When the kernel factorises through ϕ, we can reorder the matrix products and avoid materialising the $T \times T$ attention matrix entirely.

Part C — Flash Attention: The IO-Aware Revolution

8.11 The Crucial Insight: FLOPs ≠ Wall-Clock Time

Intuition

All the approximations above reduce *FLOPs*. But empirically, halving the FLOPs *does not* halve training time! Why?
Because the bottleneck isn't compute — it's memory I/O. GPUs spend most of their MHA time *moving data* between memory locations, not actually multiplying matrices. To get a real speedup, you must reduce *memory accesses*, not just FLOPs.

8.11.1 GPU memory hierarchy

A modern GPU has two main memories:

Memory tier	Size	Bandwidth	Comment
HBM (High-Bandwidth Memory)	40–80 GB	~ 1.5 TB/s	Slow but big. "nvidia-smi" memory.
SRAM (on-chip cache)	$\sim$ 20 MB (split across cores)	~ 19 TB/s	Fast but tiny.

8.11.2 Why naive MHA is memory-bound

The standard MHA pipeline:

1. Read Q, K from HBM. Compute $S = QK^\top$. Write S back to HBM.
2. Read S from HBM. Compute $P = \text{softmax}(S)$. Write P back.
3. Read P, V from HBM. Compute $O = PV$. Write O back.

Every step does an HBM round trip. Element-wise operations (softmax, dropout, masking) are *slow not because of math* but because each entry must be fetched from HBM, processed, and written back. In benchmarks, softmax+dropout+mask take *more time than the matrix multiplications*!

8.11.3 Kernel fusion — the standard speedup

Fuse adjacent operations so intermediate results stay in SRAM:

$$c = ab \to d = \sqrt{c} \quad \Rightarrow \quad \text{fused: read } a, b\text{; compute } \sqrt{ab}\text{; write } d.$$

Only *two* HBM accesses instead of four.

8.12 Flash Attention — Exact Attention via Tiling

Definition

Flash Attention (Dao et al., 2022) computes the *exact same* attention output as standard MHA, but with $O(N^2d^2/M)$ HBM accesses instead of $O(Nd+N^2)$, where M is the SRAM size. This often gives a 2–4× wall-clock speedup *and* reduces memory from $O(N^2)$ to $O(N)$.

Two innovations:

1. **Tiling.** Divide Q, K, V into blocks small enough to fit in SRAM.

2. **Online safe-softmax.** Compute softmax incrementally as new blocks arrive, using running max and sum statistics.

8.12.1 The online safe-softmax

Recall safe-softmax: subtract the row max before exponentiating to avoid overflow:

$$\text{softmax}(x)_i = \frac{\exp(x_i - m)}{\sum_j \exp(x_j - m)}, \quad m = \max_j x_j.$$

For Flash Attention we cannot see the whole row at once (we are processing blocks). The trick is to maintain a *running* max m and a running denominator ℓ, updating them as we see each block.

Worked split-merge example. Split a length-10 logit vector $x = [x_A, x_B]$ with $x_A = [-1.02, -1.09, 0.24, 0.87, -1.32]$ and $x_B = [-0.60, -5.04, -2.10, 1.03, -0.14]$.

Process block A:

$$\begin{aligned} m_1 &= \max(x_A) = 0.87, \\ f_1 &= [\exp(x_i^A - m_1)]_i = [0.15, 0.14, 0.34, 1.00, 0.11], \\ \ell_1 &= \sum_i f_{1,i} = 1.74. \end{aligned}$$

Process block B:

$$\begin{aligned} m_2 &= \max(x_B) = 1.03, \\ f_2 &= [0.20, 0.0024, 0.043, 1.00, 0.31], \\ \ell_2 &= \sum_i f_{2,i} = 1.56. \end{aligned}$$

Combine. New global max $m = \max(m_1, m_2) = 1.03$. Rescale each block's contributions to use the new global max:

$$\begin{aligned} \beta_1 &= \exp(m_1 - m) = \exp(0.87 - 1.03) = 0.85, \\ \beta_2 &= \exp(m_2 - m) = 1, \\ f &= [\beta_1 f_1 \parallel \beta_2 f_2], \\ \ell &= \beta_1 \ell_1 + \beta_2 \ell_2 \;=\; 0.85 \cdot 1.74 + 1 \cdot 1.56 \;\approx\; 3.04. \end{aligned}$$

The final normalised softmax is f/ℓ. This is **exact**, not approximate — the math is identical to what naive softmax would have produced.

Remember This

The online-softmax identity: $\text{softmax}(x_A \| x_B) = \frac{[\beta_A f_A \| \beta_B f_B]}{\beta_A \ell_A + \beta_B \ell_B}$ where $\beta_A = e^{m_A - m}, \beta_B = e^{m_B - m}, m = \max(m_A, m_B)$. This lets us compute softmax over an unbounded stream by maintaining only running (m, ℓ).

8.12.2 The Flash Attention algorithm

1. **Inputs in HBM:** $Q, K, V \in \mathbb{R}^{N \times d}$. SRAM size M.
2. **Block sizes:** $B_c = \lceil M/(4d) \rceil$, $B_r = \min(B_c, d)$. *Example:* $d = 64$, $M = 100$ KB, 4 bytes/element $\Rightarrow B_c = 100$, $B_r = 64$.

3. **Initialise** in HBM: output $O \in \mathbb{R}^{N \times d}$ to zeros, running denominators $\ell \in \mathbb{R}^N$ to zeros, running maxes $m \in \mathbb{R}^N$ to $-\infty$.
4. **Divide** Q into $T_r = \lceil N/B_r \rceil$ row blocks; K, V into $T_c = \lceil N/B_c \rceil$ column blocks.
5. **For each** K_j, V_j **block** (outer loop, $j = 1, \ldots, T_c$):
   - Load K_j, V_j from HBM to SRAM.
   - **For each** Q_i **block** (inner loop, $i = 1, \ldots, T_r$):
     - Load Q_i, O_i, ℓ_i, m_i from HBM to SRAM.
     - Compute $S_{ij} = Q_i K_j^\top$ on chip.
     - Compute block-wise statistics $m_{ij}, \ell_{ij}, P_{ij}$ on chip.
     - Update m_i, ℓ_i to reflect the new block.
     - Update O_i using $f(O_i, P_{ij}, V_j, m_i, \ell_i)$.
     - Write O_i, m_i, ℓ_i back to HBM.
6. **Return** O.

8.12.3 The big win

- **HBM accesses:** standard attention needs $O(Nd + N^2)$. Flash Attention needs $O(N^2 d^2 / M)$ — typically *less* because $M \gg d$.
- **Memory:** no $N \times N$ attention matrix is ever materialised in HBM. Memory footprint drops from $O(N^2)$ to $O(N)$.
- **Backward pass:** recomputation via the cached (m, ℓ) statistics.
- **Empirical:** 2–4× training speedup; context length jumps from $1K$ to $4K+$ in the same memory budget.

Research / Industry Insight

Flash Attention is *exact*. Unlike Linformer, Reformer, Performer, etc., which are *approximate*, Flash Attention computes the bit-identical result of standard attention. It is a faster *algorithm*, not a different mathematical operation. This is why it's the default in PyTorch 2+ and most modern training frameworks.

Part D — Fast Inference: KV Cache, MQA, GQA, Paged

8.13 Why Inference Needs a Different Treatment

Intuition

During training we have the whole target sequence (teacher forcing) so we can do a single parallel forward pass. During *inference* (autoregressive generation), we process one new token at a time. The naive approach recomputes the key-value pairs for *all* previous tokens at every step — a massive waste, since they don't change.

8.13.1 The naive inference pipeline

At time t, with context $x_0, x_1, \ldots, x_{t-1}$ already generated:

- Embed all t tokens.
- Compute $W_Q h, W_K h, W_V h$ for all t tokens.
- Run attention to predict x_t.

At time $t+1$, repeat for $t+1$ tokens. **We are recomputing the same K/V pairs over and over.**

8.14 KV Caching

Definition

KV caching: After computing k_i, v_i for token i, store them in GPU memory. At time t, only compute k_t, v_t for the *new* token; reuse cached $k_1, \ldots, k_{t-1}$ and $v_1, \ldots, v_{t-1}$ for attention. Compute scales as $O(t \cdot d)$ per step rather than $O(t^2 \cdot d)$.

8.14.1 Cache size — the cost of KV caching

Formula Insight

$$\text{KV cache per token (bytes)} \;=\; 2 \cdot n_{\text{layers}} \cdot (n_{\text{heads}} \cdot d_{\text{head}}) \cdot \text{precision_bytes}.$$

The factor 2 is for the two vectors (k and v) per token per head per layer.

Example: GPT-3 cache. $n_{\text{layers}} = 96$, $n_{\text{heads}} = 96$, $d_{\text{head}} = 128$, FP32 precision (4 bytes):

$$2 \cdot 96 \cdot 96 \cdot 128 \cdot 4 \;=\; 9{,}437{,}184 \;\approx\; 9.4 \text{ MB per token.}$$

For a 2K-token context: $2048 \cdot 9.4 = 19.3$ GB. That is *just the cache* — the model weights also need to live in GPU memory. An A100 with 40 GB cannot fit both.

Quiz Trap

Trap. "KV caching is free."
Reality. It trades compute for memory. The compute drops dramatically (one step is now $O(td)$ not $O(t^2 d)$) but memory grows linearly with context length and is paid in the most expensive memory (GPU HBM).

8.15 Multi-Query Attention (MQA)

The KV cache formula has three hyperparameters: n_{layers}, $n_{\text{heads}} \cdot d_{\text{head}} = d_{\text{model}}$, and precision. To shrink the cache, MQA sets $n_{\text{KV-heads}} = 1$: all attention heads *share* a single K and V vector, while each head retains its own query.

Definition

Multi-Query Attention. All n_h heads share one K vector and one V vector per token. Cache shrinks from $2 \cdot n_h \cdot d_{\text{head}}$ to $2 \cdot d_{\text{head}}$ per layer per token — an $n_h\times$ reduction.

Trade-off. Quality typically drops a few BLEU points. Solution: **uptraining** (continue pre-training the MQA-converted model on $\sim 5\%$ of the original data) recovers most of the lost quality.

8.16 Grouped-Query Attention (GQA)

GQA (Ainslie et al., 2023) splits the heads into g groups. All heads within a group share one K and one V.

- $g = n_h$: standard MHA. Largest cache, best quality.
- $g = 1$: MQA. Smallest cache, lower quality.
- $g = n_h/8$ (LLaMA-2 default): cache shrinks by $8\times$, quality preserved almost fully.

Remember This

Default in modern LLMs. LLaMA-2, LLaMA-3, Mistral, and most production decoder-only models use **GQA**. The number of KV-head groups is usually 4 or 8.

8.17 Paged Attention

Intuition

The naive KV-cache implementation reserves *contiguous* GPU memory for the maximum possible context length *per request*. If the actual output is shorter, the rest is wasted. Worse, when serving many requests in parallel, fragmentation makes it impossible to allocate large contiguous chunks.

Definition

Paged Attention (Kwon et al., 2023, vLLM) borrows the idea of *virtual memory pages* from OS design. The KV cache is split into fixed-size *pages* (e.g., 16 tokens each). When a request needs more, a new page is allocated. Pages can be *non-contiguous* in physical memory; a page table maps them. Different requests can even share pages (for shared prompts).

Practical impact. vLLM (the most popular inference server using Paged Attention) achieves 2–24× higher throughput than systems using contiguous KV-cache allocation. It is the standard for production LLM serving.

8.18 Comparison: Approximate vs Exact Fast Attention

Method	Time complexity	Memory complexity	Approximation?	Used by
Full attention	$O(T^2 d)$	$O(T^2)$	exact	all original Transformers
Local / strided	$O(cTd)$	$O(cT)$	yes (limited window)	Longformer
Global + local	$O((c+g)Td)$	$O((c+g)T)$	yes	Longformer, ETC
BigBird	$O((c+g+r)Td)$	$O((c+g+r)T)$	yes (universal approx.)	BigBird
LSH	$O(T \log T d)$	$O(T \log T)$	yes ($Q=K$)	Reformer
Linformer	$O(kTd)$	$O(kT)$	yes (low-rank)	Linformer
Performer	$O(rTd)$	$O(rT)$	yes (kernel)	Performer
Flash Attention	$O(T^2 d)$	$O(T)$	**exact**	PyTorch 2+, vLLM, all modern

Remember This

The crucial insight. Flash Attention is the only method on this list that is *both* exact *and* memory-linear. All the others either sacrifice exactness (approximation) or are still memory-quadratic. This is why Flash Attention has displaced most of the approximate methods in practice.

8.19 Common Mistakes & Quiz Traps — Week 11

Common Mistake

Mistake 1. "Flash Attention is an approximate attention."
Fix. It is *exact*. The output is bit-identical to standard attention. What changed is the *order* of operations and the use of SRAM tiling, not the math.

Common Mistake

Mistake 2. "Reducing FLOPs by $2\times$ halves training time."
Fix. Often it does not, because MHA is *memory-bound*, not compute-bound. The bottleneck is HBM I/O, not arithmetic. Many sub-quadratic approximations were disappointing in practice because they reduced FLOPs but not HBM accesses.

Common Mistake

Mistake 3. "KV caching is a model architecture change."
Fix. It's an *inference* optimisation. The trained model is unchanged; we just remember intermediate k, v vectors to skip recomputation.

Quiz Trap

Trap. "MQA always degrades quality significantly."
Reality. MQA from scratch can lose meaningful quality, but with $\sim 5\%$ uptraining most of the gap closes. GQA (a compromise) often matches MHA quality entirely while still cutting cache by $8\times$.

Quiz Trap

Trap. "If c is constant, local attention's $O(cTd)$ matches Flash Attention's performance."
Reality. They have different quality. Local attention is *approximate* (some long-range dependencies are lost). Flash Attention is exact. Use Flash for quality + exactness; use local methods only when extreme context lengths ($T > 100K$) force the trade-off.

8.20 Memory Tricks — Week 11

Memory Trick

"Attention is $O(T^2d)$ time and $O(T^2)$ memory."
The two-line summary of the entire bottleneck.

Memory Trick

"Sparse → LSH → low-rank → kernel → Flash".
The historical progression: sparsify (Longformer), then approximate via hashing (Reformer), then via low-rank (Linformer), then via kernel features (Performer), then finally make it *exact* and IO-aware (Flash Attention).

Memory Trick

"HBM is huge and slow; SRAM is tiny and fast."
The whole motivation for Flash Attention is to keep computation in SRAM and minimise HBM round-trips.

Memory Trick

"KV cache $= 2 \cdot L \cdot H \cdot D \cdot$ **precision per token".**
Memorise this formula. For LLaMA-2 70B, this is $\sim$ 80 KB per token — 80 MB per 1000-token context. Multiply by batch size and you understand why MQA/GQA exist.

8.21 Quick Revision — Week 11 in One Page

Quick Revision

Complexity. Self-attention is $O(T^2 d)$ time, $O(T^2)$ memory. Quadratic in T is the bottleneck.

Approximate-attention families.

- **Local** (strided / dilated): $O(cTd)$. Loses long-range dependencies.
- **Global + local** (Longformer): special tokens see everything; rest see locals. $O((c+g)Td)$.
- **BigBird:** adds random tokens. Universal approximator.
- **Block attention** with permutations: $O(T^2 d/n)$.
- **LSH (Reformer):** hash buckets, $O(T \log T)$. Requires $Q = K$.
- **Linformer:** low-rank projection $E \in \mathbb{R}^{k \times T}$, $O(kTd)$.
- **Performer:** random feature map ϕ, $O(rTd)$.

Flash Attention. Exact, IO-aware. Uses *tiling* (Q, K, V in SRAM blocks) and *online safe-softmax* (running m, ℓ). Memory drops from $O(T^2)$ to $O(T)$. Wall-clock 2–4× faster.

Online safe-softmax. $\text{softmax}(x_A \| x_B) = (\beta_A f_A \| \beta_B f_B)/(\beta_A \ell_A + \beta_B \ell_B)$, $\beta_A = e^{m_A - m}, \beta_B = e^{m_B - m}, m = \max(m_A, m_B)$.

Inference optimisations.

- **KV cache:** store k, v per token to skip recomputation. Size $= 2LHD \cdot$ precision per token.
- **MQA:** all heads share one K, V. $n_h\times$ cache reduction; some quality loss.
- **GQA:** groups of heads share K, V. Sweet spot of MHA/MQA. Standard in LLaMA-2/3.
- **Paged Attention:** non-contiguous KV cache via virtual pages. vLLM standard.

8.22 Practice Questions — Week 11

Multiple Choice

1. **(Q1)** The time complexity of standard self-attention for a single head is:

   (A) $O(Td)$ (B) $O(T \log T)$ (C) $O(T^2 d)$ (D) $O(T^3)$.

   Solution. (C).

2. **(Q2)** Flash Attention is best described as:

   (A) a sparse approximation of attention,

   (B) an exact attention computed with IO-aware tiling,

   (C) a kernel-based approximation,

(D) a low-rank approximation.

Solution. (B).

3. **(Q3)** GQA is preferred over MQA in modern LLMs because:

   (A) GQA gives lower KV cache than MQA,

   (B) GQA preserves more quality than MQA while still reducing cache,

   (C) GQA uses no shared K, V vectors,

   (D) GQA is identical to MHA.

   Solution. (B). GQA finds a sweet spot between the two extremes.

4. **(Q4)** The Reformer (LSH attention) achieves what asymptotic complexity?

   (A) $O(T)$ (B) $O(T \log T)$ (C) $O(T^2)$ (D) $O(T \log^2 T)$.

   Solution. (B).

5. **(Q5)** Reducing FLOPs by $2\times$ in MHA does not halve training time because:

   (A) modern GPUs always run at fixed clock,

   (B) MHA is memory-bound, not compute-bound,

   (C) PyTorch overhead dominates,

   (D) attention weights need re-initialisation.

   Solution. (B).

6. **(Q6)** BigBird achieves linear-time attention by combining:

   (A) only local attention,

   (B) local + global tokens only,

   (C) local + global + random tokens,

   (D) only random tokens.

   Solution. (C).

Conceptual / Short Answer

7. **(Q7)** Explain in two sentences why Flash Attention can outperform an approximate method like Performer despite both being linear-memory.

   Answer. Flash Attention is *exact* (same gradients, same loss curves) while Performer is approximate (different loss curves, sometimes worse downstream quality). Additionally, Flash's tiling matches GPU memory hierarchy so its constant factors are excellent; approximations often have higher constant factors and only win asymptotically.

8. **(Q8)** Why must LSH attention share Q and K projection matrices?

   Answer. LSH groups vectors into buckets so that similar vectors share a bucket with high probability. For a query q to find its *relevant* keys, q must hash to the same bucket as those keys — which requires q and k to be drawn from the same distribution. Sharing weights ($W_Q = W_K$, so $Q = K$) ensures this.

9. **(Q9)** Sketch the online safe-softmax in 3 bullets.

   Answer.

   - Maintain running max m and running sum ℓ.
   - When a new block of logits arrives with local max m', scale the previous sum by $e^{m-m'}$ if $m' > m$, then update $m \leftarrow m'$ and add the new block's contribution.
   - Final softmax = (running numerators) / (running sum), no full row needed at any step.

10. **(Q10)** Why does Linformer assume the attention matrix is low-rank, and how is this verified?

    Answer. Empirically (Wang et al., 2020), the singular values of attention matrices follow a long-tail distribution — a few values dominate. They confirmed this by running SVD on attention matrices of a trained RoBERTa across many samples and observing the cumulative singular-value plot saturates within the first ~ 128 ranks out of 512.

11. **(Q11)** A 7B-parameter LLaMA model uses GQA with 32 query heads and 8 KV heads, $d_{\text{head}} = 128$, 32 layers, FP16 precision. Compute the KV-cache size per token.

    Solution. Cache/token $= 2 \times n_{\text{layers}} \times (n_{\text{KV-heads}} \cdot d_{\text{head}}) \times \text{bytes} = 2 \times 32 \times (8 \cdot 128) \times 2 = 2 \times 32 \times 1024 \times 2 = 131{,}072 \approx 128$ KB per token.

Numerical / Derivation

12. **(Q12)** For $T = 4096$, $d = 64$, compute the number of FLOPs in standard attention (just $QK^\top$, ignoring softmax and AV).

    Solution. $QK^\top$ produces a $T \times T = 4096 \times 4096$ matrix, each entry is a dot product of two $d = 64$-dim vectors. FLOPs per entry: $\sim 2d = 128$. Total: $4096 \times 4096 \times 128 \approx 2.1 \times 10^9$ FLOPs.

13. **(Q13)** Verify the online-softmax identity by computing softmax$([1, 3, 2])$ both (a) directly and (b) by treating $[1, 3]$ and $[2]$ as separate blocks and combining.

    Solution. (a) Directly. $m = 3$, $e^{x-m} = [e^{-2}, 1, e^{-1}] \approx [0.135, 1, 0.368]$, sum ≈ 1.503. softmax $\approx [0.090, 0.665, 0.245]$.
    (b) Online. Block A $= [1, 3]$: $m_A = 3$, $f_A = [e^{-2}, 1] = [0.135, 1]$, $\ell_A = 1.135$. Block B $= [2]$: $m_B = 2$, $f_B = [1]$, $\ell_B = 1$. Combine: $m = 3$, $\beta_A = e^{3-3} = 1$, $\beta_B = e^{2-3} = e^{-1} = 0.368$. $f = [\beta_A \cdot f_A \| \beta_B \cdot f_B] = [0.135, 1, 0.368]$, $\ell = 1 \cdot 1.135 + 0.368 \cdot 1 = 1.503$. $f/\ell \approx [0.090, 0.665, 0.245]$. **Match!** The identity holds.

14. **(Q14)** A GPT-3-sized model has $n_{\text{layers}} = 96$, $n_{\text{heads}} = 96$, $d_{\text{head}} = 128$, FP16 precision. If you switch from MHA to GQA with $n_{\text{KV-heads}} = 12$, what is the cache-size-per-token ratio between the two?

    Solution. Ratio $= n_{\text{heads}}/n_{\text{KV-heads}} = 96/12 = 8\times$. The KV-cache shrinks by $8\times$. For a 2K context, that's 19.3 GB/8 $\approx$ 2.4 GB.

15. **(Q15)** A Performer with $r = 256$ random features, $T = 4096$, $d = 64$. Compute the wall-clock cost ratio vs full attention.

    Solution. Full attention: $T^2 d = 4096^2 \times 64 \approx 1.07 \times 10^9$ FLOPs. Performer: $rTd = 256 \times 4096 \times 64 \approx 6.7 \times 10^7$ FLOPs. Ratio: $\sim 16\times$ fewer FLOPs. But *wall-clock* may not be $16\times$ faster if the operation remains memory-bound — which is exactly Flash Attention's lesson.

9 Week 12 — Position Encodings: APE, RPE, RoPE, ALiBi, NoPE, and Length Generalization

Chapter goals. By the end of this final chapter you will be able to:

1. Articulate the two key limitations of *Absolute Position Encoding* (APE): shift sensitivity and the inability to extrapolate beyond training length.
2. Define *Relative Position Encoding* (RPE) and trace its three landmark variants (Shaw et al., T5-bias, Transformer-XL).
3. Derive **Rotary Position Embedding (RoPE)** from first principles, including the complex-exponential setup and the 2D rotation matrix.
4. Apply **ALiBi** (Attention with Linear Biases) and explain why it acts like a soft local-windowed attention.
5. Discuss the surprising result that **NoPE** (no positional encoding) can still work for decoder-only models.
6. Compare these methods on the central practical question of *length generalization*.

9.1 Recap and Why This Matters

In Week 2 we derived the original *sinusoidal* positional encoding:

$$PE(j, 2i) = \sin\big(j/10000^{2i/d}\big), \quad PE(j, 2i+1) = \cos\big(j/10000^{2i/d}\big).$$

We added it to the word embedding $h_j = x_j + p_j$ and moved on. *This chapter asks: was that a good choice?*

The answer turns out to be "no" for several practical reasons — and the cumulative fixes since 2018 are why modern LLMs (LLaMA, PaLM, Mistral, GPT-3+) all use either RoPE or ALiBi, never the original sinusoidal scheme.

9.1.1 Three questions every PE method must answer

- **How to encode?** Vector addition, multiplication, or as a bias on attention scores?
- **What to encode?** Absolute positions $0, 1, 2, \ldots$, or relative distances $i - j$ between token pairs?
- **Fixed or learned?** Hard-coded sinusoids, or trainable parameters?

9.1.2 The 2×2 design space

	Fixed (non-parametric)	**Learnable (parametric)**
Absolute	Vanilla Transformer	BERT, GPT-x, BART, OPT
Relative	T5-bias, RoPE, ALiBi, NoPE	Transformer-XL, Shaw et al.

Intuition

Notice that all modern LLMs (LLaMA, PaLM, BLOOM, GPT-J, Mistral) live in the **Fixed Relative** cell. The trajectory of the field has been: APE → RPE → *fixed* RPE. Each move solves a real problem with the previous step.

Part A — Limitations of Absolute Position Encoding

9.2 Shift Sensitivity — A First Symptom

Experimental setup. Take a pre-trained GPT-2 (which uses learned APE). Evaluate on a grammatical acceptability test using perplexity. Then *shift the entire input* by k positions to the right:

- Original positions: $0, 1, 2, \ldots, 7$.
- Shifted by $k = 100$: $100, 101, 102, \ldots, 107$.

The sentence itself is identical — only the position indices have changed.

Expectation. If the model truly learned to attend "relative-positionally", the prediction should be *invariant* to such a shift. Performance should stay constant.

Observation. *Performance degrades as k grows* (Sinha et al., 2022 and follow-ups). The model treats each absolute position as a unique signal — so a sentence starting at position 100 looks "alien" to the model, even if it would have been perfectly fine starting at position 0.

Remember This

Diagnosis 1. Learned APE does *not* learn relative-position invariance. Sentences starting at different absolute positions get genuinely different predictions. This is a real bug, not just a theoretical concern.

9.3 Length Extrapolation — The Bigger Problem

Setup. Pre-train a GPT model with a context length of $T = 512$. Now ask it to generate a 1024-token story.

Failure. Positions 513 through 1024 *never appeared during training.* For learned APE, the position embeddings for those positions are still random (or zero). The model has no idea what to do with them. **Output quality collapses.**

Why this is a big deal. Re-training the model with $T = 1024$ is computationally expensive (more memory, more time). And then you cannot extend to $T = 4096$ without yet another re-training.

9.3.1 Arithmetic generalization — a sharper test

The lecturer used a delightfully concrete example. Train a Transformer to add two n-digit integers. With training data only up to $n = 4$ digits:

$$9999 + 0001 = 10000.$$

Can the model generalize to $n = 12$ digit addition? *Yes*, if it has properly learned the algorithm of carry-and-add. *The type of position encoding is a major factor in this ability, and APE is the worst choice.*

Quiz Trap

Trap. "Length generalization is just an evaluation artefact."
Reality. Length generalization gates real-world use cases: summarizing arbitrary-length documents, processing log files of unknown size, reasoning over mathematical proofs. APE-based models silently fail at all of these beyond their training context. RoPE / ALiBi / NoPE all degrade more gracefully.

Part B — Relative Position Encoding (RPE)

9.4 The RPE Idea

Intuition

APE assigns each token a vector based on *where it sits* in the sequence. RPE assigns each *pair* of tokens a vector based on the *distance* between them.
For the sentence "somewhere something incredible is waiting", the word **"is"** (at position 3) sees:

- "somewhere" at relative distance -3.
- "something" at relative distance -2.
- "incredible" at relative distance -1.
- itself at relative distance 0.
- "waiting" at relative distance $+1$.
- etc.

This means the model now has a representation of pairwise relationships.

For a sequence of length T, there are $2T-1$ possible relative distances $(-(T-1), \ldots, -1, 0, 1, \ldots, T-1)$, versus only T possible absolute positions for APE.

9.4.1 The naive cost

If we added T relative-position embeddings to each token, we would do T additions per token $= T^2$ additions total. Way too expensive. We need a smarter way — and the trick is to inject the position information *directly into the attention score*, not into the input embedding.

9.5 Modifying the Pre-Attention Score

Recall the unmodified pre-attention score:

$$e_{ij} = q_i k_j^\top = (x_i W_Q)(x_j W_K)^\top.$$

With APE, this becomes:

$$\begin{aligned} e_{ij} &= (x_i + p_i)\, W_Q\, W_K^\top\, (x_j + p_j)^\top \\ &= \underbrace{x_i W_Q W_K^\top x_j^\top}_{\text{word-word}} + \underbrace{x_i W_Q W_K^\top p_j^\top}_{\text{word-position}} + \underbrace{p_i W_Q W_K^\top x_j^\top}_{\text{position-word}} + \underbrace{p_i W_Q W_K^\top p_j^\top}_{\text{position-position}}. \end{aligned}$$

Intuition

The decomposition reveals the problem. APE introduces three extra terms involving position-position and word-position correlations. In BERT, the position-to-position term is roughly *uniform* (no useful signal) — positional artefact.
What if we kept only the meaningful information and dropped the noise?

9.5.1 Shaw et al.'s formulation

Shaw et al. (2018) proposed dropping the input-level addition and instead injecting positions into the attention score:

$$\boxed{e_{ij} = x_i W_Q \left(x_j W_K + p_{ij}^K\right)^\top,} \tag{30}$$

where $p_{ij}^K \in \mathbb{R}^d$ is a learnable relative-position embedding shared across heads (but not across layers).

9.5.2 Distance clipping

The size of the relative-position embedding matrix would still grow with T. To make it independent of context length, Shaw et al. *clip* all distances beyond a fixed cutoff k:

$$\text{clip}(j-i,k) = \max(-k, \min(j-i,k)).$$

Then $p_{ij}^K = w^K_{\text{clip}(j-i,k)}$ uses only $2k+1$ embeddings, independent of T.

Empirical finding. Replacing APE in the Transformer with this RPE (with $k = 16$) improves performance on translation benchmarks. Clipping does not hurt: words very far apart already have little useful interaction.

9.6 Three RPE Variants Worth Knowing

9.6.1 Shaw et al. (2018) — learnable vector p_{ij}

As above. Position embeddings are d-dimensional, learnable, clipped at distance k.

9.6.2 Transformer-XL (Dai et al., 2019)

Decomposes the attention score into four interpretable terms:

$$e_{ij} = x_i W_Q W_K^\top x_j^\top + x_i W_Q R_{j-i}^\top + u\, W_K^\top x_j^\top + v\, W_K\, R_{j-i}^\top,$$

where:

- R_{j-i} is a sinusoidal (*non-learnable*) function of the relative distance.
- u, v are learnable global vectors.

This was paired with a *segment-level recurrence* that lets the model attend to hidden states from previous segments, extending the effective context length.

9.6.3 T5-bias (Raffel et al., 2020) — a scalar is enough

The radical simplification of T5: just add a scalar bias per relative distance.

$$\boxed{e_{ij} = x_i W_Q W_K^\top x_j^\top + r_{j-i},} \tag{31}$$

where $r_{j-i} \in \mathbb{R}$ is a single learnable scalar. Distance is clipped to 128 (and bucketed logarithmically beyond a small near-range).

Why this works. Most of the positional "correlation" encodes how strongly two positions should interact. A single scalar per distance is enough to capture this. Massive reduction in parameter count.

Research / Industry Insight

The T5-bias is shared across layers — a deliberate decision that further reduces parameters and helps with generalisation. T5-bias was the dominant choice for encoder-decoder models for several years.

Part C — Rotary Position Embedding (RoPE)

9.7 The Key Insight: Encode Position via Rotation

Intuition

APE rotates word vectors by an angle that grows with absolute position. RoPE has the same intuition but in a much cleaner form:
The angle between q_i and k_j should depend only on $(i-j)$, not on i or j separately.
This is exactly what we want for relative-position-aware attention: the *inner product* (which the attention score uses) is invariant to global shifts.

9.7.1 The functional requirement

We want functions $f_q(x_i, i)$ and $f_k(x_j, j)$ such that:

$$\langle f_q(x_i, i),\ f_k(x_j, j)\rangle \ =\ g(x_i, x_j,\ j-i) \tag{32}$$

i.e., the inner product depends only on the relative distance $j-i$, not on i and j individually.

9.7.2 The complex-exponential ansatz

Try

$$f_q(x_m, m) = (x_m W_Q)\, e^{jm\theta}, \quad f_k(x_n, n) = (x_n W_K)\, e^{jn\theta},$$

where θ is a non-zero constant, j is the imaginary unit (we use m, n here to avoid confusion with j), and $e^{jm\theta}$ is a complex scalar.

The inner product (Hermitian, using complex conjugate):

$$\begin{aligned}\langle f_q, f_k\rangle &= (x_m W_Q)\, e^{jm\theta} \cdot \overline{(x_n W_K\, e^{jn\theta})} \\ &= (x_m W_Q)(x_n W_K)^\top\, e^{jm\theta - jn\theta} \\ &= (x_m W_Q W_K^\top x_n^\top)\, e^{j(m-n)\theta}.\end{aligned}$$

Look! The result depends only on $(m-n)$, exactly what we wanted. The imaginary part is symmetric and cancels in the real-valued attention; the real part provides the actual score.

9.8 From Complex Numbers to Real 2D Rotations

A complex exponential $e^{jm\theta} = \cos(m\theta) + j\sin(m\theta)$ corresponds to a 2D rotation by angle $m\theta$. So we can implement the same idea in real-valued arithmetic using:

$$R_{m\theta} \ =\ \begin{bmatrix}\cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta)\end{bmatrix}. \tag{33}$$

For $x_m \in \mathbb{R}^2$:

$$f_q(x_m, m) = R_{m\theta}\, W_Q\, x_m. \tag{34}$$

9.8.1 The inner product becomes a single rotation

$$\langle f_q(x_m, m), f_k(x_n, n)\rangle \ =\ x_m^\top W_Q^\top\, R_{m\theta}^\top\, R_{n\theta}\, W_K\, x_n.$$

Because rotation matrices satisfy $R_a^\top R_b = R_{b-a}$:

$$R_{m\theta}^\top R_{n\theta} = R_{(n-m)\theta},$$

so

$$\boxed{q_m^\top k_n \;=\; x_m^\top W_Q^\top \, R_{(n-m)\theta} \, W_K \, x_n,} \tag{35}$$

which depends *only on* $(n-m)$.

9.9 Generalising RoPE to $d > 2$

For d-dimensional inputs, pair up dimensions $(1,2),(3,4),\ldots,(d-1,d)$ and apply a separate rotation to each pair, each with its own frequency θ_i:

$$R^d_{\Theta,m} \;=\; \text{block-diag}\Big(R_{m\theta_1},\; R_{m\theta_2},\; \ldots,\; R_{m\theta_{d/2}}\Big),$$

where the frequencies follow the familiar geometric progression from sinusoidal APE:

$$\theta_i \;=\; 10000^{-2(i-1)/d}, \quad i \in \{1,2,\ldots,d/2\}.$$

Remember This

Why RoPE wins.

- **No added position embeddings** in the input. RoPE multiplies $W_Q x$ and $W_K x$ by rotation matrices.
- **Inner product is relative**: $\langle q_m, k_n \rangle$ depends only on $n-m$, naturally.
- **No new parameters** (rotations are fixed by θ_i).
- **Applied at every layer**, not just at input.
- **Extrapolates better** than APE, though far from perfect.

9.9.1 Implementation note

RoPE is implemented efficiently by interleaving dimensions and using element-wise operations:

- Pre-compute $\cos(m\theta_i)$ and $\sin(m\theta_i)$ for all m and i.
- For each token's q or k vector, multiply paired dimensions by the cos / sin values using a vectorised reshape.
- No matrix multiplications by full $d \times d$ rotation matrices.

Used by: LLaMA-1/2/3, PaLM, GPT-J, GPT-NeoX, Mistral, Qwen, Yi, DeepSeek, Gemma, ... Essentially every major open LLM since 2022.

Part D — ALiBi: Attention with Linear Biases

9.10 The ALiBi Idea

ALiBi (Press, Smith, Lewis, 2021) takes the simplest possible approach to encoding position information.

Intuition

T5-bias adds a *learnable* scalar r_{j-i} to each attention score. **What if we just hard-code that scalar?** The relationship between tokens should be "more attention to nearby tokens, less to distant ones". The simplest version is a linear penalty proportional to the distance.

Definition

ALiBi attention computes:

$$\text{softmax}\Big(q_i K^\top + m \cdot \big(-(i-1), \ldots, -2, -1, 0\big)\Big),$$

where:

- The bias is a vector of negative values proportional to the distance: 0 at the current position, -1 one step back, -2 two steps back, etc.
- m is a *head-specific* positive constant; different heads use different m values.
- For n heads, $m_i = 2^{-i/n \cdot \text{const}}$, following a geometric progression (a common choice: $m_i = 1/2^i$ for $i = 1, \ldots, n$).

9.10.1 Visual structure

The attention score matrix gets an added lower-triangular pattern:

$$m \cdot \begin{bmatrix} 0 & & & & \\ -1 & 0 & & & \\ -2 & -1 & 0 & & \\ -3 & -2 & -1 & 0 & \\ -4 & -3 & -2 & -1 & 0 \end{bmatrix}$$

The further back a key is, the more negative the bias, so $\exp(\cdot)$ pushes the softmax weight toward zero.

9.10.2 Why it acts like local windowed attention

Distant tokens get strongly negative bias (e.g., -128 for a 128-back token), so $e^{-128} \approx 0$. The model effectively only attends to recent tokens.

Different heads, different windows. Heads with large m have small effective windows ($m_i = 1$, sharp drop-off); heads with small m have wide effective windows ($m_i = 1/256$, gentle drop-off). Together they cover both short- and long-range context.

9.10.3 Why ALiBi generalises remarkably to longer sequences

Train on $T = 512$. Test on $T_{\text{valid}} = 16{,}000$.

- APE / sinusoidal: perplexity *explodes* for $T_{\text{valid}} > T$.
- RoPE: better than APE but still degrades.
- T5-bias: better than RoPE.
- **ALiBi**: *stable* perplexity even at $T_{\text{valid}} = 16\text{K}$ *when trained on only* $T = 512$.

Remember This

The ALiBi paradox. The simplest method (a hard-coded linear bias) extrapolates best. Why? Because the bias is *translation-invariant* (depends only on $i - j$) and *distance-monotone* (always penalises further tokens more) — exactly the inductive bias that helps a model handle unseen-length sequences.

Used by: BLOOM, MPT, Falcon (variants), Replit-Code, several others. Not as universally adopted as RoPE, but consistently strong on length-generalization.

Part E — NoPE: No Position Encoding At All

9.11 The Provocative Question

Intuition

For decoder-only models, the *causal mask* already encodes positional information implicitly: token i can only see tokens $\leq i$, so different positions see different prefix lengths. *Do we even need an explicit position encoding?*
The NoPE hypothesis (Kazemnejad et al., 2023): **No.**

9.11.1 Theoretical claims

1. Decoder-only Transformers with NoPE can *represent* both absolute and relative position information (the causal mask provides enough structure).
2. In practice, NoPE Transformers *learn* to use relative position representations.

9.11.2 Empirical performance

For perplexity-based length generalization, NoPE behaves competitively with RoPE and T5-bias — often better than APE. But on *structured* downstream tasks (sorting, arithmetic, reversing a list), all PE methods — including NoPE — continue to struggle when extrapolating well beyond the training length.

Quiz Trap

Trap. "NoPE means encoder-only models also don't need PE."
Reality. NoPE depends crucially on the *causal mask*. Without it, the model has no way to tell which token is which (self-attention is permutation equivariant). NoPE is a decoder-only phenomenon.

Part F — The Length Generalization Problem

9.12 What is Length Generalization?

Definition

Length generalization = the ability of a model trained on sequences of length up to T to maintain performance when applied to sequences of length $T' > T$ at inference, *without re-training*.

This is the critical metric for practical deployment. Re-training a model is expensive; ideally, we train once at moderate length and use the model at arbitrary lengths.

9.12.1 Empirical comparison (perplexity on Wikipedia-style text)

Pre-train at $T = 512$. Evaluate at $T_{\text{valid}} \in [512, 16{,}000]$.

Method	Behaviour at $T_{\text{valid}} \gg T$	Reason
APE (sinusoidal)	Severe degradation	Unseen position embeddings beyond T.
APE (learned)	Catastrophic failure	No learned embeddings beyond T.
RoPE	Significant degradation	Rotation angles extrapolate but interaction gets brittle.
T5-bias	Better than RoPE	Scalar bias generalises by interpolation.
ALiBi	Stable; minor degradation	Linear-monotone bias is translation-invariant.
NoPE	Comparable to RoPE/T5-bias	Causal mask provides implicit position.

9.12.2 The open challenge

Despite all this progress, no method *perfectly* extrapolates to arbitrary lengths on *structured* tasks like:

- Reversing a list of n items, where $n \gg n_{\text{train}}$.
- Adding two n-digit numbers.
- Tracking a counter across long contexts.

A 2024 study (*Position Coupling*, Cho et al.) argues there is an inherent **limitation in Transformer design** preventing perfect length generalization on algorithmic tasks. New approaches (Universal Transformers, Recurrent extensions) try to address this; the question remains open.

Research / Industry Insight

The most recent (2024) experimental wisdom for long-context LLMs:

- RoPE with a high θ base (e.g., $\theta_{\text{base}} = 10^6$, as in LLaMA-3) extrapolates better than the original $\theta_{\text{base}} = 10{,}000$.
- RoPE with *position interpolation* (PI) or *NTK-aware* scaling (Yarn, Code-LLaMA) extends context up to $4\times$ without re-training.
- ALiBi-based models tend to win on extreme length-generalization perplexity but lose on benchmarks at the trained length.

9.13 Summary Comparison Table

Method	Added to	Type	Learnable?	Used by
Sinusoidal APE	Input embedding	Absolute fixed	No	Vanilla Transformer
Learned APE	Input embedding	Absolute learned	Yes	BERT, GPT-1/2, BART, OPT
Shaw et al. RPE	Attention score	Relative learned	Yes	Few early models
T5-bias	Attention score	Relative learned scalar	Yes	T5, mT5
Transformer-XL	Attention score	Relative fixed + learnable	Both	Transformer-XL
RoPE	Q, K rotations	Relative fixed	No	LLaMA, PaLM, Mistral, GPT-J, Gemma
ALiBi	Attention score	Relative fixed linear	No	BLOOM, MPT, Falcon (variants)
NoPE	Nothing	—	N/A	Some research models

9.14 Common Mistakes & Quiz Traps — Week 12

Common Mistake

Mistake 1. "RoPE adds a positional vector to the embedding like sinusoidal APE."
Fix. RoPE *multiplies* (or rotates) the projected $W_Q x$ and $W_K x$ by rotation matrices. Nothing is added to the input embedding. This is a fundamental architectural distinction.

Common Mistake

Mistake 2. "ALiBi has zero learnable parameters."
Fix. ALiBi's biases are fixed (not learnable), but the head-specific slope m_i is a hyperparameter chosen by hand (geometric progression). No *back-propagation* touches it, but it is a choice that affects training.

Common Mistake

Mistake 3. "T5-bias adds learnable vectors to the attention score."
Fix. T5-bias adds a learnable *scalar* per relative distance bucket. A scalar, not a vector. This is much smaller than Shaw et al.'s d-dimensional embedding.

Quiz Trap

Trap. "Larger context length always requires re-training the model."
Reality. For RoPE models, techniques like Position Interpolation (PI), NTK-aware scaling, or Yarn can extend context by 2–4× *without re-training*. For ALiBi models, extrapolation comes for free. Only APE-based models need re-training to handle longer contexts.

Quiz Trap

Trap. "NoPE means the model is permutation-invariant."
Reality. For an encoder-only model, yes. For a *decoder-only* model with a causal mask, no — the causal mask itself breaks permutation invariance, allowing NoPE to work.

9.15 Memory Tricks — Week 12

Memory Trick

"APE adds vectors. RoPE rotates them. ALiBi adds scalars. NoPE does nothing."
The four families in one line.

Memory Trick

"RoPE: 2D rotation per dimension pair."
Each adjacent pair $(2i, 2i+1)$ gets its own rotation angle $m\theta_i$, with $\theta_i = 10000^{-2(i-1)/d}$.

Memory Trick

"ALiBi: head i has slope $1/2^i$."
8 heads $\rightarrow$ slopes $1/2, 1/4, 1/8, \ldots, 1/256$. Sharp heads = local; gentle heads = global.

Memory Trick

"Inner product invariance: $\langle q_m, k_n \rangle$ depends only on $n - m$."
The core mathematical requirement that RoPE satisfies by construction.

9.16 Quick Revision — Week 12 in One Page

Quick Revision

The two problems with APE.
- *Shift sensitivity:* model predictions change if you shift the input by k.
- *No length extrapolation:* no embeddings learned for positions beyond T.

RPE families.
- **Shaw et al. (2018):** learnable vector $p_{ij}^K \in \mathbb{R}^d$ added in attention score, clipped at distance $k = 16$.
- **Transformer-XL:** decompose attention into 4 terms; use sinusoidal R_{j-i}.
- **T5-bias:** learnable scalar $r_{j-i} \in \mathbb{R}$ per relative distance.

RoPE (Rotary). Multiply $W_Q x_m$ by rotation $R_{m\theta}$, $W_K x_n$ by $R_{n\theta}$. Inner product becomes $x_m^\top W_Q^\top R_{(n-m)\theta} W_K x_n$ — relative-position only. Applied at every layer; no extra parameters. Dominant in modern LLMs.

ALiBi. Add $m \cdot (-(i-1), \ldots, -1, 0)$ to attention scores. m_i geometric per head $(1/2^i)$. Distant tokens are suppressed; effectively a learnable local window. **Best extrapolation properties.**

NoPE. Decoder-only models work without any explicit PE thanks to the causal mask. Competitive with RoPE / T5-bias on perplexity.

Length generalization ranking (perplexity, $T_{\text{train}} = 512$, $T_{\text{valid}} = 16$K).

$$\text{ALiBi} > \text{T5-bias} > \text{NoPE} \approx \text{RoPE} > \text{APE}.$$

What modern LLMs use.
- LLaMA-1/2/3, PaLM, Mistral, Gemma, GPT-J, GPT-NeoX, Qwen, Yi → RoPE.
- BLOOM, MPT, some Falcon variants → ALiBi.
- BERT, GPT-1/2, BART, OPT → learned APE (legacy).

9.17 Practice Questions — Week 12

Multiple Choice

1. **(Q1)** The primary limitation of learned Absolute Position Encoding (APE) that motivates RPE is:

   (A) it is computationally too expensive,

   (B) it cannot extrapolate beyond the training context length,

   (C) it requires more parameters than RPE,

   (D) it is incompatible with the softmax function.

   Solution. (B). Positions beyond T never received gradient updates during training.

2. **(Q2)** RoPE encodes position by:

   (A) adding a sinusoidal vector to the input embedding,

   (B) rotating $W_Q x$ and $W_K x$ by an angle proportional to position,

   (C) adding a scalar to the attention score,

   (D) training a separate position embedding matrix.

   Solution. (B).

3. **(Q3)** ALiBi achieves length generalization because:

   (A) it learns a separate embedding for each context length,

   (B) it ignores the position completely,

   (C) its bias is translation-invariant and distance-monotone,

   (D) it uses sinusoidal frequencies.

   Solution. (C).

4. **(Q4)** The T5-bias position encoding adds:

   (A) a vector $p \in \mathbb{R}^d$ to the input embedding,

   (B) a learnable matrix to each head,

   (C) a learnable scalar to each pair (i, j) in the attention score, bucketed by distance,

   (D) a sinusoidal value to the value vector.

   Solution. (C).

5. **(Q5)** For an 8-head ALiBi model, the slopes m_i typically:

   (A) are all equal,

   (B) are sampled randomly,

   (C) follow a geometric progression like $1/2, 1/4, 1/8, \ldots$,

   (D) are learned via backpropagation.

   Solution. (C).

6. **(Q6)** NoPE (no position encoding) can work for:

   (A) encoder-only models,

   (B) decoder-only models with a causal mask,

   (C) encoder-decoder models,

   (D) only the topmost layer.

   Solution. (B). The causal mask provides positional structure.

Conceptual / Short Answer

7. **(Q7)** Why does the shift-sensitivity experiment (shifting the input by k positions) reveal that APE has not learned relative-position behaviour?

   Answer. If the model had learned to attend purely relatively, shifting all positions by a constant k would not change pairwise distances, and hence not change attention weights or predictions. The fact that performance degrades with k shows that APE genuinely encodes *absolute* positions, and the model has come to rely on this absolute information.

8. **(Q8)** Derive in one line why $\langle f_q(x_m, m), f_k(x_n, n)\rangle$ depends only on $(m - n)$ in RoPE.

   Answer. With $f_q = e^{jm\theta} W_Q x_m$ and $f_k = e^{jn\theta} W_K x_n$:

$$\langle f_q, f_k\rangle = (x_m W_Q)(x_n W_K)^\top \, e^{jm\theta}\, \overline{e^{jn\theta}} = x_m W_Q W_K^\top x_n^\top \, e^{j(m-n)\theta},$$

   which depends only on $(m - n)$.

9. **(Q9)** Why does ALiBi naturally behave like local windowed attention?

   Answer. The bias is $m \cdot (-(i-1), \ldots, -1, 0)$, so a key k positions back has bias $-mk$. Inside softmax, this is e^{-mk}, which goes to zero for large k. Hence tokens far in the past contribute negligibly to attention, just like windowed attention — but it is *soft*, not hard.

10. **(Q10)** Sketch in one sentence why RoPE has no learnable position parameters yet still works well.

    Answer. The rotation frequencies $\theta_i = 10000^{-2(i-1)/d}$ are fixed by design, but the *model parameters* W_Q, W_K are learned and can adapt how the rotated queries and keys interact — so the position information enters the model purely through the rotation, but the model still has full flexibility to use it.

11. **(Q11)** Suppose you have a RoPE-based LLaMA-2 with training context 4K and you want to extend to 32K. Why is this easier than for a GPT-2 with learned APE?

    Answer. RoPE's rotation angles smoothly extrapolate: a position $m = 30{,}000$ just gives a rotation angle $30{,}000 \cdot \theta_i$, which is a valid computation. The model may still lose some accuracy because it hasn't *seen* those rotations, but the computation is well-defined. Learned APE has no embedding for position 30,000 at all — it's a hard wall.

Numerical

12. **(Q12)** For RoPE with $d = 4$ and the standard frequency formula, compute θ_1 and θ_2.

    Solution. $\theta_1 = 10000^{-2(1-1)/4} = 10000^0 = 1$. $\theta_2 = 10000^{-2(2-1)/4} = 10000^{-1/2} = 1/100 = 0.01$. Lower dimension index = higher frequency = faster oscillation with position.

13. **(Q13)** For ALiBi with 8 heads using slopes $m_i = 1/2^i$, what is the effective bias for a key 10 tokens back in head $i = 1$ vs head $i = 8$?

    Solution. Head 1: $m_1 = 1/2$, bias $= -10/2 = -5$. Head 8: $m_8 = 1/256$, bias $= -10/256 \approx -0.04$. **Head 1 strongly suppresses the 10-back token; head 8 barely notices the distance.** This is exactly the multi-resolution behaviour ALiBi achieves across heads.

14. **(Q14)** A T5-bias model has max distance $k = 128$ and 12 heads, 24 layers. How many learnable scalars are added by the bias scheme? (Note: bias is shared across layers in T5.)

    Solution. $2k + 1 = 257$ distances, $\times 12$ heads (per-head bias) $= 3084$ scalars total. Compare with APE: $T \cdot d_{\text{model}}$ which is typically $\sim 10^5$.

15. **(Q15)** A RoPE-based model with $d_{\text{head}} = 64$. How many distinct rotation frequencies are used per head?

    Solution. $d_{\text{head}}/2 = 32$ frequencies $\theta_1, \ldots, \theta_{32}$. Each frequency rotates one pair of adjacent dimensions.

Closing Remarks — The End of the E-book

Congratulations. You have reached the end of the course.
This e-book has taken you from the seq2seq + attention mechanisms of 2015 (Week 1) to the cutting edge of position-encoding research in 2024 (Week 12). Along the way you have built up:

- The full Transformer architecture — encoder, decoder, attention, FFN, LayerNorm, residual connections, positional encoding (Weeks 1–2).
- The pre-train + fine-tune paradigm (Week 3, GPT-1).
- Decoding strategies for generation and the BERT family for understanding (Week 4).
- Tokenizers — BPE, WordPiece, SentencePiece (Week 5).
- Encoder-decoder LLMs — BART, GPT-2, T5 with its systematic study (Week 7).
- The LLM landscape, scaling laws, and pre-training data pipelines (Week 8).
- Long-context techniques — sparse attention, Flash Attention, KV-caching, GQA, PagedAttention (Week 11).
- Position encodings — APE, RPE, RoPE, ALiBi, NoPE — and the length generalization challenge (Week 12).

What's next? The frontier moves quickly. Stay close to the literature on:

- Alignment: RLHF, DPO, Constitutional AI, ORPO.
- Multimodal: GPT-4V, LLaVA, Gemini's native multimodality.
- Reasoning: Chain-of-thought, o1-style inference-time scaling.
- Agents: ReAct, tool-use, web browsing, code execution.
- Efficient training: ZeRO, FSDP, sequence parallelism.
- Inference systems: vLLM, TensorRT-LLM, exllamav2.

The principles in this e-book — attention as soft retrieval, scaling laws, denoising objectives, IO-aware implementations, relative-position invariance — will keep paying dividends as new architectures emerge. Good luck.

Made by Anmol Kansal with AI.
